



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech



JavaScript Basics

APRIL, 2024

Scripting languages

- **Scripting languages** are programming languages that a program called an interpreter translates into a code that a computer program can understand.
- It provides instructions, called scripts, for software in apps and websites.
- Scripts contain a series of commands that a software, application or scripting engine interprets one at a time within a **runtime environment**.
- They are **interpreted** rather than compiling.

Examples: JavaScript, PHP, Python, VBScript, etc.,

JavaScript : Introduction

- JavaScript often abbreviated as JS, is a programming language and core technology of the World Wide Web, alongside HTML and CSS.
- JavaScript is an **object-based, lightweight** and **interpreted** programming language.
- JavaScript allows you to **add interactivity** to a web page.
- It is **open source** and **cross-platform**.
- As of 2024, **98.9%** of websites use JavaScript on the client side for webpage behavior, often incorporating third-party libraries.
- All major web browsers have a dedicated **JavaScript engine** to execute the code on users' devices.



JavaScript: Introduction

- JavaScript is a multi-paradigm, dynamic language with types and operators, standard built-in objects, and methods.
- Its syntax is based on the Java and C languages — many structures from those languages apply to JavaScript as well.
- JavaScript is versatile and beginner-friendly.
- JavaScript itself is relatively compact, yet very flexible.

**The Man behind JavaScript:
Brendan Eich**



JavaScript: Introduction

- Core JavaScript can be extended for a variety of purposes by supplementing it with additional objects;

For Example,
 - **Client-side JavaScript** extends the core language by supplying objects to control a browser and its Document Object Model (DOM).
 - **Server-side JavaScript** extends the core language by supplying objects relevant to running JavaScript on a server.
 - JavaScript is standardized at **Ecma International**.

Javascript : Introduction

- **Client-side JavaScript:**
 - Client-side JavaScript is the most **common form** of the language.
 - To be interpreted by the browser, the **script must be included** in or linked from an HTML document.
 - It means that a web page can now incorporate programs that interact with the user, control the browser, and dynamically create HTML content, rather than just static HTML.
 - **For ex. Use JavaScript to verify that a user has provided a valid email address in a form field.**
 - When the user submits the form, the Script code gets executed, and the form is submitted to the Web Server **if all of the entries are valid as per the script.**

Javascript : Advantages

- **Less server interaction** –Before transmitting the page to the server, we can validate the user's input.
This reduces server traffic, resulting in a lower load on your server.
- **Immediate feedback to the visitors** –They don't have to wait for the page to reload to see if they lost something.
- **Increased interactivity** –We can make interfaces that respond when the user moves their mouse over them or activates them with the keyboard.
- **Richer interfaces** – To provide a Rich Interface to your site users, we can utilize JavaScript to incorporate features like drag-and-drop components and sliders.

Javascript : Advantages

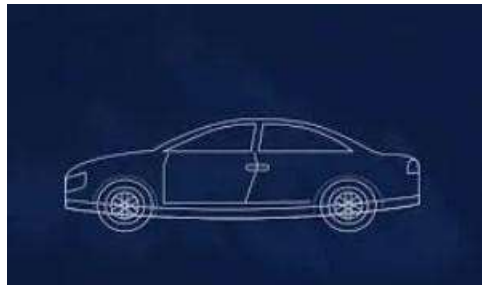
- **Simplicity** - JavaScript is easy to understand and learn.
- **Interoperability** - We can embed it into any webpage or inside the script of another programming language.
- **Versatility** - JavaScript can now be used for both front-end and back-end development.
 - Ex: NodeJS, AngularJS, ReactJS, and many more.
- **Less Overhead**
 - By reducing the code length, JavaScript enhances the efficiency of websites and web apps.
 - The usage of numerous built-in methods reduces the amount of overhead in the code.

Javascript : Advantages

- **Speed** - Since JavaScript is an 'interpreted' language, it reduces the time required
- **Popularity** - Since all modern browsers support JavaScript, it is seen almost everywhere.
- **Extended Functionality** - Third-party add-ons allow the developers to add snippets of predefined code in their code to save time and money.
- **Asynchronous Programming** – This model facilitates non-blocking operations, making it well-suited for tasks that involve fetching data from servers, handling user input, and managing concurrent processes.

HTML Vs CSS Vs JavaScript

HTML (Structure)



The actual content of
the page

HTML + CSS (Presentation)



The look of the page
like color, style

HTML + CSS + JavaScript (Functionality)



Easily control and alter
the HTML

Javascript : A heed

- **File reading and writing on the client side are not supported by JavaScript.**
 - For security reasons, this has been preserved.
- JavaScript doesn't have any **multi-threading** or **multiprocessor** capabilities.
- JavaScript only **allows for a single inheritance, not multiple inheritance.**
- This object-oriented language feature may be required by some programs.
- A single code error can **cause the entire JavaScript code on the page to stop rendering.**

Javascript : Syntax

- A JavaScript consists of JavaScript statements that are placed within the **<script>... </script>** HTML tags in a web page.
- The **<script>** element, which contains JavaScript, can be placed anywhere on the page, however it is advised to put it within the **<head>** tags.
- The **<script>** tag alerts the browser program to begin interpreting all the text between these tags as a script.
- **Syntax of JavaScript** will be as follows:

```
<script>
```

```
    Javascript Code
```

```
</script>
```


First JavaScript

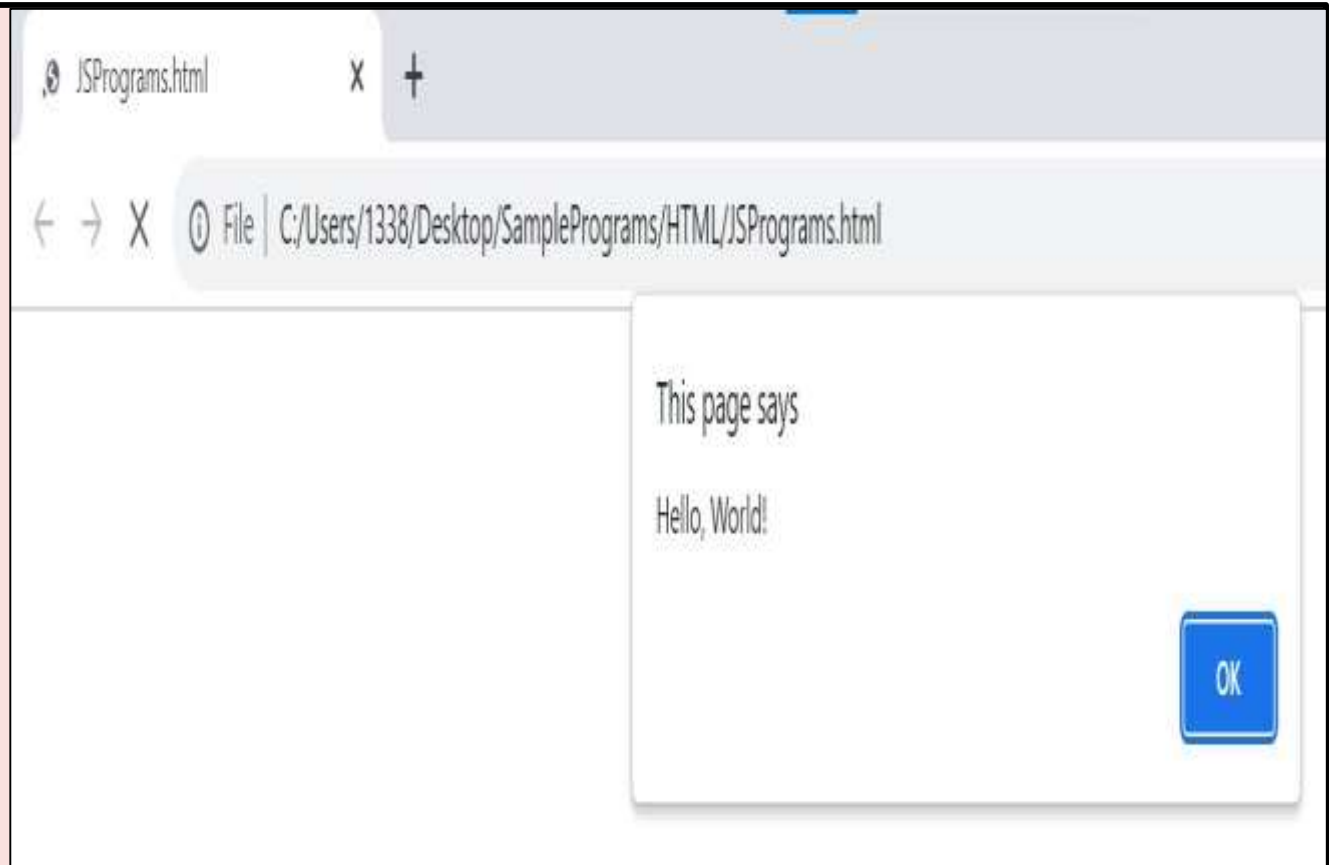
- The script tag takes **two important** attributes:
- **language:**
 - The scripting language you're using is specified by this attribute.
 - The most common value is javascript.
 - The use of this feature has been phased out in current versions of HTML (and its successor, XHTML).
- **type:**
 - The value of this attribute should be set to "**application/javascript**" or "**text/javascript**" in order to identify the scripting language in use.

JavaScript Basics

First JavaScript

- The JavaScript code in the **<script>** element is interpreted from **top to bottom**.

```
<!DOCTYPE html>
<html>
<head>
<script type="application/javascript">
    alert('Hello, World!');
</script>
</head>
<body>
</body>
</html>
```



JavaScript Placement in HTML File

- There is a flexibility given to include **JavaScript code anywhere** in an HTML document. But there are following **most preferred ways** to include JavaScript in your HTML file.
- Script in **<head>...</head> section**: If you want to have a **script run on some event**, such as when a user clicks somewhere, then you will place that script in the head.
- Script in **<body>...</body> section**: If you need a **script to run as the page loads** so that the script generates content in the page, the script goes in the <body> portion of the document.
- Script in **<body>...</body> and <head>...</head> sections**.
- Script in and **external file** and then include in <head>...</head> section.

Javascript : First example program

JavaScript in External File:

- The script tag provides a mechanism to allow you to **store JavaScript in an external file** and then include it in your HTML files.
- To use JavaScript from an external file source, you need to write all JavaScript source code in a simple text **file with the extension ".js"**.

```
//filename.js
```

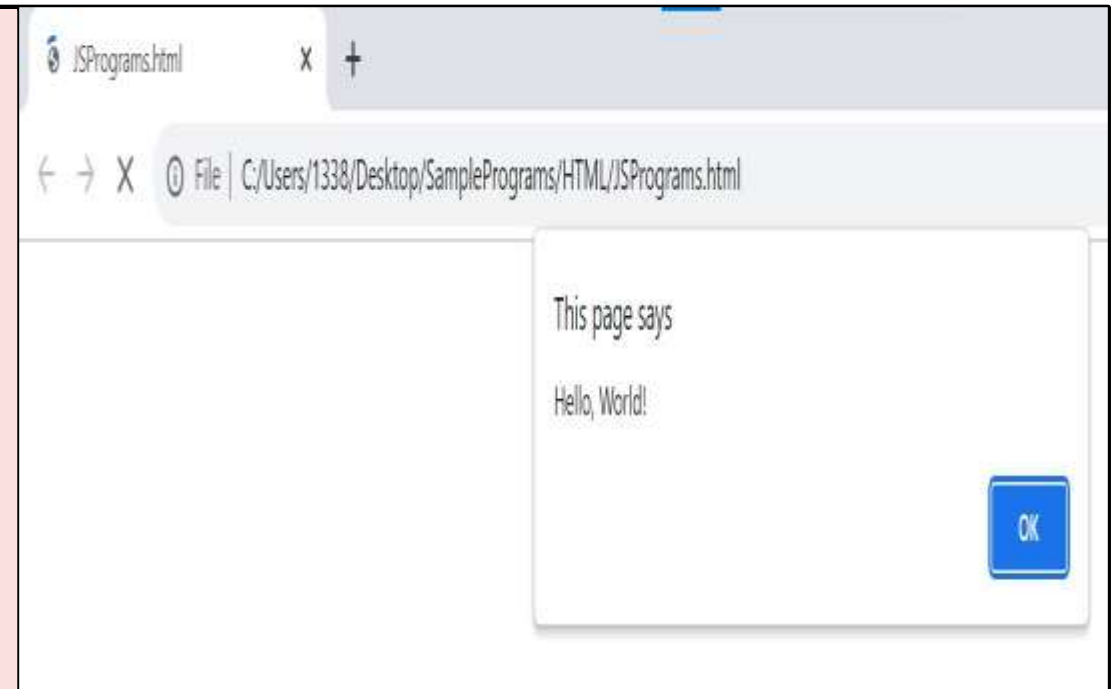
```
alert('Hello, World!');
```

Javascript : First example program

JavaScript in External File:

- Then include in the html file using the `<script>` element.

```
<!DOCTYPE html>
<html>
<head>
  <script type="text/javascript" src="filename.js" ></script>
</head>
<body>
</body>
</html>
```



JavaScript Basics

Basic Elements

JavaScript is case-sensitive

- Everything in JavaScript is case-sensitive, including variables, function names, class names, and operators.
- It indicates that the counter and Counter variables are not the same.

Identifiers

- An identifier is the name of a variable, function, parameter, or class.
- An identifier consists of one or more characters in the following format:
 - The first character must be a letter (a-z, or A-Z), an underscore(_), or a dollar sign (\$).
 - The other characters can be letters (a-z, A-Z), numbers (0-9), underscores (_), and dollar signs (\$).

Basic Elements

Comments

- JavaScript supports both single-line and block comments.
- A single-line comment starts with two forward-slash characters (//).

```
//This is a single-line comment
```

- A block comment starts with forward-slash(/) and asterisk(*) characters and ends with asterisk(*) and forward-slash(/) characters.

```
/*  
This is a block comment that can span multiple lines  
*/
```


Basic Elements

Statements

- JavaScript **does not require** to end a statement with a **semicolon (;)**
- It is **recommended** to always use the semicolon to end a statement.

```
var a = 10;
```

```
var b = 20;
```

JavaScript Basics

Basic Elements

- We can use a code block that begins with a left curly brace ({) and ends with the right curly brace (}) to combine multiple statements as follows:

```
if( a > b)
{
  console.log('a is greater than b');
  return 1;
}
```

Expressions

- An expression is a piece of code that evaluates to a value.

For example:

```
c=2 + 1
```

- The above expression assigns 3 to c.

Basic Elements

Keywords & Reserved words

- JavaScript defines a list of keywords and reserved words that have special uses.
- We cannot use the keywords and reserved words as the identifiers.

abstract	arguments	await	boolean	break	byte	case	catch
char	class	const	continue	debugger	default	delete	do
double	else	enum	eval	export	extends	false	final
finally	float	for	function	goto	if	implements	import
in	instanceof	int	interface	let	long	native	new
null	package	private	protected	public	return	short	static
super	switch	synchronized	this	throw	throws	transient	true
try	typeof	var	void	volatile	while	with	yield

JavaScript Basics

Basic Elements: Variables

- Variables are containers for storing data (values).
- There are **3 ways** to declare a JavaScript variable:
 - var
 - let
 - const

Basic Elements: Variables

Using var

- Creating a variable in JavaScript is called "**declaring**" a **variable**.
- After the declaration, the variable has no value (technically it has the value of undefined).
- To assign a value to the variable using the **assignment operator**.

```
var message;  
message = "Hello";
```

- You can also assign a value to the variable when you declare it:

```
var message = "Hello";
```

Basic Elements: Variables

- You can declare many variables in one statement. Start the statement with `var` and separate the variables by **comma**:

```
var FirstName= "Sachin", LastName = "Tendulkar";
```

Note:

Using `var`, we can redeclare variables

Basic Elements: Variables

- The **general rules** for constructing names for variables (unique identifiers) are:
 - Names can contain letters, digits, underscores, and dollar signs
 - Names must begin with a letter
 - Names can also begin with \$ and _
 - Names are case sensitive (y and Y are different variables)
 - Reserved words (like JavaScript keywords) cannot be used as names

Basic Elements: Variables

Using let

- The let keyword was introduced in ES6 (2015).
- Variables defined with let **cannot be redeclared**.
- Variables defined with let **must be declared before use**.
- Variables defined with let **have block scope**.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    let x = "Sachin";
    let x = 0;

    document.write(x);
  </script>
</body>
</html>
```

Output:

Uncaught SyntaxError SyntaxError: Identifier 'x' has already been declared

Basic Elements: Variables

- With var we can:

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    var x = "Sachin";
    var x = 0;
    document.write(x);
  </script>
</body>
</html>
```

Output:

0

Basic Elements: Variables

Block Scope

- ES6 introduced two important new JavaScript keywords: **let** and **const**.
- These two keywords provide **block scope** in JavaScript.
- Variables declared inside a `{ }` block cannot be accessed from outside the block:

```
{  
  let x = 2;  
}  
  
// x cannot be used here
```

```
{  
  var x = 2;  
}  
  
// x can be used here
```

Basic Elements: Variables

Using const

- The const keyword was introduced in **ES6 (2015)**.
- Variables defined with const **cannot be redeclared**.
- Variables defined with const **cannot be reassigned**.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    const PI = 3.141592653589793;
    PI = 3.14;
    PI = PI + 10;
    document.write(PI);
  </script>
</body>
</html>
```

Output:

Uncaught TypeError TypeError: Assignment to constant variable.

Basic Elements: Variables

Using const

```
<!DOCTYPE html>

<html>

<body>
  <script type="text/javascript">
    const PI = 3.141592653589793;

    document.write(PI);
  </script>
</body>

</html>
```

Output:

3.141592653589793

Basic Elements: Variables

Using const

```
<!DOCTYPE html>

<html>
<body>

  <script type="text/javascript">
    const PI ;
    PI = 3.141592653589793;
    document.write(PI);
  </script>

</body>
</html>
```

Output:

Uncaught SyntaxError SyntaxError: Missing initializer in const declaration

Basic Elements: Variables

Block Scope

- Declaring a variable with `const` is **similar to `let`** when it comes to Block Scope.
- **For Example:** The `x` declared in the block is not the same as the `x` declared outside the block

```
const x = 10; // Here x is 10
{
    const x = 2; // Here x is 2
}
// Here x is 10
```

Basic Elements : Const

Redeclaring

- Redeclaring a JavaScript **var** variable is **allowed** anywhere in a program:

```
var x = 2;    // Allowed  
x = 4;       // Allowed
```

- Redeclaring an existing **var** or **let** variable to **const**, in the same scope, is **not allowed**:

```
var x = 2;    // Allowed  
const x = 2;  // Not allowed  
{  
let x = 2;    // Allowed  
const x = 2;  // Not allowed  
}  
{  
const x = 2;  // Allowed  
const x = 2;  // Not allowed  
}
```

Basic Elements : Operators

Arithmetic Operators:

- Arithmetic operators are used to performing arithmetic on numbers

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Basic Elements : Operators

Assignment Operators:

- Assignment operators assign a value to a variable.

Operator	Example	Same As
=	<code>x = y</code>	<code>x = y</code>
+=	<code>x += y</code>	<code>x = x + y</code>
-=	<code>x -= y</code>	<code>x = x - y</code>
*=	<code>x *= y</code>	<code>x = x * y</code>
/=	<code>x /= y</code>	<code>x = x / y</code>
%=	<code>x %= y</code>	<code>x = x % y</code>
**=	<code>x **= y</code>	<code>x = x ** y</code>

Basic Elements : Operators

Comparison Operators:

- Comparison operators compare the two operands.

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Basic Elements : Operators

Logical Operators:

- The logical operators are used to combine two or more conditions.

Operator	Description
&&	logical and
	logical or
!	logical not

Basic Elements : Operators

Bitwise Operators:

- The bitwise operators perform bitwise operations on operands.

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0001	0001	1
	OR	5 1	0101 0001	0101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0001	0100	4
<<	Zero fill left shift	5 << 1	0101 << 1	1010	10
>>	Signed right shift	5 >> 1	0101 >> 1	0010	2
>>>	Zero fill right shift	5 >>> 1	0101 >>> 1	0010	2

Basic Elements : Operators

Special Operators:

Operator	Description
(?:)	Conditional Operator returns value based on the condition. It is like if-else.
,	Comma Operator allows multiple expressions to be evaluated as single statement.
delete	Delete Operator deletes a property from the object.
in	In Operator checks if object has the given property
instanceof	checks if the object is an instance of given type
new	creates an instance (object)
typeof	checks the type of object.
void	it discards the expression's return value.
yield	checks what is returned in a generator by the generator's iterator.

Basic Elements : Data Types

- JavaScript variables can hold different data types: **numbers, strings, objects**, and more:

```
let length = 16;           // Number
let lastName = "Johnson"; // String
let x = {firstName:"Rahul", lastName:"Dravid"}; // Object
```

- JavaScript Types are Dynamic**

- JavaScript has dynamic types. i.e. the same variable can be used to hold different data types.

```
let x;           // Now x is undefined
x = 5;           // Now x is a Number
x = "John";      // Now x is a String
```

Basic Elements : Data Types

Strings

- A string (or a text string) is a **series of characters** like "Sachin".
- Strings are written with quotes. **single or double quotes** can be used.
- We can use quotes inside a string, as long as they don't match the quotes surrounding the string:

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    let Name1 = "Sachin"; // Using double quotes
    let Name2 = 'Sachin'; // Using single quotes

    let str1 = "It's alright"; // Single quote inside double quotes
    let str2 = "He is called 'Johnny'"; // Single quotes inside double quotes
    let str3 = 'He is called "Johnny"'; // Double quotes inside single quotes
```

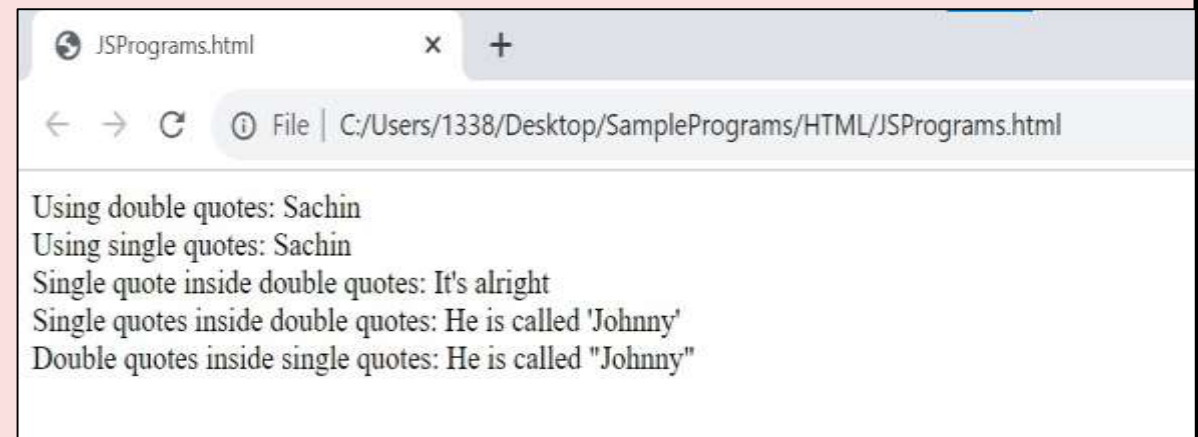
Basic Elements : Data Types

```
document.write("Using double quotes: "+Name1+"<br/>");  
document.write("Using single quotes: "+Name2+"<br/>");  
document.write("Single quote inside double quotes: "+str1+"<br/>");  
document.write("Single quotes inside double quotes: "+str2+"<br/>");  
document.write("Double quotes inside single quotes: "+str3);
```

```
</script>
```

```
</body>
```

```
</html>
```



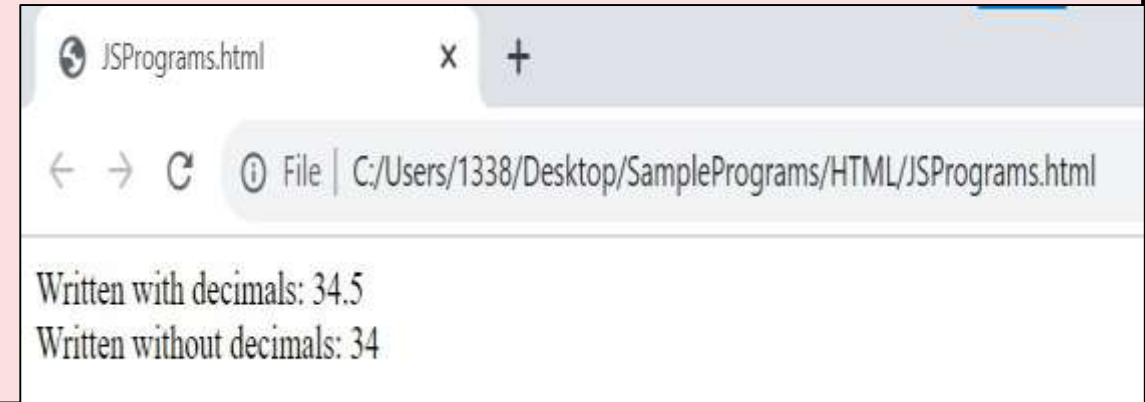
Basic Elements : Data Types

Numbers

- Numbers can be written **with, or without** decimals.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    let x1 = 34.50;  // Written with decimals
    let x2 = 34;     // Written without decimals

    document.write("Written with decimals: "+x1+"<br/>");
    document.write("Written without decimals: "+x2);
  </script>
</body>
</html>
```



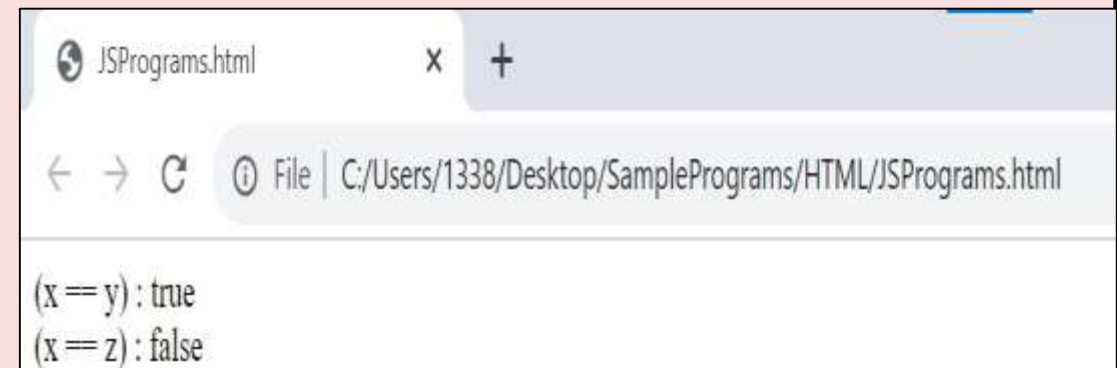
Basic Elements : Data Types

Booleans

- Booleans can only have two values: **true** or **false**.

```
<!DOCTYPE html>
<html>

<body>
  <script type="text/javascript">
    let x = 5;
    let y = 5;
    let z = 6;
    document.write("(x == y) : "+(x == y) + "<br/>");
    document.write("(x == z) : "+(x == z) );
  </script>
</body>
</html>
```

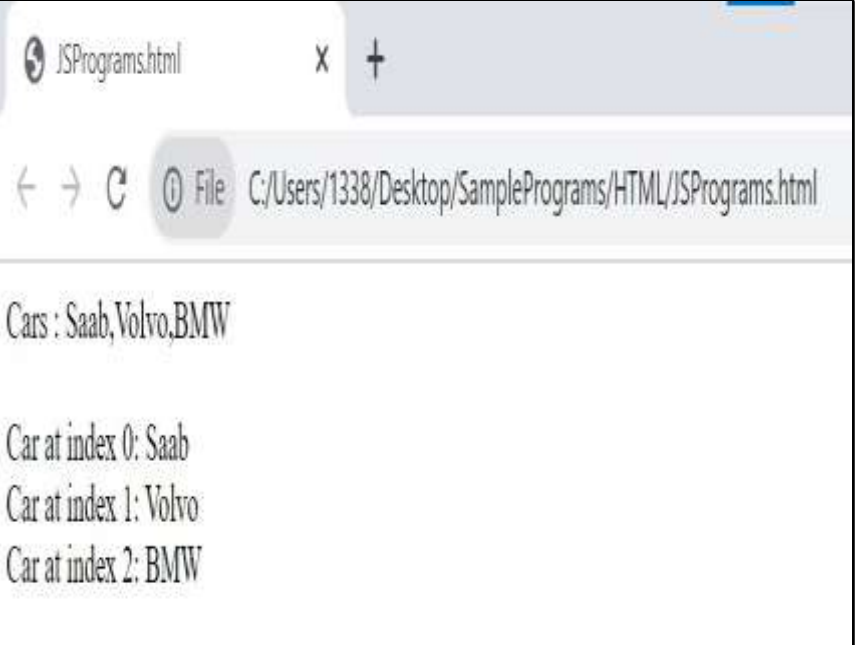


Basic Elements : Data Types

Arrays

- JavaScript arrays are written with square brackets.
- Array items are separated by commas.
- Array indexes are **zero-based**, which means the first item is [0], the second is [1], and so on.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    const cars = ["Saab", "Volvo", "BMW"];
    document.write("Cars : "+cars + "<br/><br/>");
    for (let i = 0; i < cars.length; i++) {
      document.write("Car at index " + i + ": " + cars[i] + "<br/>"); }
  </script>
</body>
</html>
```



Basic Elements : Data Types

Objects

- JavaScript objects are written with curly braces {}.
- Object properties are written as **name: value pairs**, separated by commas.

```
<!DOCTYPE html>

<html>
<body>

  <script type="text/javascript">
    const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};

    document.write(person.firstName + " " + person.lastName + "<br/>" + person.age + "<br/>" + person.eyeColor);

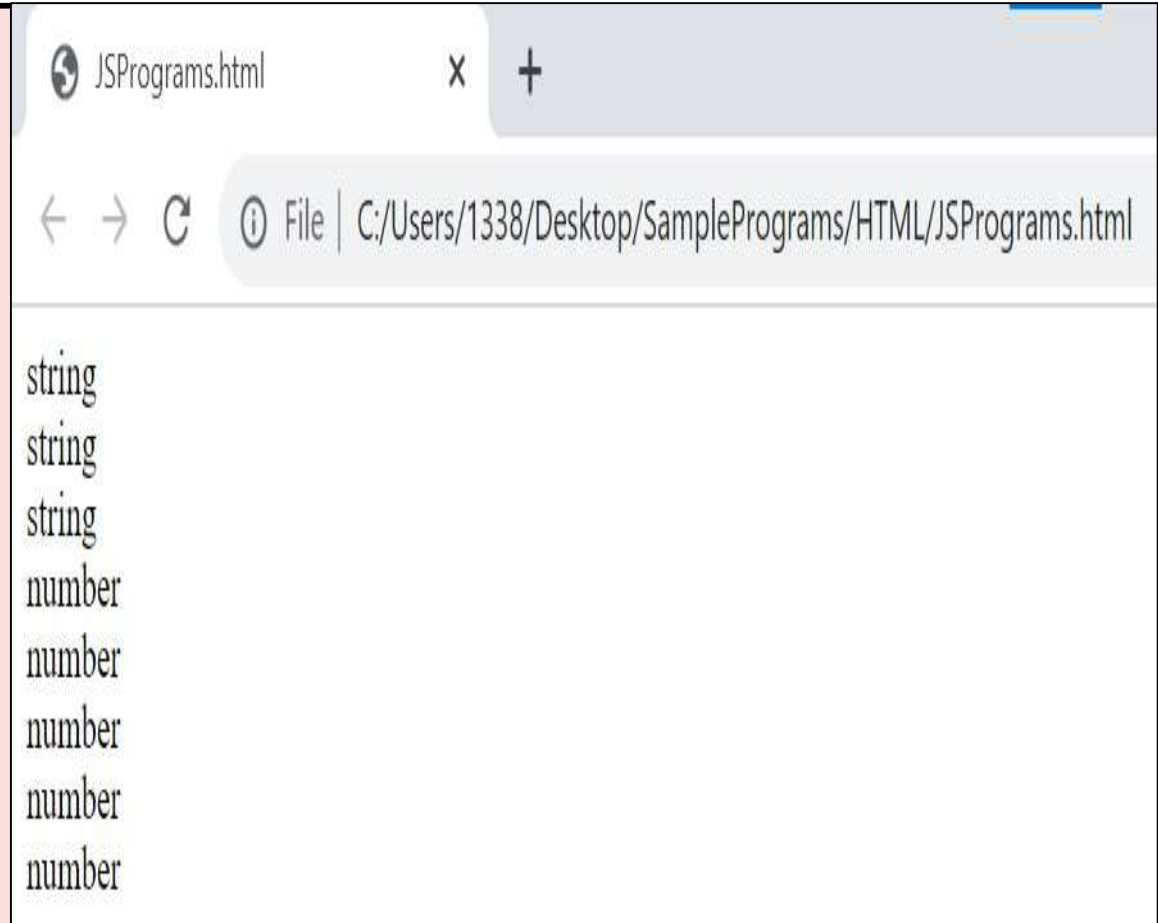
  </script>
</body>
</html>
```



Basic Elements : typeof Operator

- **typeof operator** can be used to find the type of a JavaScript variable.
- The **typeof operator** returns the type of a **variable** or an **expression**.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    document.write( typeof "" + "<br/>" );
    document.write( typeof "Sachin" + "<br/>" );
    document.write( typeof "Sachin Tendulkar" + "<br/>" );
    document.write( typeof 0 + "<br/>" );
    document.write( typeof 314 + "<br/>" );
    document.write( typeof 3.14 + "<br/>" );
    document.write( typeof 3 + "<br/>" );
    document.write( typeof (3+4) );
  </script>
</body>
</html>
```



Basic Elements : Output

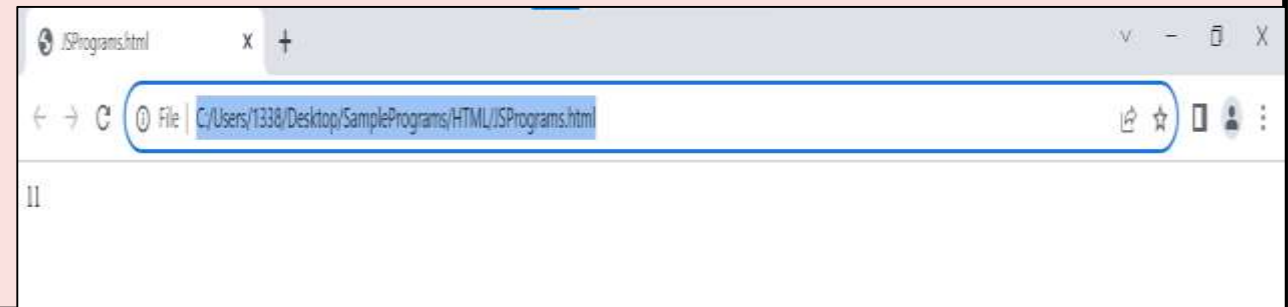
- JavaScript can "display" data in **different ways**:
 - Writing into the HTML output using **document.write()**
 - Writing into an alert box, using **window.alert()**
 - Writing into the browser console, using **console.log()**
 - Writing into an HTML element, using **innerHTML**. (**Will Discuss Later**)

Basic Elements : Output

Using document.write()

- For testing purposes, it is convenient to use document.write():

```
<!DOCTYPE html>
<html>
<body>
<script>
document.write(5 + 6);
</script>
</body>
</html>
```



Basic Elements : Output

Using window.alert()

- You can use an alert box to display data
- The window object is the global scope object, that means that variables, properties, and methods by default belong to the window object.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

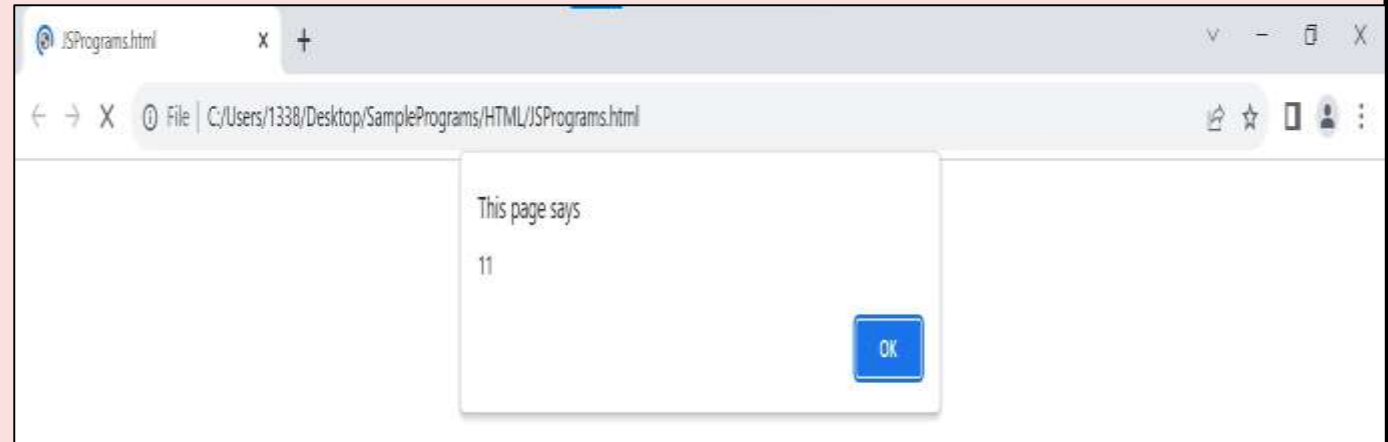
```
<script>
```

```
window.alert(5 + 6);
```

```
</script>
```

```
</body>
```

```
</html>
```



Basic Elements : Output

Using console.log()

- For debugging purposes, you can call the **console.log()** method in the browser to display data.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

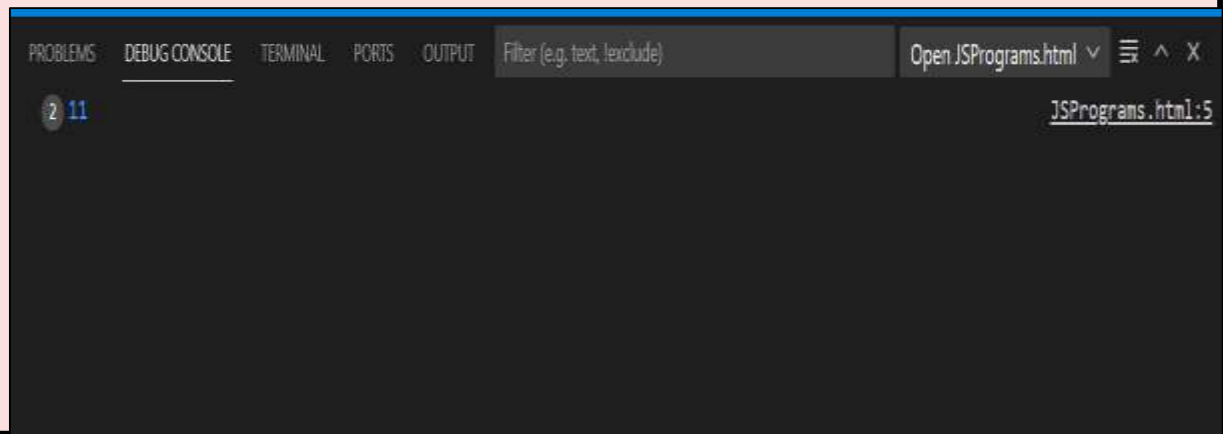
```
<script>
```

```
console.log(5 + 6);
```

```
</script>
```

```
</body>
```

```
</html>
```

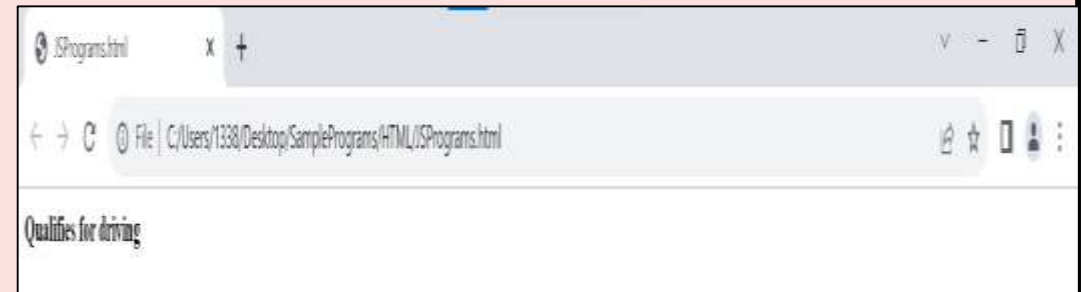


Basic Elements : Control Flow Statements

if Statement

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

```
<script type="application/javascript">  
var age = 20;  
if( age > 18 ){  
    document.write("<b>Qualifies for driving</b>");  
}  
</script>
```



Basic Elements : Control Flow Statements

if-else Statement

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

```
<script type="application/javascript">  
var age = 15;  
if( age > 18 ){  
    document.write("<b>Qualifies for driving</b>");  
}else{  
    document.write("<b>Does not qualify for driving</b>");  
}  
</script>
```



Basic Elements : Control Flow Statements

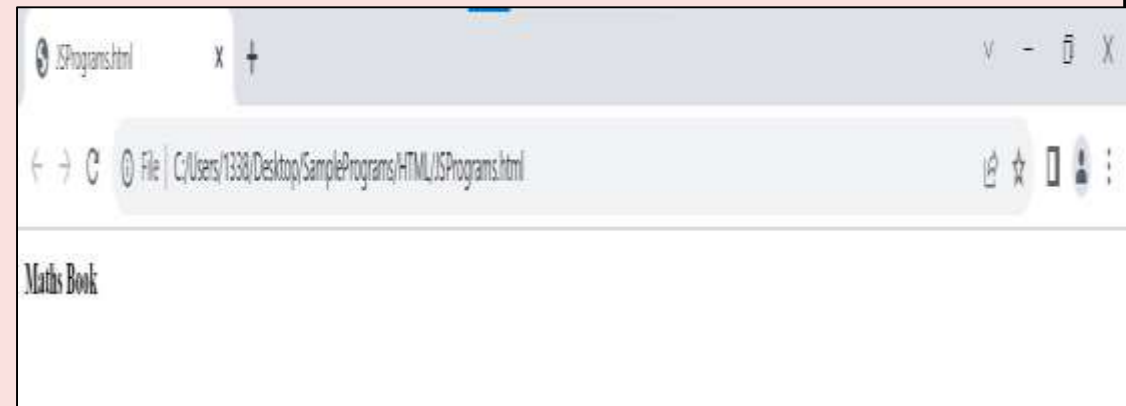
if...else if... statement:

```
if (condition1) {  
  
    // block of code to be executed if condition1 is true  
  
    } else if (condition2) {  
  
    // block of code to be executed if the condition1 is false and condition2 is true  
  
    } else {  
  
    // block of code to be executed if the condition1 is false and condition2 is false  
  
    }
```

Basic Elements : Control Flow Statements

if...else if... statement:

```
<script type="application/javascript">
var book = "maths";
if( book == "history" ){
    document.write("<b>History Book</b>");
}else if( book == "maths" ){
    document.write("<b>Maths Book</b>");
}else if( book == "economics" ){
    document.write("<b>Economics Book</b>");
}else{
    document.write("<b>Unknown Book</b>");
}
</script>
```



Basic Elements : Control Flow Statements

switch-case Statement

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```

Basic Elements : Control Flow Statements

Switch-case Statement

```
<script type="application/javascript">

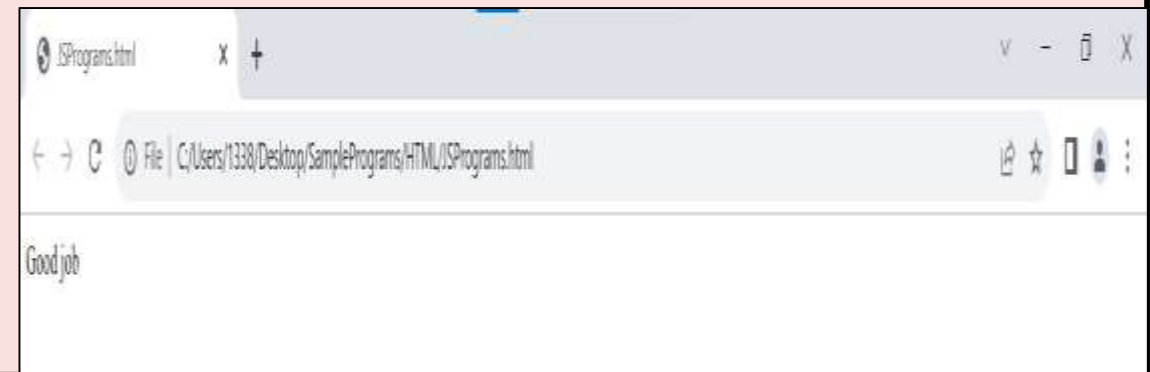
var grade='A';

switch (grade) {
  case 'A': document.write("Good job<br />");
    break;
  case 'B': document.write("Pretty good<br />");
    break;
  case 'C': document.write("Passed<br />");
    break;
}
```

Basic Elements : Control Flow Statements

Switch-case Statement

```
case 'D': document.write("Not so good<br />");  
    break;  
case 'F': document.write("Failed<br />");  
    break;  
default: document.write("Unknown grade<br />")  
}  
</script>
```



Basic Elements : Control Flow Statements

while Loop

```
while (condition) {  
    // code block to be executed  
}
```

```
<script type="application/javascript">  
var count = 0;  
    document.write("Starting Loop...." + "<br /><br />");  
while (count < 10){  
    document.write("Current Count : " + count + "<br />");  
    count++; }  
document.write("<br />Loop stopped!");  
</script>
```

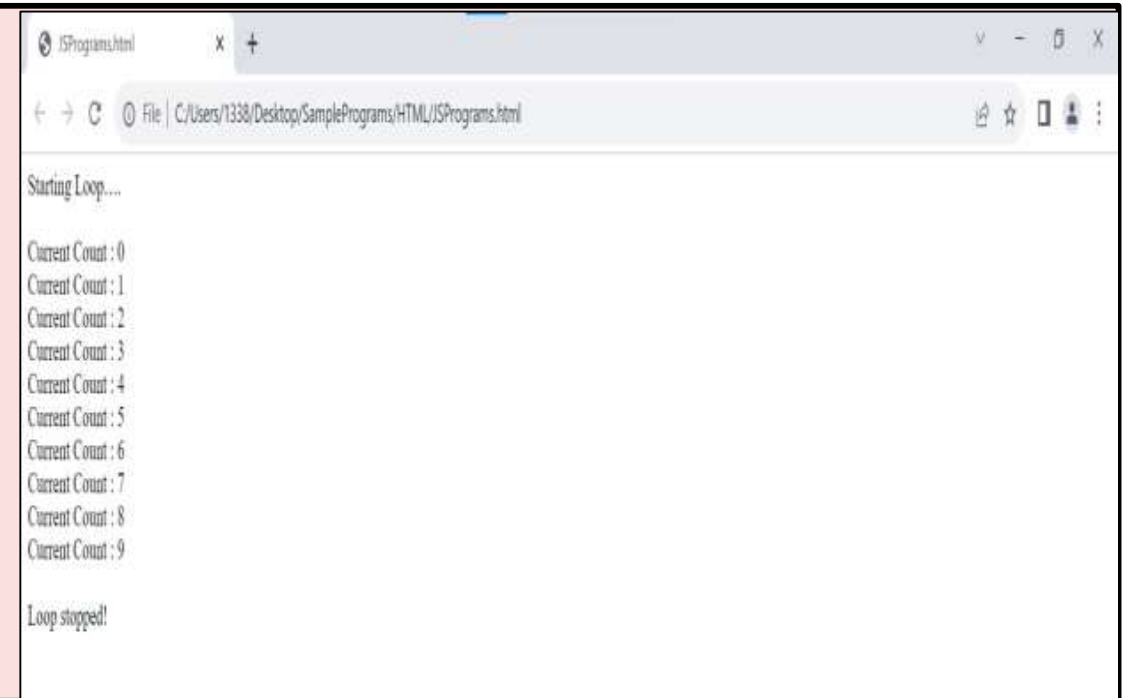


Basic Elements : Control Flow Statements

do...while Loop

```
do{  
    Statement(s) to be executed;  
} while (expression);
```

```
<script type="application/javascript">  
  
var count = 0;  
document.write("Starting Loop...." + "<br /><br />");  
do{  
    document.write("Current Count : " + count + "<br />");  
    count++;  
}while (count < 10);  
document.write("<br />Loop stopped!");  
  
</script>
```



Basic Elements : Control Flow Statements

for Loop

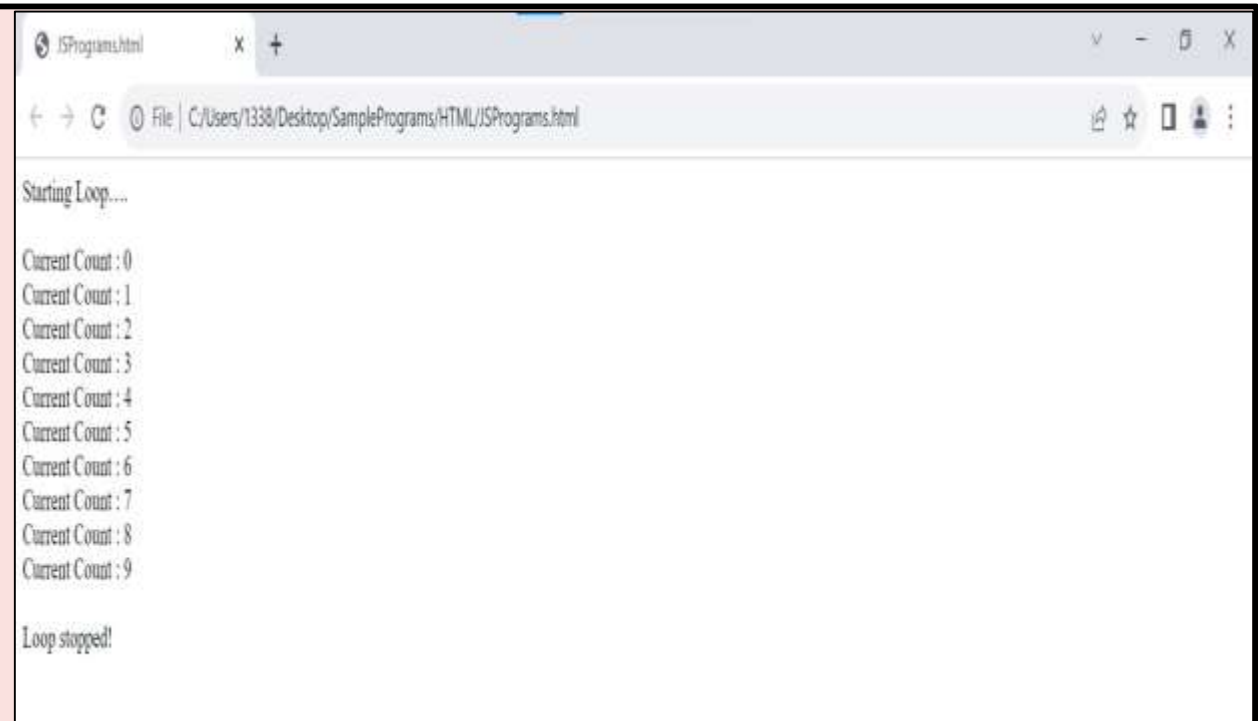
```
for (initialization; test condition; iteration statement){
    Statement(s) to be executed if test condition is true
}
```

```
<script type="application/javascript">

document.write("Starting Loop...." + "<br /><br />");

for(let count = 0; count < 10; count++){
    document.write("Current Count : " + count );
    document.write("<br />");
}
document.write("<br />Loop stopped!");

</script>
```



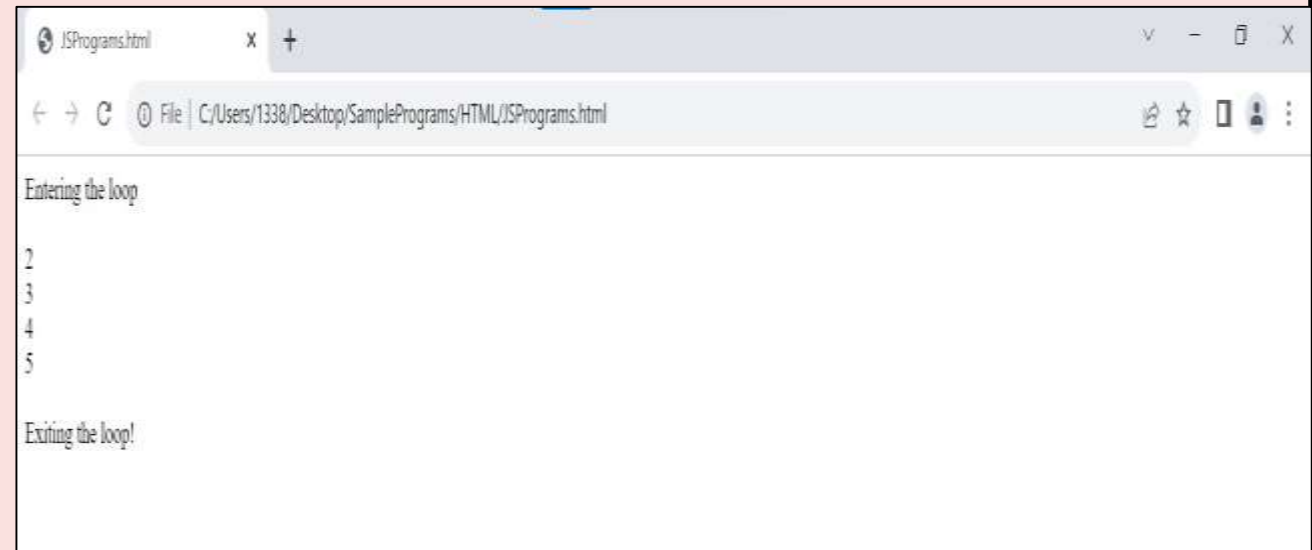
Basic Elements : Control Flow Statements

break Statement

```
<script type="application/javascript">

var x = 1;
document.write("Entering the loop<br /> <br />");
while (x < 20){
  if (x == 5){
    break; // breaks out of the loop completely
  }
  x = x + 1;
  document.write( x + "<br />");
}
document.write(" <br /> Exiting the loop!<br /> ");

</script>
```



Basic Elements : Control Flow Statements

label break Statement

```
<script type="application/javascript">  
document.write("Entering the loop!<br /><br />");  
  
outerloop: // This is the label name  
  
for (var i = 0; i < 5; i++){  
  
    document.write("<br />Outerloop: " + i + "<br />");
```

Basic Elements : Control Flow Statements

label break Statement

innerloop:

```
for (var j = 0; j < 5; j++) {  
    if (j > 3 ) break ;      // Quit the innermost loop  
    if (i == 2) break innerloop; // Do the same thing  
    if (i == 4) break outerloop; // Quit the outer loop  
    document.write("Innerloop: " + j + " <br />");  
}  
}  
  
document.write("<br />Exiting the loop!<br /> ");  
  
</script>
```

Basic Elements : Control Flow Statements

label break Statement



The screenshot shows a web browser window with the title 'JSPrograms.html'. The address bar displays the file path: 'C:/Users/1338/Desktop/SamplePrograms/HTML/JSPrograms.html'. The main content area of the browser displays the following text:

```
Entering the loop!  
  
Outerloop: 0  
Innerloop: 0  
Innerloop: 1  
Innerloop: 2  
Innerloop: 3  
  
Outerloop: 1  
Innerloop: 0  
Innerloop: 1  
Innerloop: 2  
Innerloop: 3  
  
Outerloop: 2  
  
Outerloop: 3  
Innerloop: 0  
Innerloop: 1  
Innerloop: 2  
Innerloop: 3  
  
Outerloop: 4  
Exiting the loop!
```

Basic Elements : Control Flow Statements

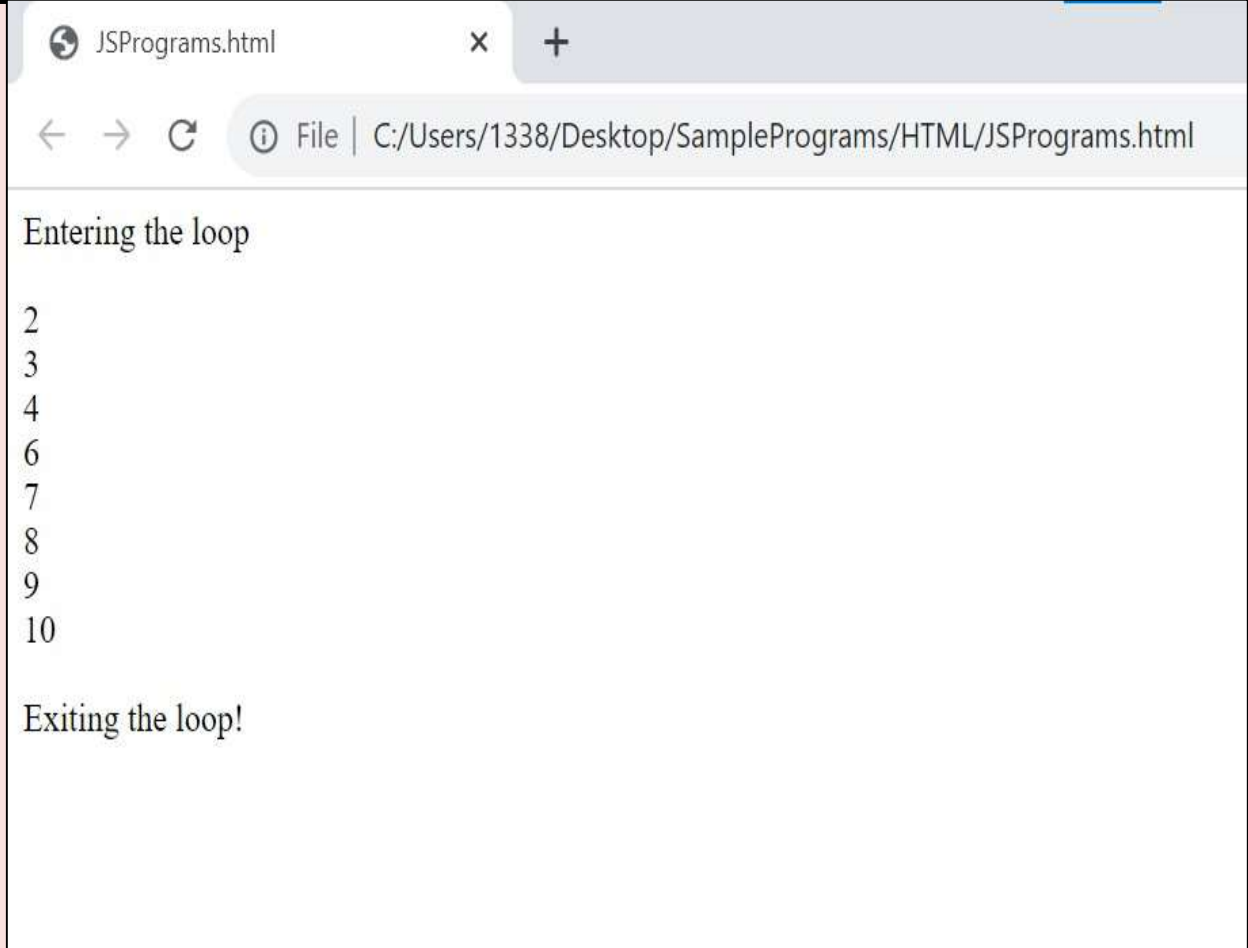
continue Statement

```
<script type="application/javascript">

var x = 1;

document.write("Entering the loop<br /><br /> ");
while (x < 10) {
    x = x + 1;
    if (x == 5){
        continue; // skip rest of the loop body
    }
    document.write( x + "<br />");
}
document.write("<br /> Exiting the loop!<br /> ");

</script>
```



Basic Elements : Control Flow Statements

label continue Statement

```
<script type="application/javascript">  
  
document.write("Entering the loop!<br /><br /> ");  
  
outerloop: // This is the label name  
  
for (var i = 0; i < 3; i++){  
  
    document.write("<br />Outerloop: " + i + "<br />");  
  
}
```

Basic Elements : Control Flow Statements

label continue Statement

```
for (var j = 0; j < 5; j++) {  
  
    if (j == 3){  
  
        continue outerloop;  
  
    }  
  
    document.write("Innerloop: " + j + "<br />");  
  
}  
  
}  
  
document.write("<br />Exiting the loop!<br /> ");  
  
</script>
```

Basic Elements : Control Flow Statements

label continue Statement



```
JSPrograms.html
File | C:/Users/1338/Desktop/SamplePrograms/HTML/JSPrograms.html

Entering the loop!

Outerloop: 0
Innerloop: 0
Innerloop: 1
Innerloop: 2

Outerloop: 1
Innerloop: 0
Innerloop: 1
Innerloop: 2

Outerloop: 2
Innerloop: 0
Innerloop: 1
Innerloop: 2

Exiting the loop!
```

Basic Elements : Functions

- A JavaScript function is a **block of code** designed to perform a particular task.
- A JavaScript function is executed when "**something**" **invokes it** (calls it).

Syntax:

```
function functionName(parameter-list){  
    statements  
}
```

- A JavaScript function is defined with the **function keyword**, followed by a name, followed by parentheses ().
- Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).
- The parentheses may include **parameter names separated by commas**: (parameter1, parameter2, ...)
- The code to be executed, by the function, is placed inside curly brackets: {}

Basic Elements : Functions

```
<!DOCTYPE html>

<html>

<head>

<!-- Called function -->
  <script type="text/javascript">

    function sayHello() {

      alert("Hello there");

    }

  </script>

</head>
```

Basic Elements : Functions

```
<body>
```

```
<!-- Calling function -->
```

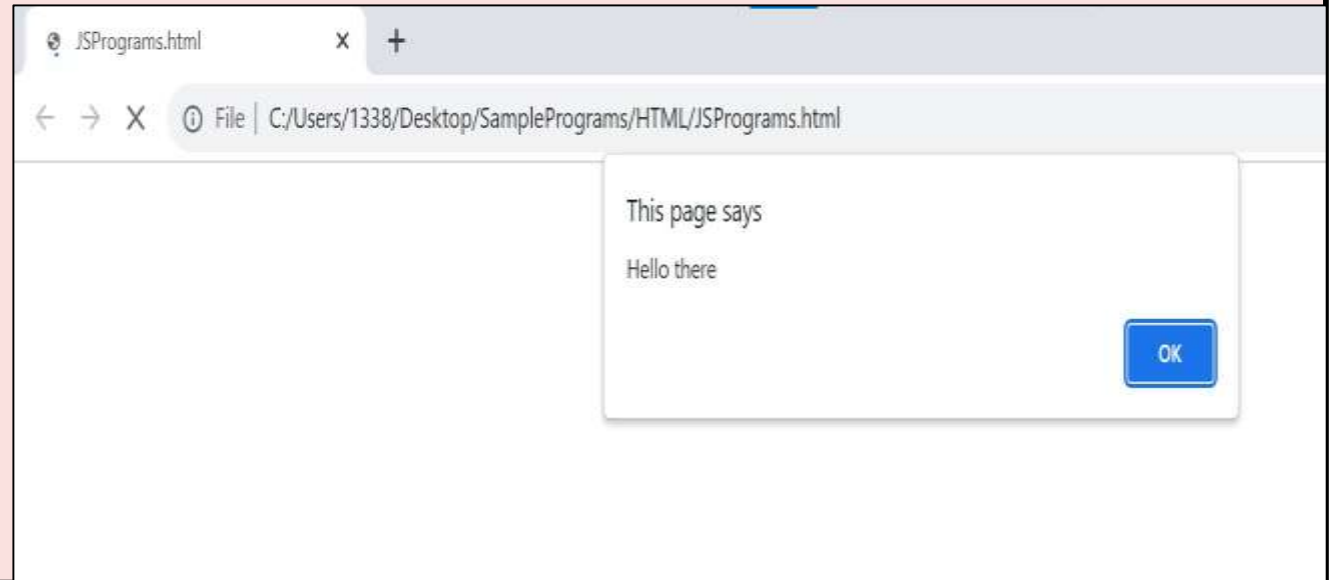
```
<script type="text/javascript">
```

```
    sayHello();
```

```
</script>
```

```
</body>
```

```
</html>
```



Basic Elements : DialogBoxes

Confirmation Dialog Box:

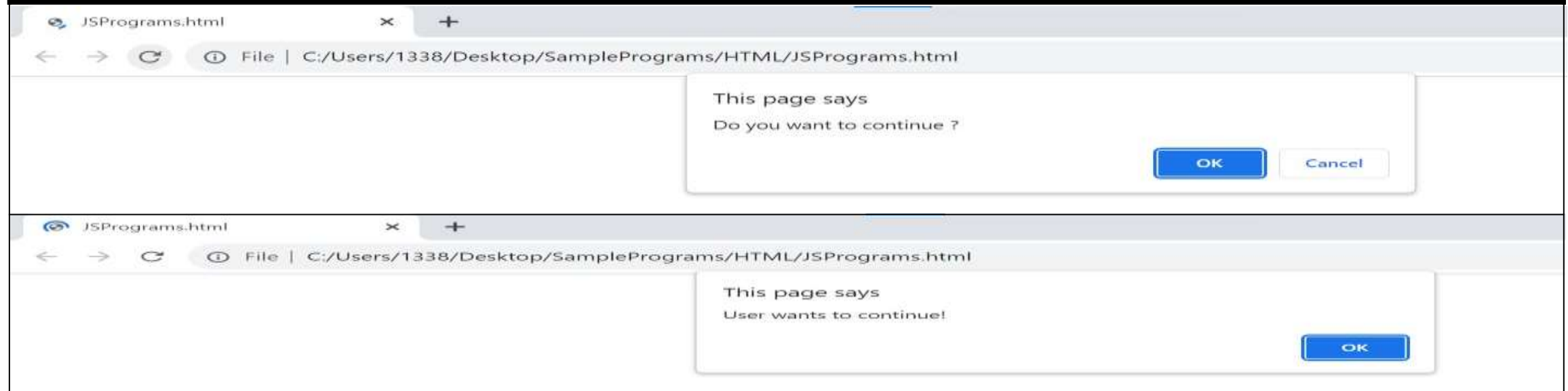
- A confirmation dialog box is mostly used to take the user's consent on any option. It displays a dialog box with two buttons: **OK** and **Cancel**.
- If the user clicks on the **OK** button the window method **confirm()** will return true.

```
<html>
<head>
<script>
  function myConfirmbox(){
    var retVal = confirm("Do you want to continue ?");
    if( retVal == true ){
      alert("User wants to continue!");
    }
    else{
      alert("User does not want to continue!");
    }
  }
</script>
</head>
```

Basic Elements : DialogBoxes

Confirmation Dialog Box:

```
<body>
<script>
    myConfirmbox();
</script>
</body>
</html>
```



Basic Elements : DialogBoxes

Prompt Dialog Box:

- The prompt dialog box is very useful when you want to pop-up a text box to get user input.
- This dialog box with two buttons: **OK** and **Cancel**.
- The window method **prompt()** returns the entered value when the user clicks on **OK** button.

```
<html>

<head>
<script>
function myPrompt(){
    var retVal = prompt("Enter your name : ", "your name here");
    alert("You have entered : " + retVal );
}
</script>
</head>
```

Basic Elements : DialogBoxes

Prompt Dialog Box:

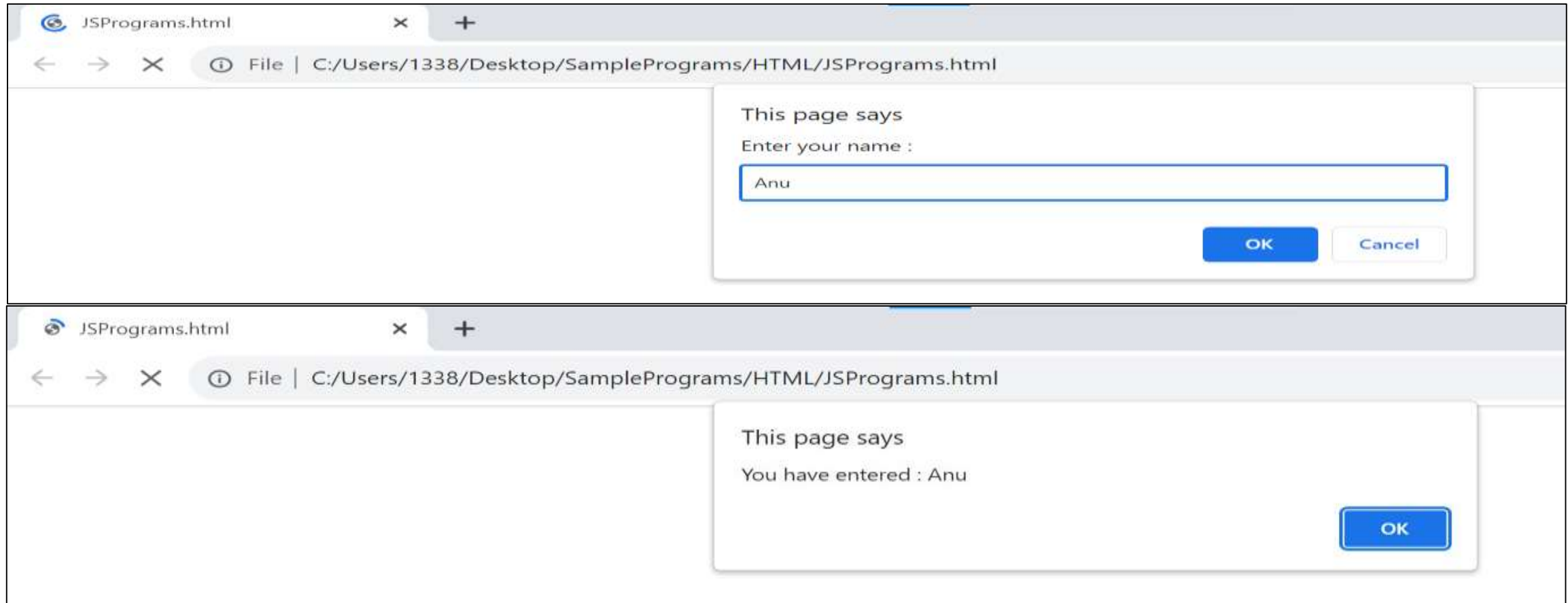
```
<body>  
<script>  
  myPrompt();  
</script>  
</body></html>
```



JavaScript Basics

Basic Elements : DialogBoxes

Prompt Dialog Box:



JavaScript Objects



JavaScript Objects

Introduction

- **"Everything"** in JavaScript is an Object. Even **primitive data types** (except null and undefined) can be treated as objects.
 - Booleans can be objects (or primitive data treated as objects)
 - Numbers can be objects (or primitive data treated as objects)
 - Strings are also objects (or primitive data treated as objects)
 - Dates are always objects
 - Maths and Regular Expressions are always objects
 - Arrays are always objects

JavaScript Objects

Introduction

Object Properties

- Properties are the values associated with an object.

Syntax:

- `objectName.propertyName`
- `objectName["propertyName"]`

JavaScript Objects

Introduction

Object Definition

- We define (and create) a JavaScript object with an object literal:

```
const person = {firstName:"Sachin", lastName:"Tendulkar", age:50};
```

Object Properties:

Property	Property Value
firstName	Sachin
lastName	Tendulkar
age	50

Example:

```
person.lastName; // Tendulkar
```

Or

```
person["lastName"];
```

JavaScript Objects

Introduction

Object Methods

- Objects can also have methods.
- Methods are actions that can be performed on objects.
- Methods are stored in properties as function definitions.

Accessing Object Methods:

- You access an object method with the following syntax:

```
objectName.methodName()
```


JavaScript Objects

Introduction

Examples:

```
<!DOCTYPE html>

<html>

<body>
  <script type="text/javascript">
    person = {firstName: "Sachin",  lastName : "Tendulkar", fullName : function() {
return this.firstName + " " + this.lastName;}};

document.write("The person full name is: " + person.fullName());

</script>

</body>

</html>
```



JavaScript Objects

Introduction

Examples:

```
<!DOCTYPE html>

<html>

<body>

  <script type="text/javascript">

    person = {firstName:"Sachin", lastName:"Tendulkar", age:50};

    person.fullName=function(){

      return person.firstName+" "+person.lastName; }

    document.write("The person full name is: " + person.fullName());

  </script>

</body>

</html>
```



JavaScript Objects

Introduction

Examples:

```
<!DOCTYPE html>

<html>
<body>
  <script type="text/javascript">
    person = {firstName:"Sachin", lastName:"Tendulkar", age:50};

    person.fullName=function(){
      return this.firstName+" "+this.lastName; }
    document.write("The person full name is: " + person.fullName());
  </script>
</body>
</html>
```



Arrays

- In JavaScript, arrays aren't primitives but are instead Array **objects** with the following core characteristics:
- JavaScript arrays are **resizable** and can contain a mix of **different data types**.
- JavaScript arrays are **not associative arrays** and so, array elements cannot be accessed using arbitrary strings as indexes, but must be accessed using nonnegative integers.
- JavaScript arrays are **zero-indexed**.
- JavaScript array-copy operations create **shallow copies**.

Arrays

Creating an array

- Using literal
 - `let arrayName = [value1, value2, ...];`
- Using Constructor
 - `let arrayName = new Array();`
 - `let arrayName = new Array(size);`
 - `let arrayName = new Array(value1, value2, ...);`

Arrays: Iteration

- Using `forEach`:

```
const arrayItems = ['item1', 'item2', 'item3'];
const copyItems = [];
arrayItems.forEach(function(item){
  copyItems.push(item);
})
console.log(copyItems);
```

- Using `for...of`:

```
const students = ['John', 'Sara', 'Jack'];
// using for...of
for ( let element of students ) {
  console.log(element);
}
```

Array: Properties and Methods

Methods	Description
push()	Adds new elements to the end of an array, and returns the new length
pop()	Removes the last element of an array, and returns that element
shift()	Removes the first element of an array, and returns that element
unshift()	Adds new elements to the beginning of an array, and returns the new length
find()	Returns the value of the first element in an array that pass a test
filter()	Creates a new array with every element in an array that pass a test
from()	Creates an array from an object
includes()	Check if an array contains the specified element
keys()	Returns a Array Iteration Object, containing the keys of the original array
map()	Creates a new array with the result of calling a function for each array element

Array: Properties and Methods

Methods	Description
reduce()	Reduce the values of an array to a single value (going left-to-right)
reverse()	Reverses the order of the elements in an array
slice()	Selects a part of an array, and returns the new array
sort()	Sorts the elements of an array
splice()	Adds/Removes elements from an array
toString()	Converts an array to a string, and returns the result
indexOf()	Search the array for an element and returns its position
lastIndexOf()	Search the array for an element, starting at the end, and returns its position
of()	Creates an array from a number of arguments
join()	Joins all elements of an array into a string

Strings

- A JavaScript string is **zero or more characters** written inside quotes.
- Normally, JavaScript strings are **primitive values**, created from literals:

Example: `let firstName = "Rahul";`

- But strings can also be **defined as objects** with the **keyword new**:

Example: `let firstName = new String("Rahul");`

Strings

Comparison of == and === operator

- == operator compares two string values.
- === operator checks equality in **both data type and value but not with objects.**

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    let x = "Sachin";
    let y = "Sachin";
    let z = new String("Sachin");
    let z1 = new String("Sachin");

    //checks if the values are equal
    document.write("Comparison using == operator: "+(x==y)+"<br/>");
    //checks both the values and data types are equal
    document.write("Comparison using === operator: "+(x===y)+"<br/><br/>");
```

JavaScript Objects

Strings

//converts the String object to its primitive value before comparing and checking that both values are equal

```
document.write("Comparison using == operator: "+(x==z)+"<br/>");
```

//checks both the values and the data types, and they are different (string vs. object)

```
document.write("Comparison using === operator: "+(x===z)+"<br/>");
```

//checking the two different string objects but they have the same string value

```
document.write("Comparison using == operator: "+(z==z1)+"<br/>");
```

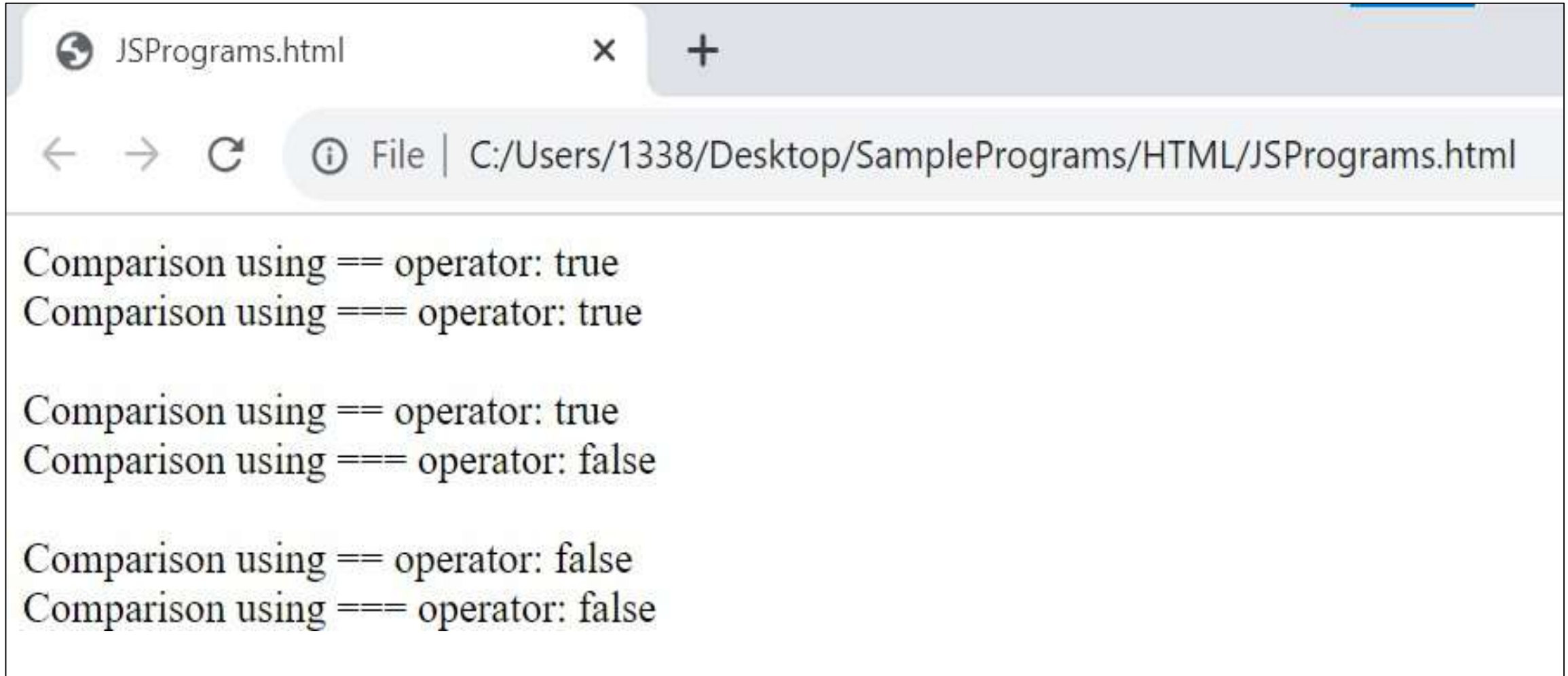
//checks both values and data types but not with objects

```
document.write("Comparison using === operator: "+(z===z1)+"<br/>");
```

```
</script>
```

```
</body>
```

```
</html>
```



```
Comparison using == operator: true
Comparison using === operator: true

Comparison using == operator: true
Comparison using === operator: false

Comparison using == operator: false
Comparison using === operator: false
```

JavaScript Objects

Strings

String Length:

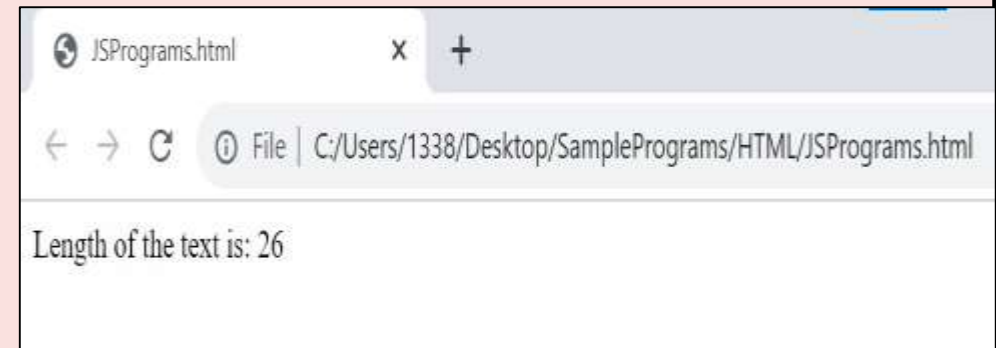
- To find the length of a string, use the **built-in length property**.

```
<!DOCTYPE html>
<html>

<body>
  <script type="text/javascript">

    let text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    document.write("Length of the text is: "+text.length);

  </script>
</body>
</html>
```



String Methods

Possible String Methods:

Method	Description
charAt()	Returns the character at the specified index.
concat()	Combines the text of two strings and returns a new string.
indexOf()	Returns the index within the calling String object of the first occurrence of the specified value, or -1 if not found.
lastIndexOf()	Returns the index within the calling String object of the last occurrence of the specified value, or -1 if not found.
match()	Used to match a regular expression against a string.
replace()	Used to find a match between a regular expression and a string, and to replace the matched substring with a new substring.
search()	Executes the search for a match between a regular expression and a specified string.
slice()	Extracts a section of a string and returns a new string.
split()	Splits a String object into an array of strings by separating the string into substrings.

JavaScript Objects

String Methods

Possible String Methods:

Method	Description
substr()	Returns the characters in a string beginning at the specified location through the specified number of characters.
substring()	Returns the characters in a string between two indexes into the string.
toLocaleLowerCase()	The characters within a string are converted to lower case while respecting the current locale.
toLocaleUpperCase()	The characters within a string are converted to upper case while respecting the current locale.
toLowerCase()	Returns the calling string value converted to lower case.
toString()	Returns a string representing the specified object.
toUpperCase()	Returns the calling string value converted to uppercase.
valueOf()	Returns the primitive value of the specified object.

String Methods: Example #1

The slice() Method

- **slice()** extracts a part of a string and returns the extracted part in a new string.
- The method takes **2 parameters**: the start position, and the end position (end not included).
- The positive starting index(from left to right) is 0 and the negative starting index(from right to left) is -1.

```
<!DOCTYPE html>
<html>

<body>
  <script type="text/javascript">

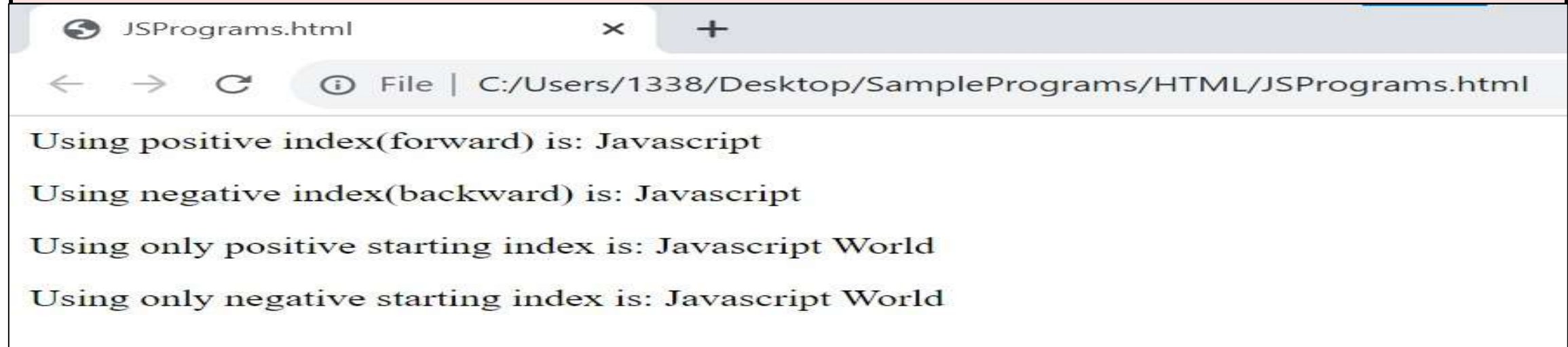
    let str = "Welcome to Javascript World";

    document.write("Using positive index(forward) is: "+str.slice(11, 21)+"<br/><br/>");
```


String Methods: Example #1

```
document.write("Using negative index(backward) is: "+str.slice(-16, -6)+"<br/><br/>");  
document.write("Using only positive starting index is: "+str.slice(11)+"<br/><br/>");  
document.write("Using only negative starting index is: "+str.slice(-16));
```

```
</script>  
</body>  
</html>
```

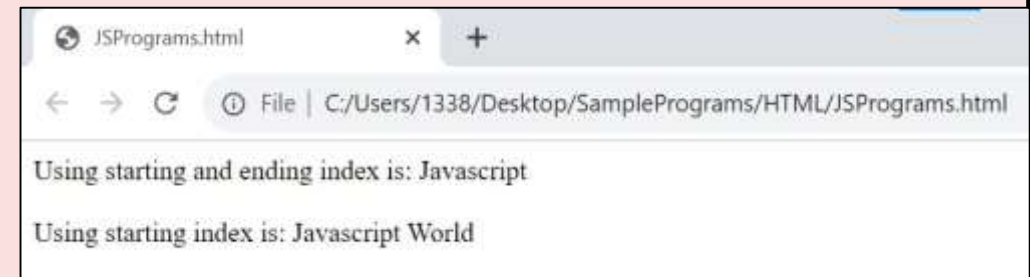


String Methods: Example #2

The substring() Method

- substring() is similar to slice().
- If start > stop, then the function swaps both arguments.
- If any argument is negative or is NaN, it is treated as 0.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    let str = "Welcome to Javascript World";
    document.write("Using starting and ending index is: "+str.substring(11, 21)+"<br/><br/>");
    document.write("Using starting index is: "+str.substring(11));
  </script>
</body>
</html>
```



String Methods: Example #3

The substr() Method

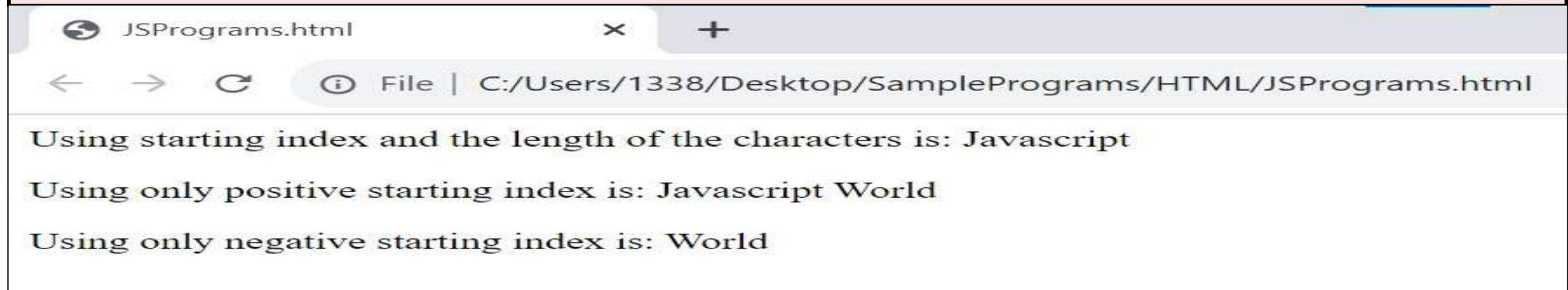
- substr() is similar to slice().
- The difference is that the second parameter specifies the length of the extracted part.
- If you omit the second parameter, substr() will slice out the rest of the string.

String Methods: Example #3

```
<!DOCTYPE html>
<html>
<body>

  <script type="text/javascript">
    let str = "Welcome to Javascript World";
    document.write("Using starting index and the length of the characters is: "+str.substr(11, 10)+"<br/><br/>");
    document.write("Using only positive starting index is: "+str.substr(11)+"<br/><br/>");
    document.write("Using only negative starting index is: "+str.substr(-5));

  </script>
</body>
</html>
```



String Methods: Example #4

Replacing String

- The `replace()` method replaces a specified value with another value in a string.
- The `replace()` method does not change the string it is called on.
- It returns a new string.
- By default, the `replace()` method replaces only the first match.
- To replace case insensitive, use a regular expression with an `/i` flag (insensitive).
- To replace all matches, use a regular expression with a `/g` flag (global match).

String Methods: Example #4

```
<!DOCTYPE html>
<html>

<body>
  <script type="text/javascript">

    let text1 = "Welcome to Nepal";
    //Replace Nepal to India
    let newText1 = text1.replace("Nepal", "India");
    document.write("Replaced text is: "+newText1+"<br/><br/>");

    let text2 = "Welcome to Nepal and Nepal Country";
    //Replace the first occurrence of Nepal to India
    let newText2 = text2.replace("Nepal", "India");

    document.write("Replaced only the first-occurrence is: "+newText2+"<br/><br/>");
```

String Methods: Example #4

//Replace the text using case-insensitive

```
let newText3 = text1.replace(/NEPAL/i, "India");
```

```
document.write("Replaced text using case-insensitive is: "+newText3+"<br/><br/>");
```

//Replace the global occurrence of Nepal to India

```
let newText4 = text2.replace(/Nepal/g, "India");
```

```
document.write("Replaced text using global is: "+newText4+"<br/><br/>");
```

```
</script>
```

```
</body>
```

```
</html>
```



Replaced text is: Welcome to India

Replaced only the first-occurrence is: Welcome to India and Nepal Country

Replaced text using case-insensitive is: Welcome to India

Replaced text using global is: Welcome to India and India Country

JavaScript Objects

Number Methods

- The Number object represents numerical data, either **integers** or **floating-point numbers**.

```
let x = 3.14; // A number with decimals  
let y = 3;    // A number without decimals
```

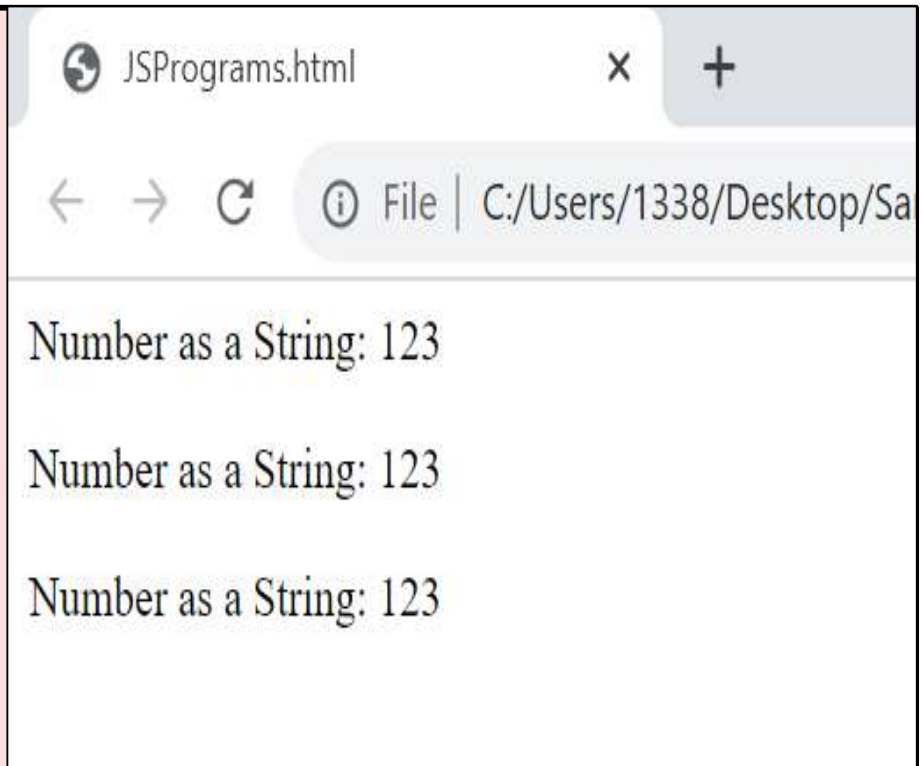
Method	Description
<code>constructor()</code>	Returns the function that created this object's instance. By default this is the Number object.
<code>toExponential()</code>	Forces a number to display in exponential notation, even if the number is in the range in which JavaScript normally uses standard notation.
<code>toFixed()</code>	Formats a number with a specific number of digits to the right of the decimal.
<code>toLocaleString()</code>	Returns a string value version of the current number in a format that may vary according to a browser's locale settings.
<code>toPrecision()</code>	Defines how many total digits (including digits to the left and right of the decimal) to display of a number.
<code>toString()</code>	Returns the string representation of the number's value.
<code>valueOf()</code>	Returns the number's value.

Number Methods: Example

The toString() Method:

- The toString() method returns a **number as a string**.
- All number methods can be used on any type of number (literals, variables, or expressions).

```
<!DOCTYPE html>
<html>
<body>
  <script>
    let x = 123;
    document.write("Number as a String: "+x.toString()+"<br/><br/>");
    document.write("Number as a String: "+(123).toString()+"<br/><br/>");
    document.write("Number as a String: "+(100 + 23).toString());
  </script>
</body>
</html>
```



Global Methods

Converting Variables to Numbers

- There are 3 JavaScript methods that can be used to convert a variable to a number:

Method	Description
Number()	Returns a number converted from its argument.
parseInt()	Parses its argument and returns a whole number
parseFloat()	Parses its argument and returns a floating point number

Global Methods: Number(): Example

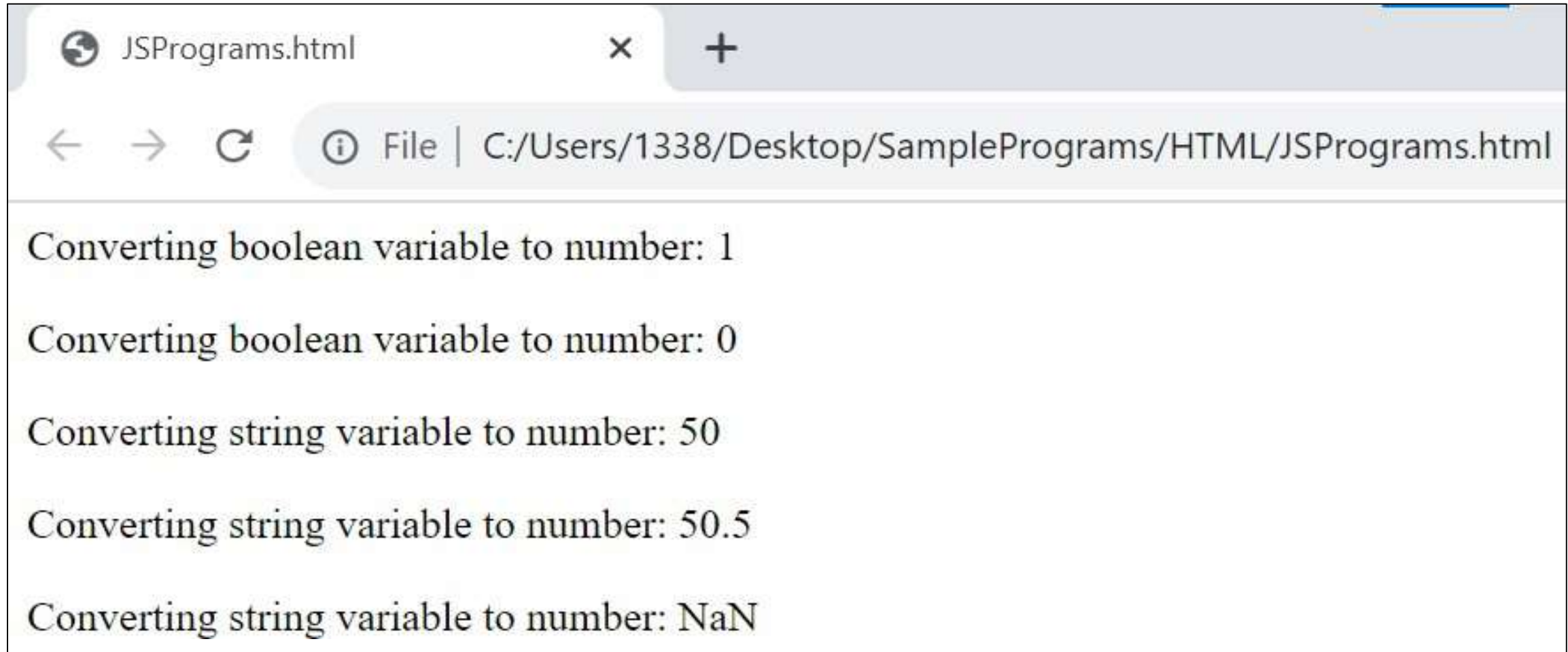
- This method can be used to convert JavaScript variables to numbers.

```
<!DOCTYPE html>
<html>
<body>

  <script type="text/javascript">
    document.write("Converting boolean variable to number: "+Number(true)+"<br/><br/>");
    document.write("Converting boolean variable to number: "+Number(false)+"<br/><br/>");
    document.write("Converting string variable to number: "+Number("50")+"<br/><br/>");
    document.write("Converting string variable to number: "+Number("50.50")+"<br/><br/>");
    document.write("Converting string variable to number: "+Number("Hi"));
  </script>

</body>
</html>
```

Global Methods: Number(): Example

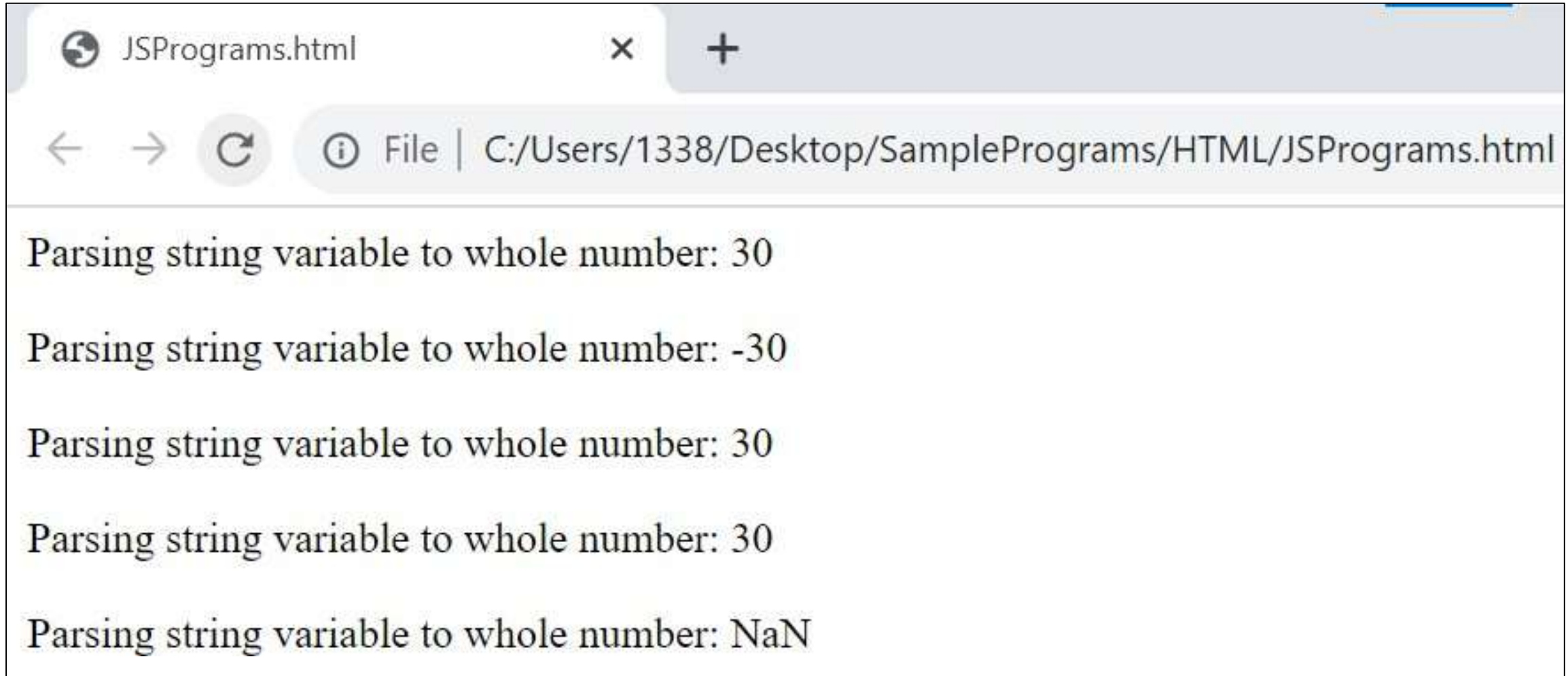


Global Methods: parseInt(): Example

- This method parses a string and returns a whole number.
- Spaces are allowed.
- Only the first number is returned.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    document.write("Parsing string variable to whole number: "+parseInt("30")+"<br/><br/>");
    document.write("Parsing string variable to whole number: "+parseInt("-30")+"<br/><br/>");
    document.write("Parsing string variable to whole number: "+parseInt("30.50")+"<br/><br/>");
    document.write("Parsing string variable to whole number: "+parseInt("30 60 90")+"<br/><br/>");
    document.write("Parsing string variable to whole number: "+parseInt("true"));
  </script>
</body>
</html>
```

Global Methods: parseInt(): Example



```

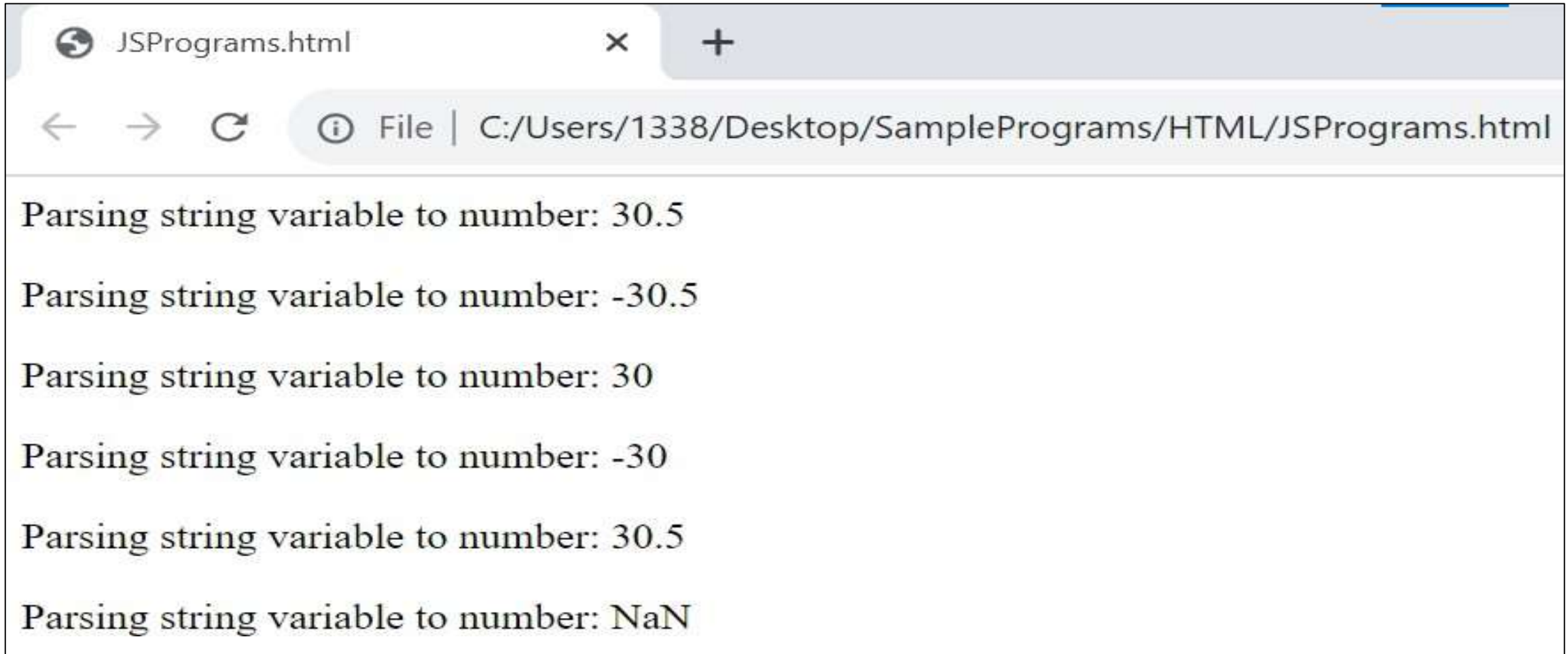
Parsing string variable to whole number: 30
Parsing string variable to whole number: -30
Parsing string variable to whole number: 30
Parsing string variable to whole number: 30
Parsing string variable to whole number: NaN
  
```

Global Methods: parseFloat(): Example

- This method parses a string and returns a number.
- Spaces are allowed.
- Only the first number is returned.

```
<!DOCTYPE html>
<html>
<body>
  <script type="text/javascript">
    document.write("Parsing string variable to number: "+parseFloat("30.50")+"<br/><br/>");
    document.write("Parsing string variable to number: "+parseFloat("-30.50")+"<br/><br/>");
    document.write("Parsing string variable to number: "+parseFloat("30")+"<br/><br/>");
    document.write("Parsing string variable to number: "+parseFloat("-30")+"<br/><br/>");
    document.write("Parsing string variable to number: "+parseFloat("30.50 60.50")+"<br/><br/>");
    document.write("Parsing string variable to number: "+parseFloat("true"));
  </script>
</body>
</html>
```

Global Methods: parseFloat(): Example



The screenshot shows a web browser window with a single tab titled "JSPrograms.html". The address bar displays the file path "C:/Users/1338/Desktop/SamplePrograms/HTML/JSPrograms.html". The main content area of the browser displays six lines of text, each representing the output of the `parseFloat()` method for a different string input. The outputs are: "Parsing string variable to number: 30.5", "Parsing string variable to number: -30.5", "Parsing string variable to number: 30", "Parsing string variable to number: -30", "Parsing string variable to number: 30.5", and "Parsing string variable to number: NaN".

```
Parsing string variable to number: 30.5  
Parsing string variable to number: -30.5  
Parsing string variable to number: 30  
Parsing string variable to number: -30  
Parsing string variable to number: 30.5  
Parsing string variable to number: NaN
```


Date

- JavaScript **Date objects** represent a single moment in time in a platform-independent format.
- Date objects encapsulate an integral number that represents milliseconds since the midnight at the beginning of January 1, 1970, UTC (the epoch).
- This timestamp is timezone-agnostic and uniquely defines an instant in history.
- A date is represented internally **as a single number**, the timestamp.
- There are various methods that allow you to interact with the timestamp stored in the date.
- There are always two ways to interpret a timestamp: as a local time or as a Coordinated Universal Time (UTC), the global standard time defined by the World Time Standard.

Creating Date

- To create a new Date object call **new Date()**. That is creating an object or instance of Date.
- The following snippet creates a date object with current date and time.

```
const date = new Date();  
console.log(date);
```

```
Fri Mar 01 2024 15:12:45 GMT+0530 (India Standard Time)
```

- You can create a Date object by passing individual date and **time components** (year, month, day, hour, minute, second, and millisecond) as arguments to the constructor.

```
const date = new Date(2023, 8, 10, 23, 15, 34, 0);  
console.log(date);
```

```
Sun Sep 10 2023 23:15:34 GMT+0530 (India Standard Time)
```

Creating Date

- You can create a Date object by passing a **date and time string** as an argument to the constructor. The string should be in a standardized date and time format.

```
const date = new Date("2013-09-10T13:24:00");  
console.log(date);
```

- You can create a Date Object by passing a **timestamp**, representing the total number of seconds (or even milliseconds), since the epoch snapshot (January 1, 1970).

```
const date = new Date(1688737100000);  
console.log(date);
```

Date Get Methods

Methods	Description
getFullYear()	Get year as a four digit number (yyyy)
getMonth()	Get month as a number (0-11)
getDate()	Get day as a number (1-31)
getDay()	Get weekday as a number (0-6)
getHours()	Get hour (0-23)
getMinutes()	Get minute (0-59)
getSeconds()	Get second (0-59)
getMilliseconds()	Get millisecond (0-999)
getTime()	Get time (milliseconds since January 1, 1970)

JavaScript Set Methods

Methods	Description
setDate()	Set the day as a number (1-31)
setFullYear()	Set the year (optionally month and day)
setHours()	Set the hour (0-23)
setMilliseconds()	Set the milliseconds (0-999)
setMinutes()	Set the minutes (0-59)
setMonth()	Set the month (0-11)
setSeconds()	Set the seconds (0-59)
setTime()	Set the time (milliseconds since January 1, 1970)

Regular Expressions and RegExp Object

- A regular expression is a **sequence of characters** that forms a search pattern.
- The search pattern can be used for **text search** and **text replace operations**.
- When you search for data in a text, you can use this search pattern to **describe what you are searching for**.
- A regular expression can be a **single character** or a **more complicated pattern**.
- Regular expressions can be used to perform all types of **text search** and **text replace operations**.

Regular Expressions and RegExp Object

Syntax:

```
var pattern = new RegExp(pattern, attributes);  
    (OR)  
var pattern = /pattern/attributes;
```

Here,

- **pattern:**
 - A string that specifies the **pattern of the regular expression**.
- **attributes:**
 - An optional string containing any of the **"g", "i", and "m"** attributes that specify **global, case-insensitive, and multiline matches**, respectively.

Regular Expressions: Quantifiers

- Possible quantifiers

Expression	Description
[...]	Any one character between the brackets.
[^...]	Any one character not between the brackets.
[0-9]	It matches any decimal digit from 0 through 9.
[a-z]	It matches any character from lowercase a through lowercase z.
[A-Z]	It matches any character from uppercase A through uppercase Z.
[a-Z]	It matches any character from lowercase a through uppercase Z.
[aeiou]	matches a single character in the given set
[^aeiou]	matches a single character outside the given set
(foo bar baz)	matches any of the alternatives specified

Regular Expressions: Quantifiers

- Possible quantifiers

Expression	Description
p+	It matches any string containing at least one p.
p*	It matches any string containing zero or more p's.
p?	It matches any string containing one or more p's.
p{N}	It matches any string containing a sequence of N p's
p{2,3}	It matches any string containing a sequence of two or three p's.
p{2, }	It matches any string containing a sequence of at least two p's.
p\$	It matches any string with p at the end of it.
^p	It matches any string with p at the beginning of it.

RegExp Object Methods

- Possible Methods

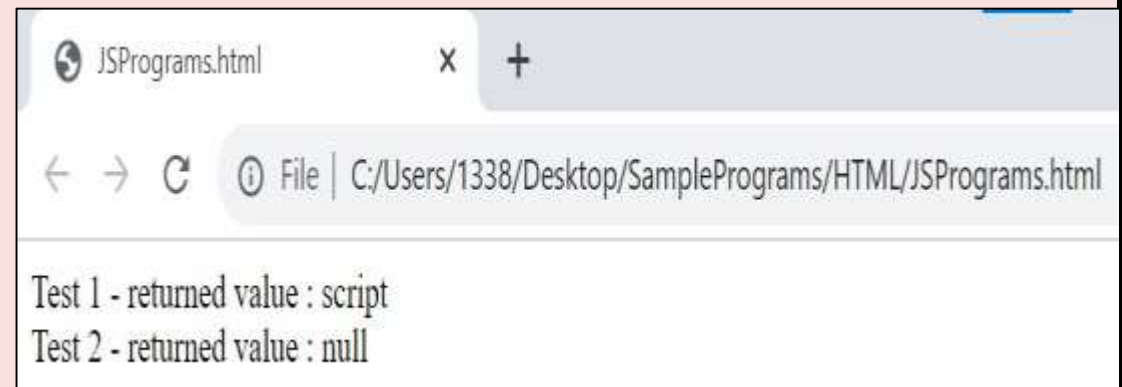
Method	Description
exec()	Executes a search for a match in its string parameter. If it finds a match, it returns the matched text ; otherwise, it returns null.
test()	Tests for a match in its string parameter. This method returns true if it finds a match, otherwise it returns false.
toSource()	Returns an object literal representing the specified object; you can use this value to create a new object.
toString()	Returns a string representing the specified object.

RegExp Object Methods: Example #1

exec() method:

```
<!DOCTYPE html>

<html>
<body>
<script type="text/javascript">
  var str = "Javascript is an interesting scripting language";
  var re = new RegExp( "script", "g" );
  var result = re.exec(str);
  document.write("Test 1 - returned value : " + result);
  re = new RegExp( "pushing", "g" );
  var result = re.exec(str);
  document.write("<br />Test 2 - returned value : " + result);
</script>
</body>
</html>
```

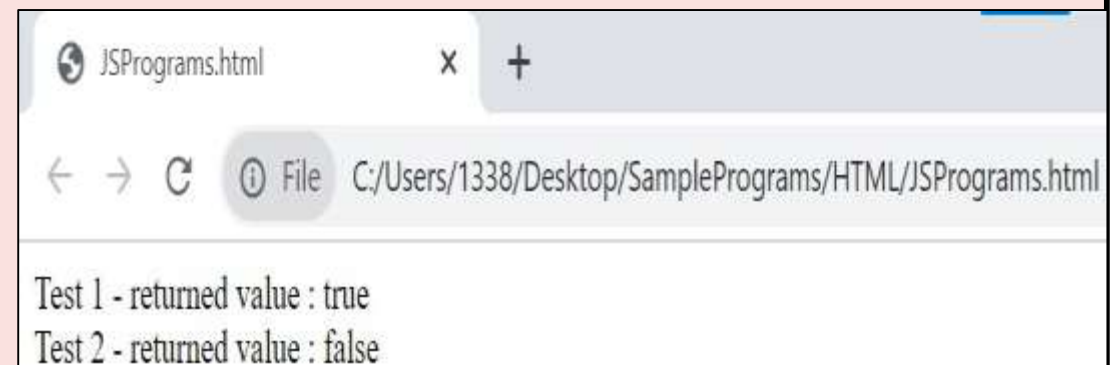


RegExp Object Methods: Example #2

test() method:

```
<!DOCTYPE html>

<html>
<body>
<script type="text/javascript">
  var str = "Javascript is an interesting scripting language";
  var re = new RegExp( "script", "g" );
  var result = re.test(str);
  document.write("Test 1 - returned value : " + result);
  re = new RegExp( "pushing", "g" );
  var result = re.test(str);
  document.write("<br />Test 2 - returned value : " + result);
</script>
</body>
</html>
```



JavaScript Events



Introduction

- **JavaScript's interaction** with **HTML** is handled through **events** that occur when the user or browser manipulates a page.
- A JavaScript can be executed when an event occurs, like when a **user clicks** on an HTML element.
- To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute.
- **Examples of HTML events:**
 - when a web page has loaded
 - when an image has been loaded
 - when the mouse moves over an element
 - when an input field is changed
 - when an HTML form is submitted
 - when a user strokes a key

JavaScript Events

Introduction

• Possible Events

Event	Description
onchange	Script runs when the element changes
onsubmit	Script runs when the form is submitted
onreset	Script runs when the form is reset
onselect	Script runs when the element is selected
onblur	Script runs when the element loses focus
onfocus	Script runs when the element gets focus
onkeydown	Script runs when key is pressed
onkeypress	Script runs when key is pressed and released
onkeyup	Script runs when key is released
onclick	Script runs when a mouse click
ondblclick	Script runs when a mouse double-click

JavaScript Events

Introduction

• Possible Events

Event	Description
onmousedown	Script runs when mouse button is pressed
onmousemove	Script runs when mouse pointer moves
onmouseout	Script runs when mouse pointer moves out of an element
onmouseover	Script runs when mouse pointer moves over an element
onmouseup	Script runs when mouse button is released
onload	The browser has finished loading the page

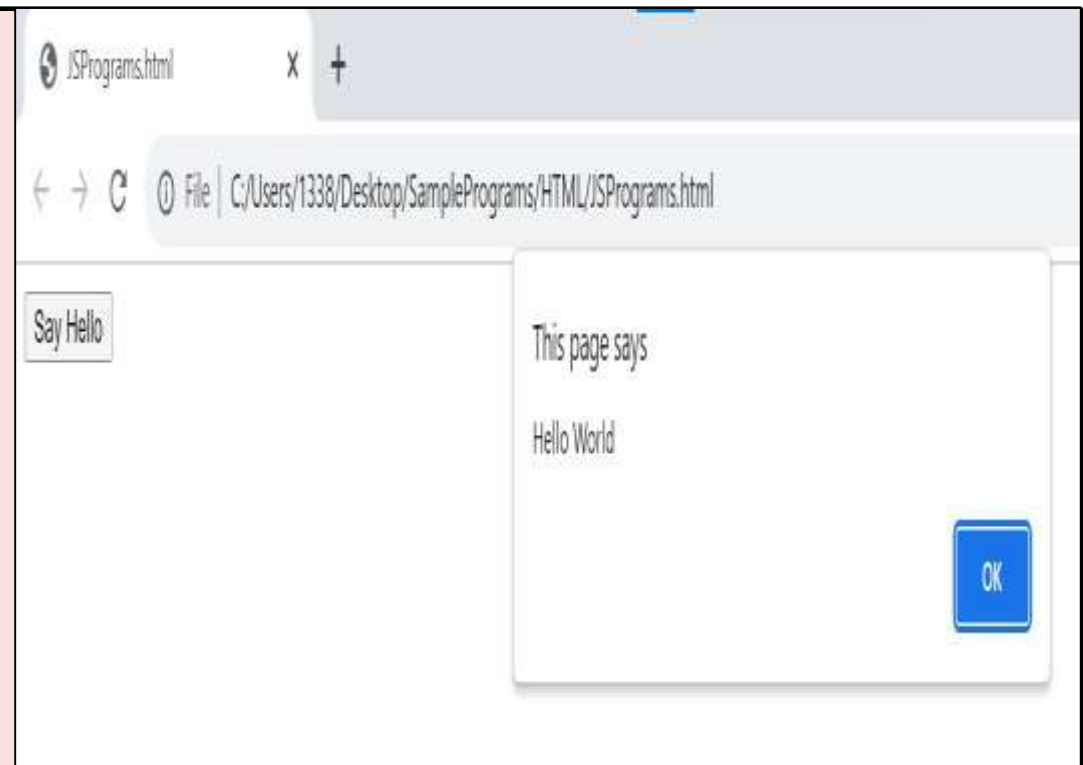
JavaScript Events

Example: #1

onclick:

- This is the most frequently used event type which occurs **when a user clicks the mouse's left button**.

```
<html>
<head>
<script type="text/javascript">
    function sayHello() {
        alert("Hello World")
    }
</script>
</head>
<body>
<input type="button" onclick="sayHello()" value="Say Hello"/>
</body>
</html>
```



JavaScript Events

Example: #2

onmouseover and onmouseout:

- The events can be used to trigger a function when the user mouses over, or out of, an HTML element.

```
<html>
<head>
<script type="text/javascript">
function over() { alert("Mouse Over"); }
function out() { alert("Mouse Out"); }
</script>
</head>
<body>
<p>Bring your mouse inside the division to see the result:</p>
<div onmouseover="over()" onmouseout="out()">
<h2> This is inside the division </h2>
</div>
</body>
</html>
```

JavaScript Events

Example: #2

 JSPrograms.html
 × +

← → ↻
File | C:/Users/1338/Desktop/SamplePrograms/HTML/JSPrograms.html

Bring your mouse inside the division to see the result:

This is inside the division

This page says

Mouse Over

OK

 JSPrograms.html
 × +

← → ↻
File | C:/Users/1338/Desktop/SamplePrograms/HTML/JSPrograms.html

Bring your mouse inside the division to see the result:

This is inside the division

This page says

Mouse Out

OK

Document Object Model (DOM)



DOM(Document Object Model)

Introduction

- The DOM is a W3C (World Wide Web Consortium) standard.
- It defines a standard for accessing documents:
- "The W3C **Document Object Model** (DOM) is a platform and language-neutral interface that **allows programs and scripts to dynamically access and update the content, structure, and style of a document.**"
- The W3C DOM standard is separated into **3 different parts**:
 - **Core DOM** - standard model for all document types
 - **XML DOM** - standard model for XML documents
 - **HTML DOM** - standard model for HTML documents

Introduction

- The HTML DOM is a standard object model and **programming interface for HTML**.
- It **defines**:
 - The HTML elements as **objects**
 - The **properties** of all HTML elements
 - The **methods** to access all HTML elements

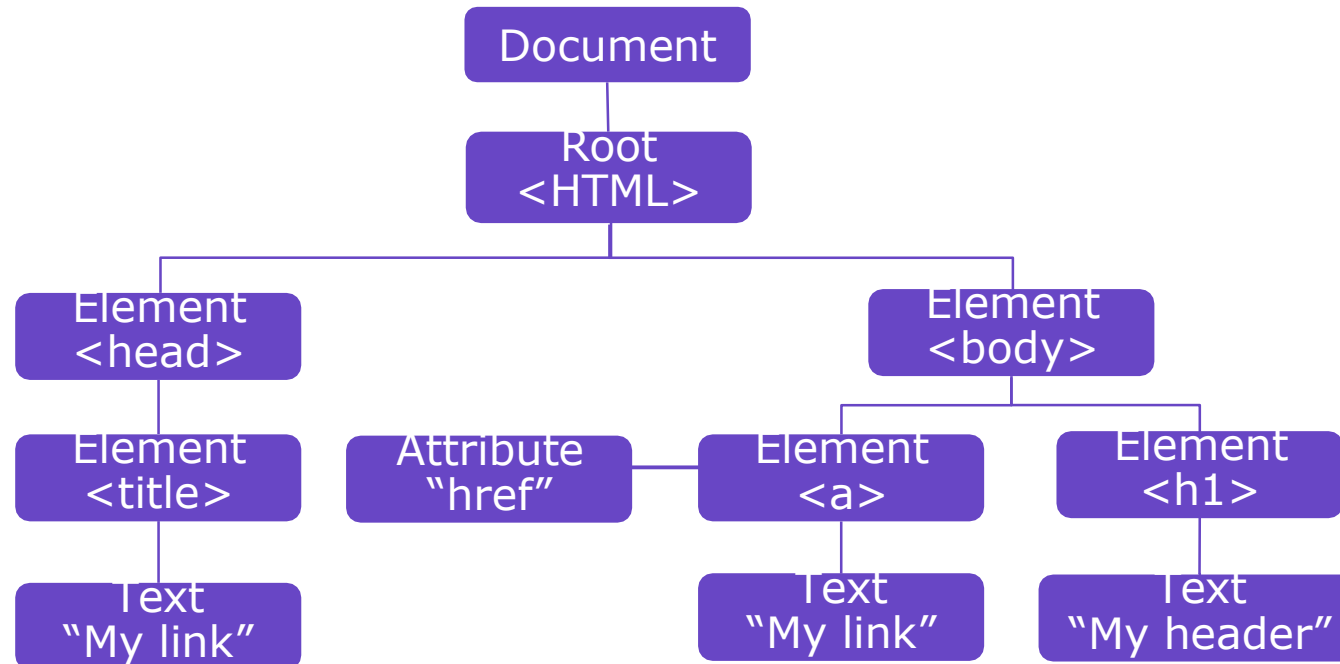
Note:

- When a web page is loaded, the browser creates a **Document Object Model** of the page.
- The **HTML DOM** model is constructed as a tree of **Objects**.

DOM(Document Object Model)

Introduction

- The HTML DOM model is constructed as a tree of Objects.
 - For Ex: sample tree**



DOM(Document Object Model)

HTML DOM Methods and Property

- HTML DOM methods are **actions** you can perform (on HTML Elements).
- HTML DOM properties are **values** (of HTML Elements) that you can set or change.

The innerHTML Property

- The innerHTML property is the simplest way to **access the content of an element**. The innerHTML property can be used to **access or replace** HTML element content.
- Any element's HTML content, including **<html>** and **<body>**, can be **retrieved or changed** using the innerHTML property.

DOM(Document Object Model)

HTML DOM Methods

- In the **HTML DOM object model**, the document object **represents a web page**.
- The document object is the **owner of all other objects** in a web page.
- Always start with accessing the document object to **access objects** in an HTML page.

Finding HTML Elements

Method	Description
document.getElementById()	Find an element by element id
document.getElementsByTagName()	Find elements by tag name
document.getElementsByClassName()	Find elements by class name

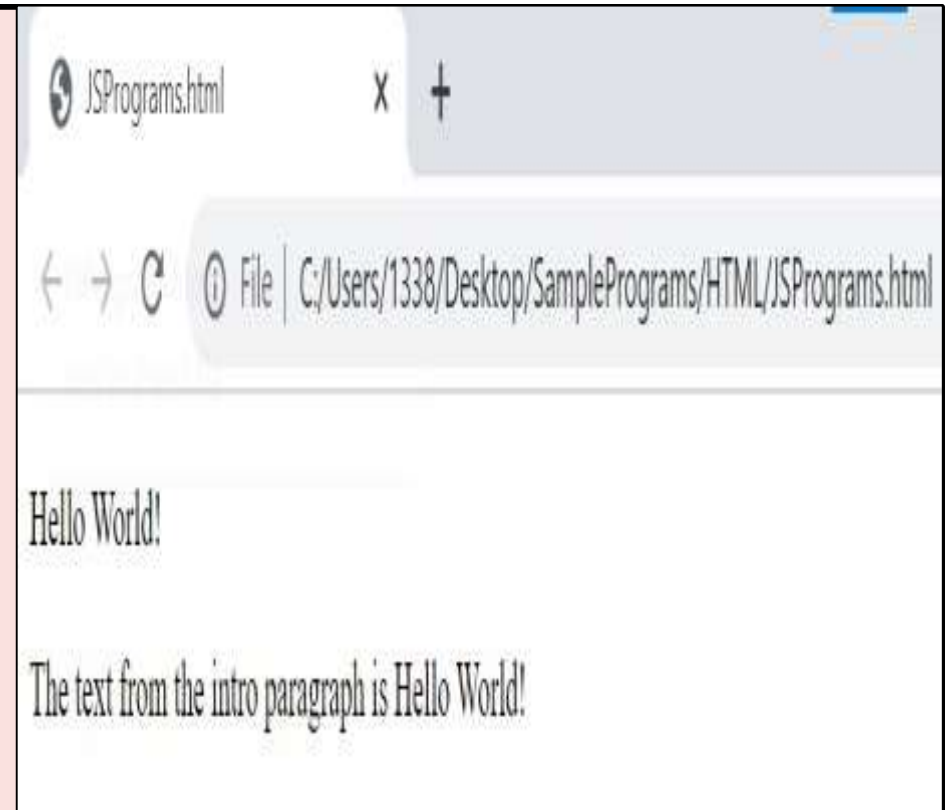
DOM(Document Object Model)

HTML DOM Methods : Finding HTML Elements

Finding HTML Elements by Id

- The easiest way to find HTML elements in the DOM is by using the **element ID**.

```
<!DOCTYPE html>
<html>
<body>
<p id="intro">Hello World!</p>
<p id="demo"></p>
<script>
  myElement = document.getElementById("intro");
  document.getElementById("demo").innerHTML =
    "The text from the intro paragraph is " + myElement.innerHTML;
</script>
</body>
</html>
```



HTML DOM Methods : Finding HTML Elements

Finding HTML Elements by Tag Name

- This example finds the element with id="main", and then finds all by **tag name** <p> elements inside "main".

```
<!DOCTYPE html>
<html>
<body>
<p>Hello World!</p>
<div id="main">
<p>The DOM is very useful.</p>
<p>This example demonstrates the <b>getElementsByTagName</b> method</p>
</div>
<p id="demo"></p>
```

DOM(Document Object Model)

HTML DOM Methods : Finding HTML Elements

Finding HTML Elements by Tag Name

```
<script>

var x = document.getElementById("main");

var y = x.getElementsByTagName("p");

document.getElementById("demo").innerHTML =

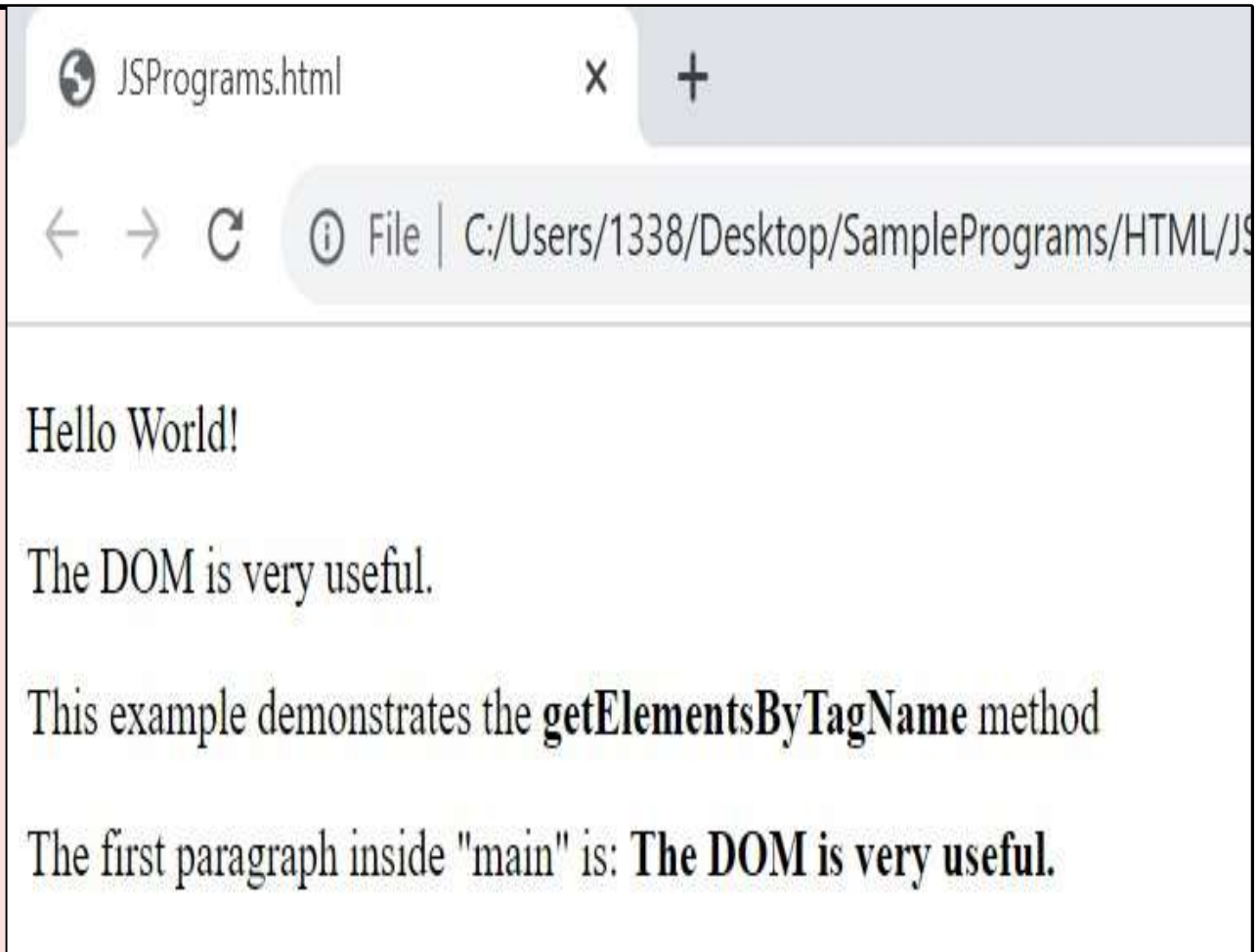
    'The first paragraph inside "main" is: ' +

    "<b>" + y[0].innerHTML + "</b>";

</script>

</body>

</html>
```



HTML DOM Methods : Finding HTML Elements

Finding HTML Elements by Class Name

- This example returns a list of all elements with class="intro".

```
<!DOCTYPE html>

<html>

<body>

<h2>JavaScript HTML DOM</h2>

<p>Finding HTML Elements by Class Name.</p>

<p class="intro">Hello World!</p>

<p class="intro">This example demonstrates the <b>getElementsByClassName</b> method.</p>

<p id="demo"></p>
```

DOM(Document Object Model)

HTML DOM Methods : Finding HTML Elements

Finding HTML Elements by Class Name

```
<script>

const x = document.getElementsByClassName("intro");

document.getElementById("demo").innerHTML =

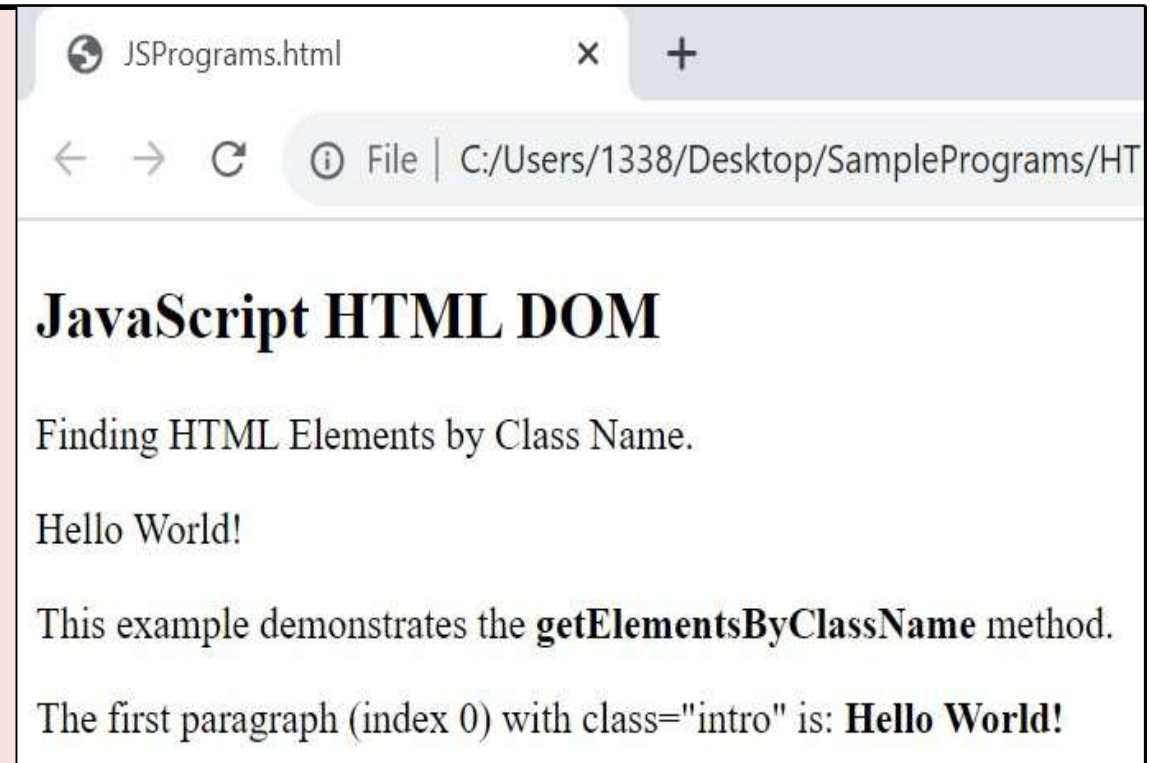
    'The first paragraph (index 0) with class="intro" is: '

    + "<b>" + x[0].innerHTML + "</b>";

</script>

</body>

</html>
```



DOM(Document Object Model)

HTML DOM Methods

Changing HTML Elements

Method	Description
<code>element.innerHTML=</code>	Change the inner HTML of an element
<code>element.attribute=</code>	Change the attribute of an HTML element
<code>element.setAttribute(attribute,value)</code>	Change the attribute of an HTML element
<code>element.style.property=</code>	Change the style of an HTML element

DOM(Document Object Model)

HTML DOM Methods : Changing HTML Elements

Changing HTML content

- The easiest way to modify the content of an HTML element is by using the innerHTML property.
- To change the content of an HTML element, use the following syntax.

Syntax:

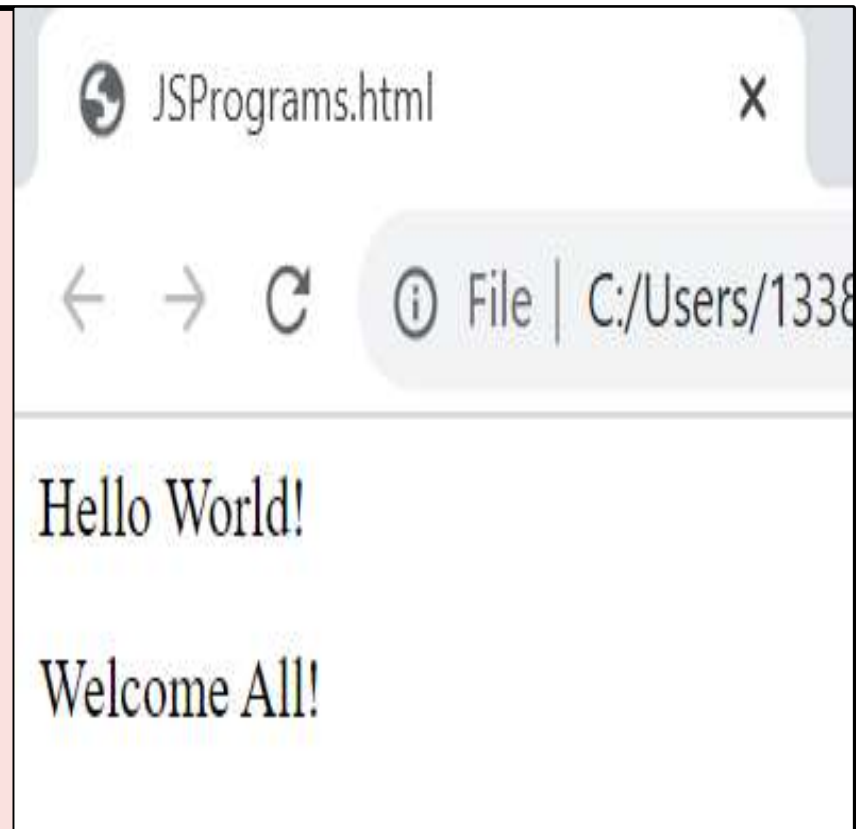
```
document.getElementById(id).innerHTML = new HTML
```


DOM(Document Object Model)

HTML DOM Methods : Changing HTML Elements

Changing HTML content

```
<!DOCTYPE html>
<html>
<body>
<p id="p1">Hello World!</p>
<p id="p2"></p>
<script>
    document.getElementById("p2").innerHTML = "Welcome All!";
</script>
</body>
</html>
```



DOM(Document Object Model)

HTML DOM Methods : Changing HTML Elements

Changing the value of an attribute

- To change the value of an HTML attribute, use the following syntax.

Syntax:

```
document.getElementById(id).attribute = new value
```

DOM(Document Object Model)

HTML DOM Methods : Changing HTML Elements

Changing the value of an attribute

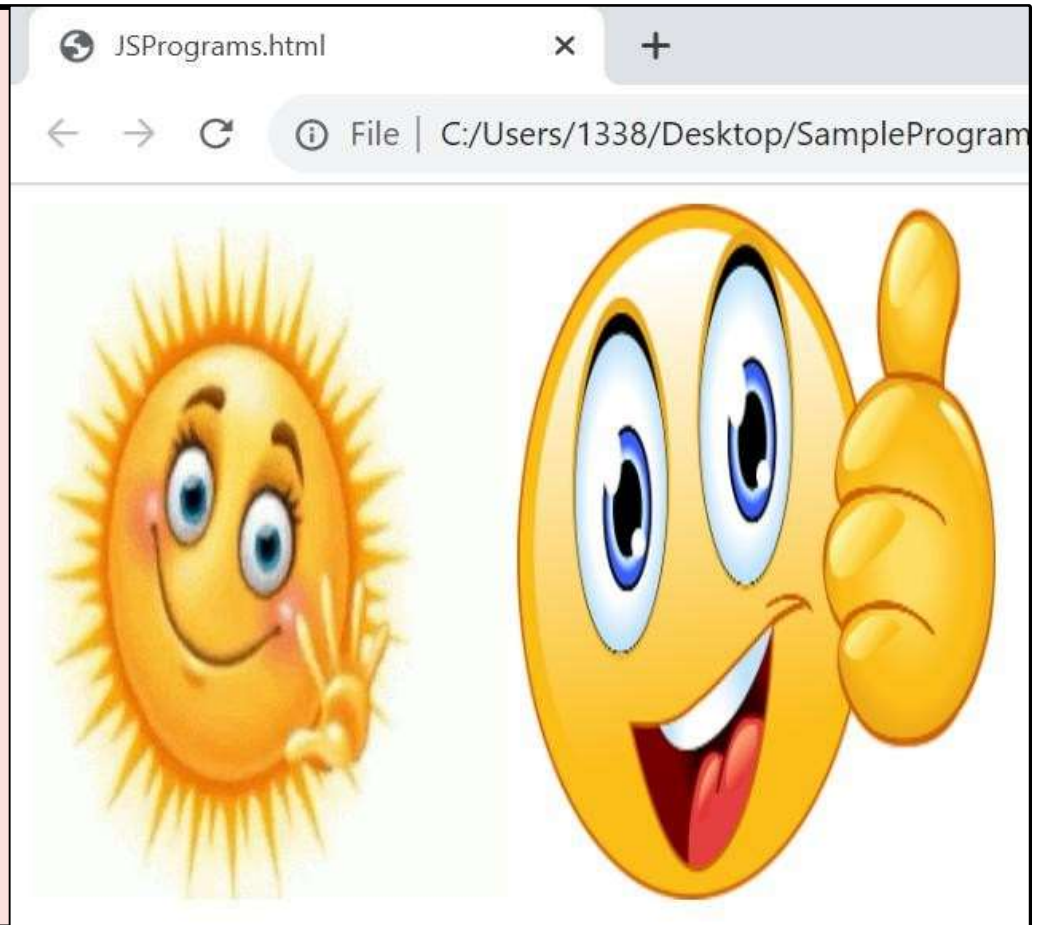
```
<!DOCTYPE html>

<html>

<body>

<img id="Img2" width="200" height="300">

<script>
    document.getElementById("Img2").src = "smiley.png";
</script>
</body>
</html>
```



DOM(Document Object Model)

HTML DOM Methods : Changing HTML Elements

Setting the value of an attribute

- To set the value of an HTML attribute, use the following syntax.

Syntax:

```
document.getElementById(id).setAttribute("attribute", "new value");
```

DOM(Document Object Model)

HTML DOM Methods : Changing HTML Elements

Setting the value of an attribute

```
<!DOCTYPE html>

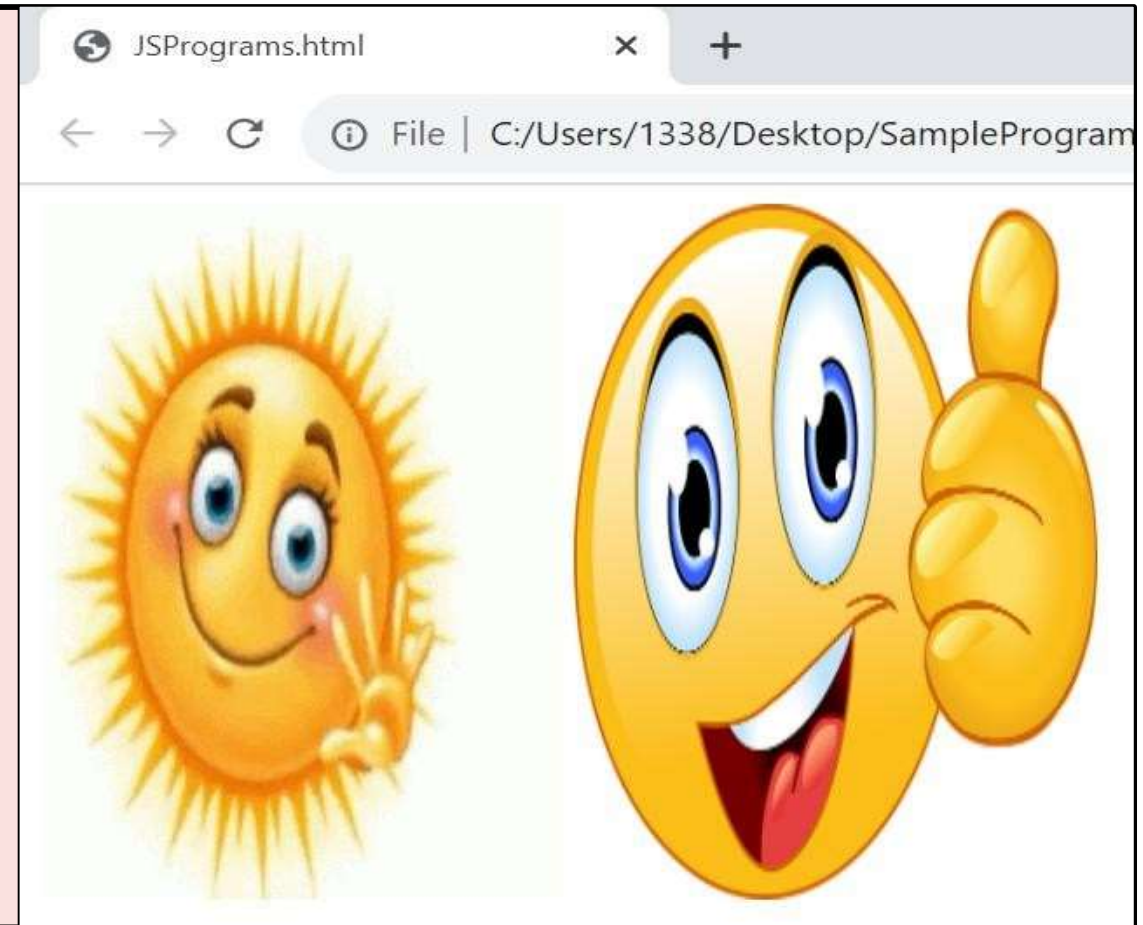
<html>

<body>

<img id="Img2" width="200" height="300">

<script>
    document.getElementById("Img2").
    setAttribute("src","smiley.png");

</script>
</body>
</html>
```



DOM(Document Object Model)

HTML DOM Methods : Changing HTML Elements

Changing the style attribute value

- To change the value of any attribute, use the following syntax.

Syntax:

```
document.getElementById(id).style.attribute = new value
```

DOM(Document Object Model)

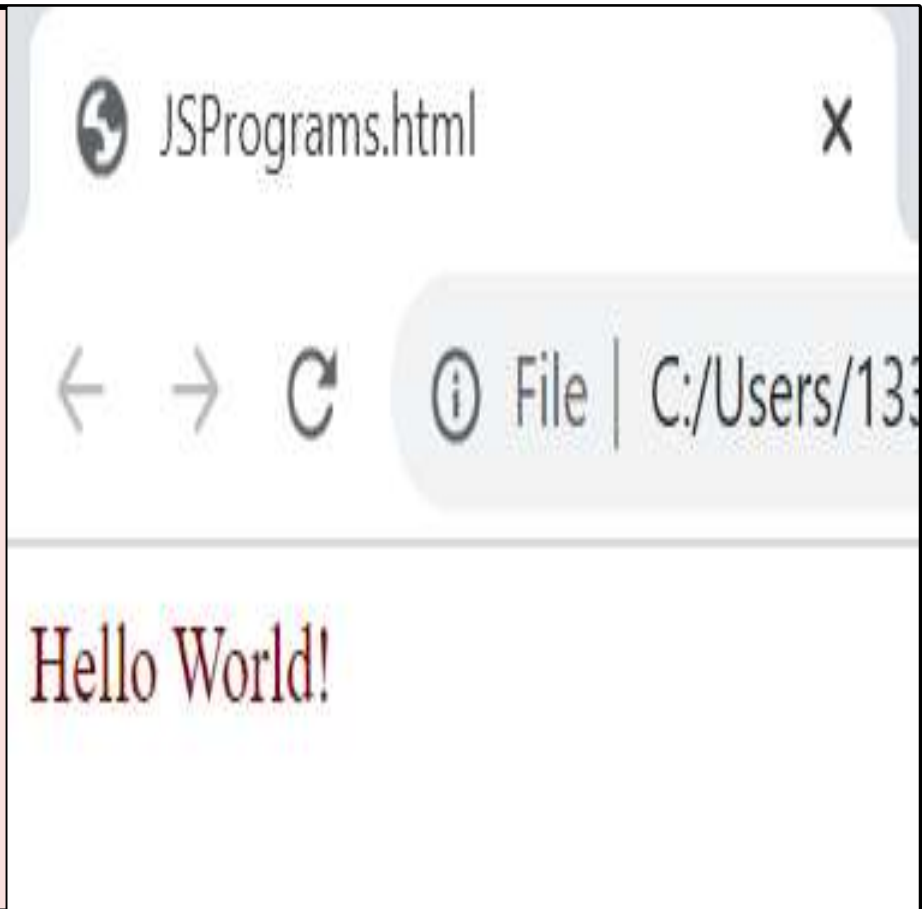
HTML DOM Methods : Changing HTML Elements

Changing the style attribute value

```
<!DOCTYPE html>
<html>
<body>
<p id="p1">Hello World!</p>

<script>
    document.getElementById("p1").style.color='maroon';
</script>

</body>
</html>
```



DOM(Document Object Model)

HTML DOM Methods

Adding and Deleting Elements

Method	Description
<code>document.createElement()</code>	Create an HTML element
<code>document.removeChild()</code>	Remove an HTML element
<code>document.appendChild()</code>	Add an HTML element
<code>document.replaceChild()</code>	Replace an HTML element
<code>document.write(text)</code>	Write into the HTML output stream

DOM(Document Object Model)

HTML DOM Methods: Example

```
<!DOCTYPE html>
<html>
  <body>
    <script>
      document.write("HTML DOM Methods");
    </script>
    <div id="section">
      <h1>Welcome All!!</h1>
      <h2>You are the Best</h2>
    </div>
    <p id="para">Have a Great day</p>
    <script>
      var div, heading, paragraph, oldValue, newValue;
```

HTML DOM Methods: Example

```
function CreateElement() {  
    div = document.getElementById("section");  
    heading = document.createElement("h3");  
    paragraph = document.createElement("p");  
    AppendElement();  
}  
  
function AppendElement() {  
    heading.textContent = "You can do Everything";  
    div.appendChild(heading);  
    paragraph.textContent = "Have a good day";  
    div.appendChild(paragraph);  
}
```

HTML DOM Methods: Example

```
function ReplaceElement() {  
    oldValue=document.getElementById("para");  
    newValue = document.createElement("p");  
    newValue.appendChild(document.createTextNode('Have a Wonderful Day'));  
    oldValue.replaceChild(newValue, oldValue.lastChild);  
}  
function RemoveElement() {  
    div.removeChild(paragraph);  
}  
</script>  
  
<button onclick="CreateElement()">Create and Append Element</button>  
<button onclick="ReplaceElement()">Replace Element</button>  
<button onclick="RemoveElement()">Remove Element</button>  
</body>  
</html>
```

DOM(Document Object Model)

HTML DOM Methods: Example

JSPrograms.html	JSPrograms.html
File C:/Users/1338/Desktop/SamplePrograms/H	File C:/Users/1338/Desktop/SamplePrograms/H
HTML DOM Methods Welcome All!! You are the Best Have a Great day Create and Append Element Replace Element Remove Element	HTML DOM Methods Welcome All!! You are the Best You can do Everything Have a good day Have a Great day Create and Append Element Replace Element Remove Element

DOM(Document Object Model)

HTML DOM Methods: Example

×
+

←
→
↻
ⓘ File | C:/Users/1338/Desktop/SamplePrograms/H

HTML DOM Methods

Welcome All!!

You are the Best

You can do Everything

Have a good day

Have a Wonderful Day

Create and Append Element

Replace Element

Remove Element

×
+

←
→
↻
ⓘ File | C:/Users/1338/Desktop/SamplePrograms/H

HTML DOM Methods

Welcome All!!

You are the Best

You can do Everything

Have a Wonderful Day

Create and Append Element

Replace Element

Remove Element

DOM(Document Object Model)

HTML DOM Events: onchange Event

- The onchange event occurs when the **value of an element has been changed**.
- For radio buttons and checkboxes, the onchange event occurs when the checked state has been changed.
- The upperCase() function will be called when a user changes the content of an input field.

```
<html>
<head>
<script>
    function upperCase(){
        var x = document.getElementById("fname");
        x.value = x.value.toUpperCase();
    }
</script>
</head>
```

DOM(Document Object Model)

HTML DOM Events: onchange Event : Example #1

```
<body>
```

```
Enter your name: <input type="text" id="fname" onchange="upperCase()">
```

```
<p>When you leave the input field, a function is triggered which transforms the input text to upper case.</p>
```

```
</body>
```

```
</html>
```



Event2.html



File

F:/Personal/Training/Internal%20Review/Web/JS/Example/Event2.html

Enter your name:

When you leave the input field, a function is triggered which transforms the input text to upper case.

DOM(Document Object Model)

HTML DOM Events: onfocus and onblur Events

onfocus Event:

- This event occurs when an element gets focus.
- This event is often used on input fields.

onblur Event:

- This event occurs when an **HTML element loses focus**.
- This event is often used on input fields.
- This event is often used with form validation (when the user leaves a form field).

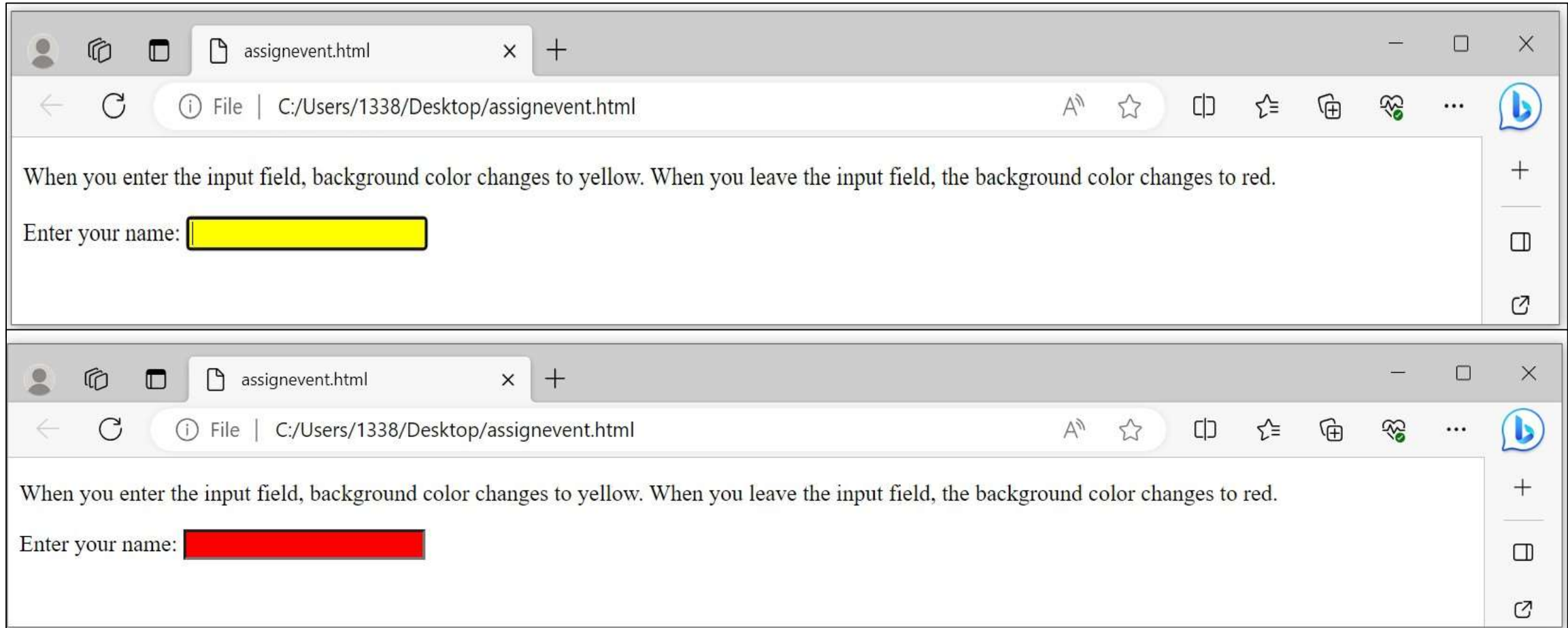
HTML DOM Events : onfocus and onblur Events : Example #2

```
<!DOCTYPE html>
<html>
<body>
<p>When you enter the input field, background color changes to yellow. When you leave the input field, the
background color changes to red.</p>
Enter your name: <input type="text" id="myInput" onfocus="focusFunction()" onblur="blurFunction()">
<script>

// Focus = Changes the background color of input to yellow
function focusFunction() {
    document.getElementById("myInput").style.background = "yellow";
}
// No focus = Changes the background color of input to red
function blurFunction() {
    document.getElementById("myInput").style.background = "red";
}
</script>
</body>
</html>
```

DOM(Document Object Model)

HTML DOM Events : onfocus and onblur Events : Example #2



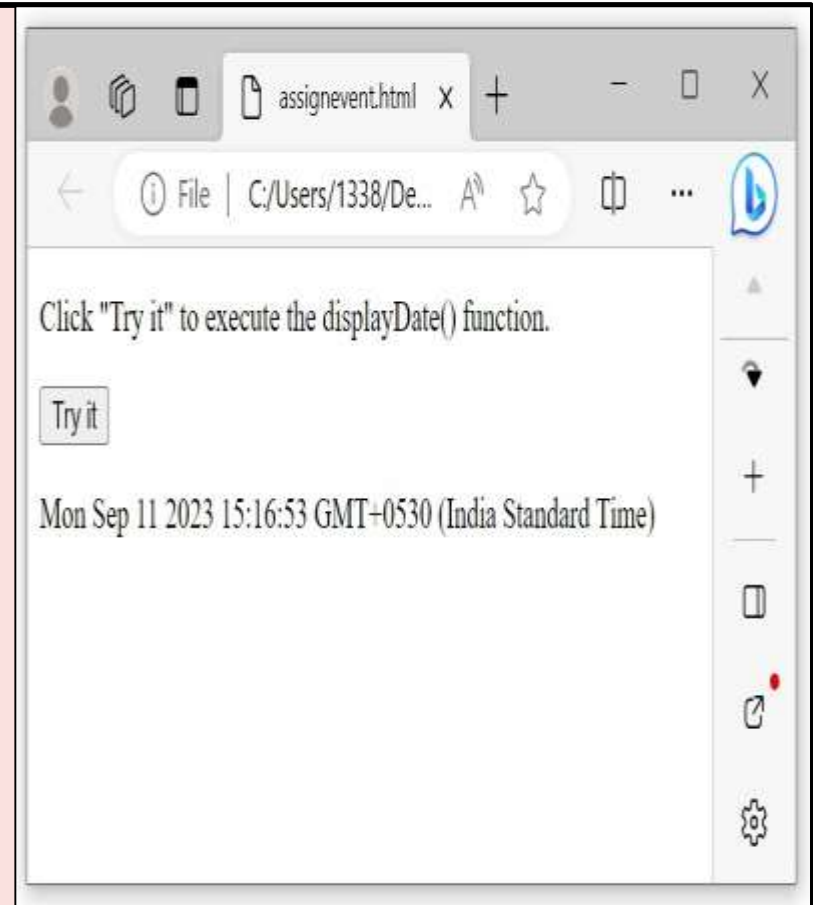
DOM(Document Object Model)

HTML DOM Events: onclick Event : Example #3

- This event occurs when the user **clicks on an HTML element**.

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<p>Click "Try it" to execute the displayDate() function.</p>
<button id="myBtn" onclick="displayDate()">Try it</button>

<p id="demo"></p>
<script>
  function displayDate() {
    document.getElementById("demo").innerHTML = Date();
  }
</script>
</body>
</html>
```



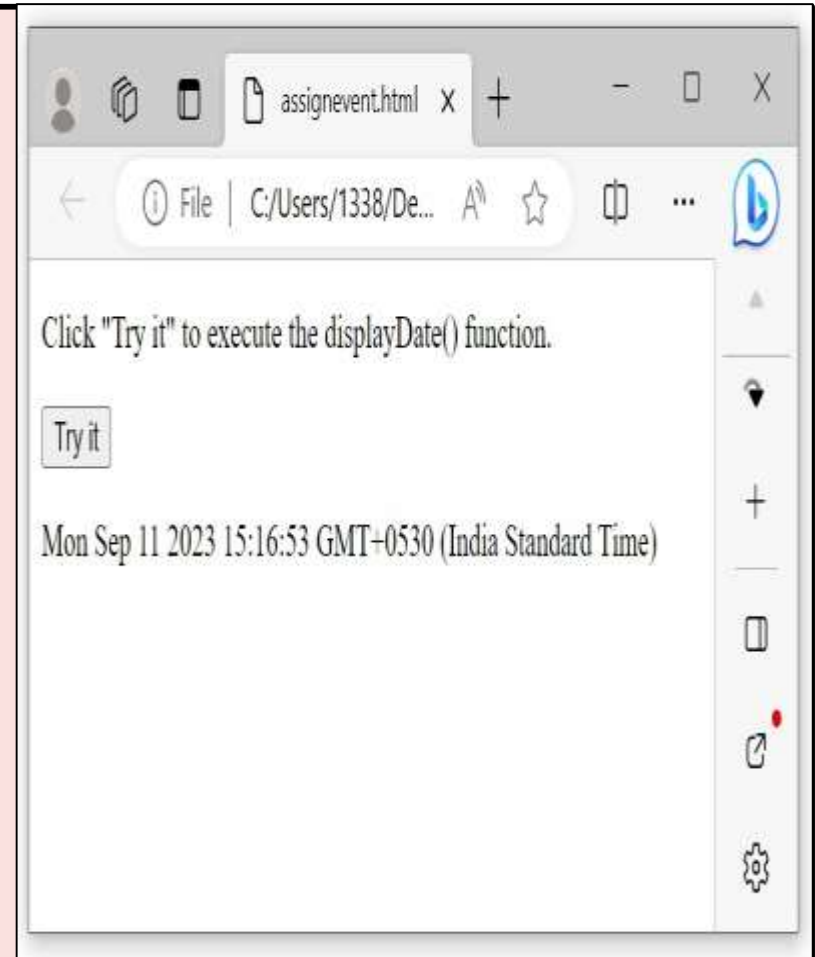
DOM(Document Object Model)

HTML DOM Events : Assign Events : Example

- A function named **displayDate()** is assigned to an HTML element with the **id="myBtn"**.

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<p>Click "Try it" to execute the displayDate() function.</p>
<button id="myBtn">Try it</button>

<p id="demo"></p>
<script>
  function displayDate() {
    document.getElementById("demo").innerHTML = Date();
  }
  document.getElementById("myBtn").onclick = function(){displayDate()};
</script>
</body>
</html>
```



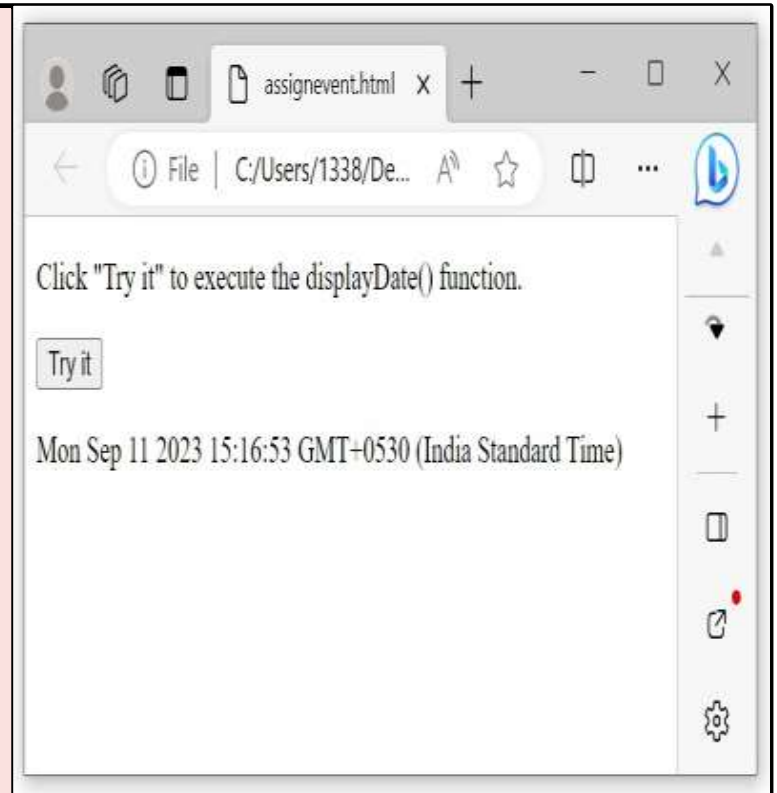
DOM(Document Object Model)

HTML DOM Event Listener : Example #1

addEventListener() method:

- This method attaches an event handler to the specified element.
- This method attaches an event handler to an element without overwriting existing event handlers.

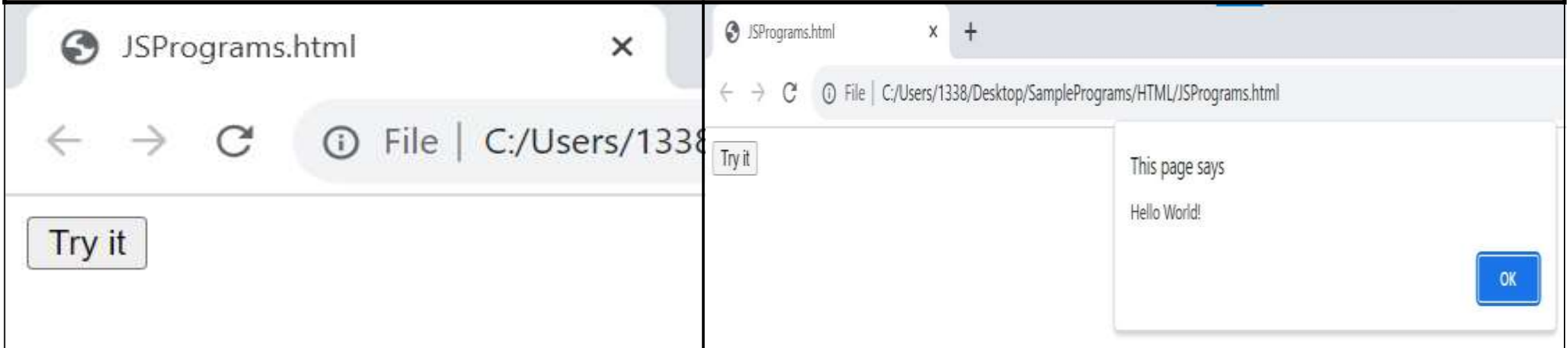
```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<p>Click "Try it" to execute the displayDate() function.</p>
<button id="myBtn">Try it</button>
<p id="demo"></p>
<script>
document.getElementById("myBtn"). addEventListener("click", displayDate);
function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>
</body>
</html>
```



DOM(Document Object Model)

HTML DOM Event Listener : Example #1

```
<!DOCTYPE html>
<html>
<body>
<button id="myBtn">Try it</button>
<script>
document.getElementById("myBtn").addEventListener("click", myFunction);
function myFunction() {
alert ("Hello World!");
}
</script>
</body>
</html>
```



DOM(Document Object Model)

HTML DOM Event Listener : Example #2

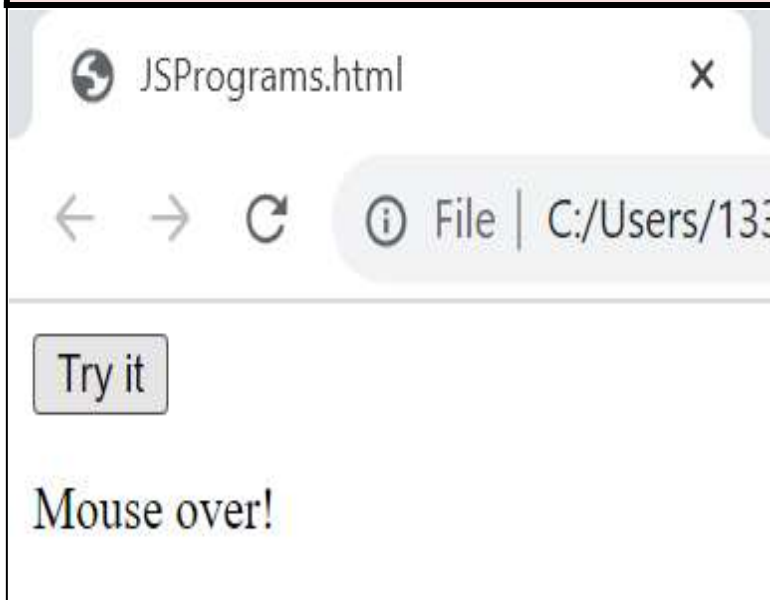
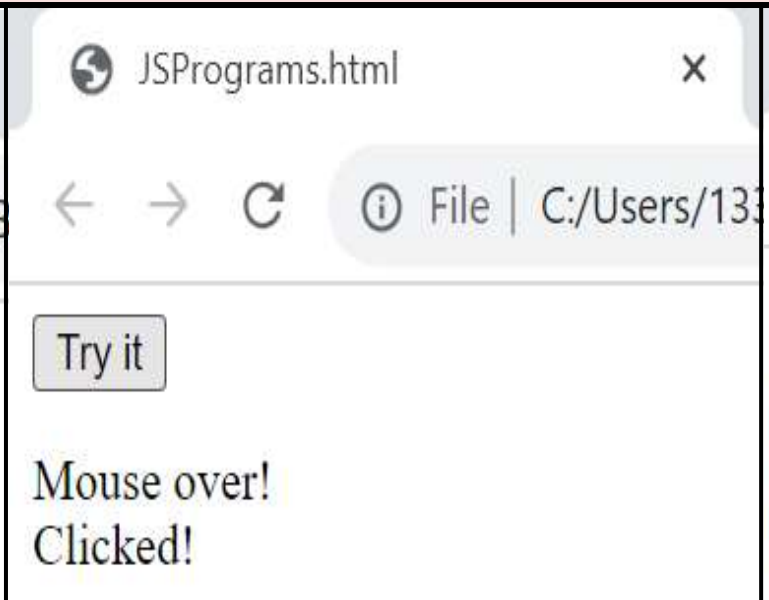
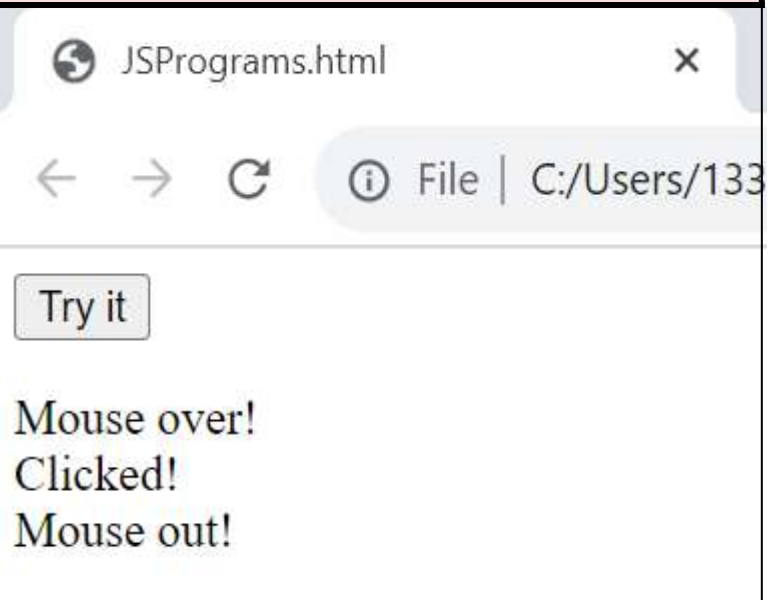
- The **addEventListener()** method allows you to add many events to the same element.
- We can add events of different types to the same element.

```
<!DOCTYPE html>
<html>
<body>
<button id="myBtn">Try it</button>
<p id="demo"></p>
<script>
var x = document.getElementById("myBtn");
x.addEventListener("mouseover", myFunction);
x.addEventListener("click", mySecondFunction);
x.addEventListener("mouseout", myThirdFunction);
```

DOM(Document Object Model)

HTML DOM Event Listener : Example #2

```
function myFunction() {
document.getElementById("demo").innerHTML += "Mouse over!<br>"; }
function mySecondFunction() {
document.getElementById("demo").innerHTML += "Clicked!<br>"; }
function myThirdFunction() {
document.getElementById("demo").innerHTML += "Mouse out!<br>";
}
</script>
</body>
</html>
```

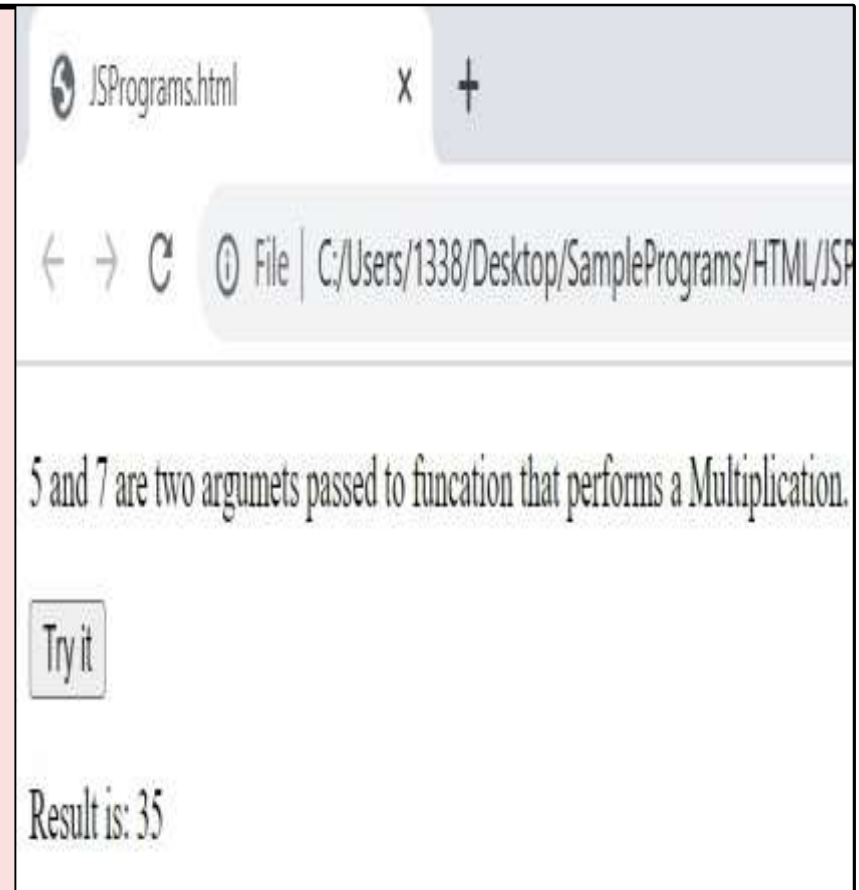
		
--	---	--

DOM(Document Object Model)

HTML DOM Event Listener : Passing parameters : Example #3

- When passing parameter values, use an "anonymous function" that calls the specified function with the parameters.

```
<!DOCTYPE html>
<html>
<body>
<p>5 and 7 are two argumets passed to funcation that
  performs a Multiplication.</p>
<button id="myBtn">Try it</button>
<p id="demo"></p>
<script>
let p1 = 5;
let p2 = 7;
document.getElementById("myBtn").addEventListener("click", function() {
  myFunction(p1, p2);});
function myFunction(a, b) {
document.getElementById("demo").innerHTML = "Result is: "+(a * b);
}
</script>
</body>
</html>
```



DOM(Document Object Model)

HTML DOM Event Listener : Bubbling : Example #4

- Event bubbling is **a method of event propagation** in the HTML DOM API when an event is in an element inside another element, and both elements have registered a handle to that event.

```
<!DOCTYPE html>
<html>
<head>

<style>
#myDiv1 {
background-color: coral;
padding: 50px;
}
#myP1 {
background-color: white;
font-size: 20px;
border: 1px solid;
padding: 20px;
}
</style>

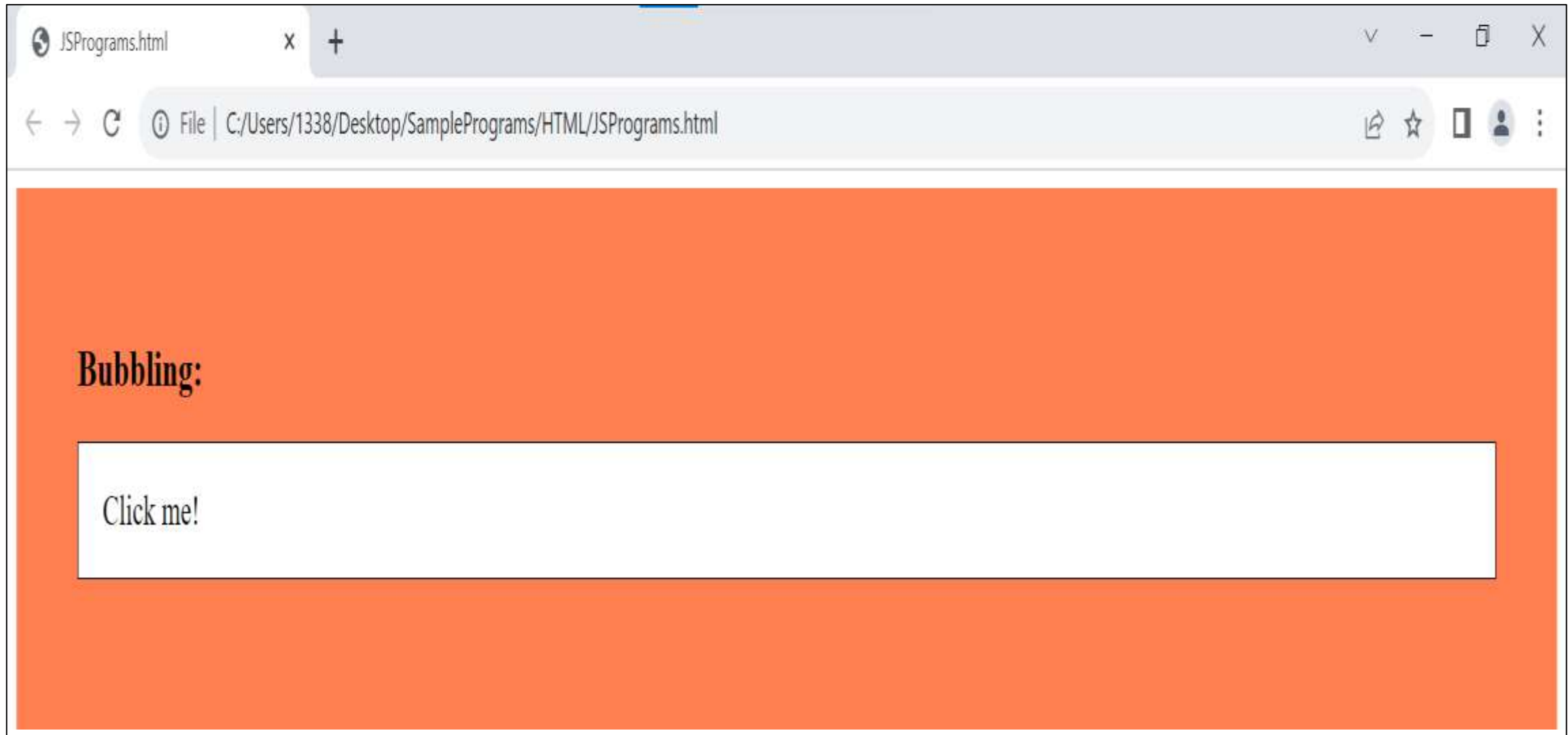
</head>
```

HTML DOM Event Listener : Bubbling : Example #4

```
<body>
<div id="myDiv1">
  <h2>Bubbling:</h2>
  <p id="myP1">Click me!</p>
</div><br>
<script>
document.getElementById("myP1").addEventListener("click", function() {
alert("You clicked the white element!"); });
document.getElementById("myDiv1").addEventListener("click", function() {
alert("You clicked the orange element!"); });
</script>
</body>
</html>
```

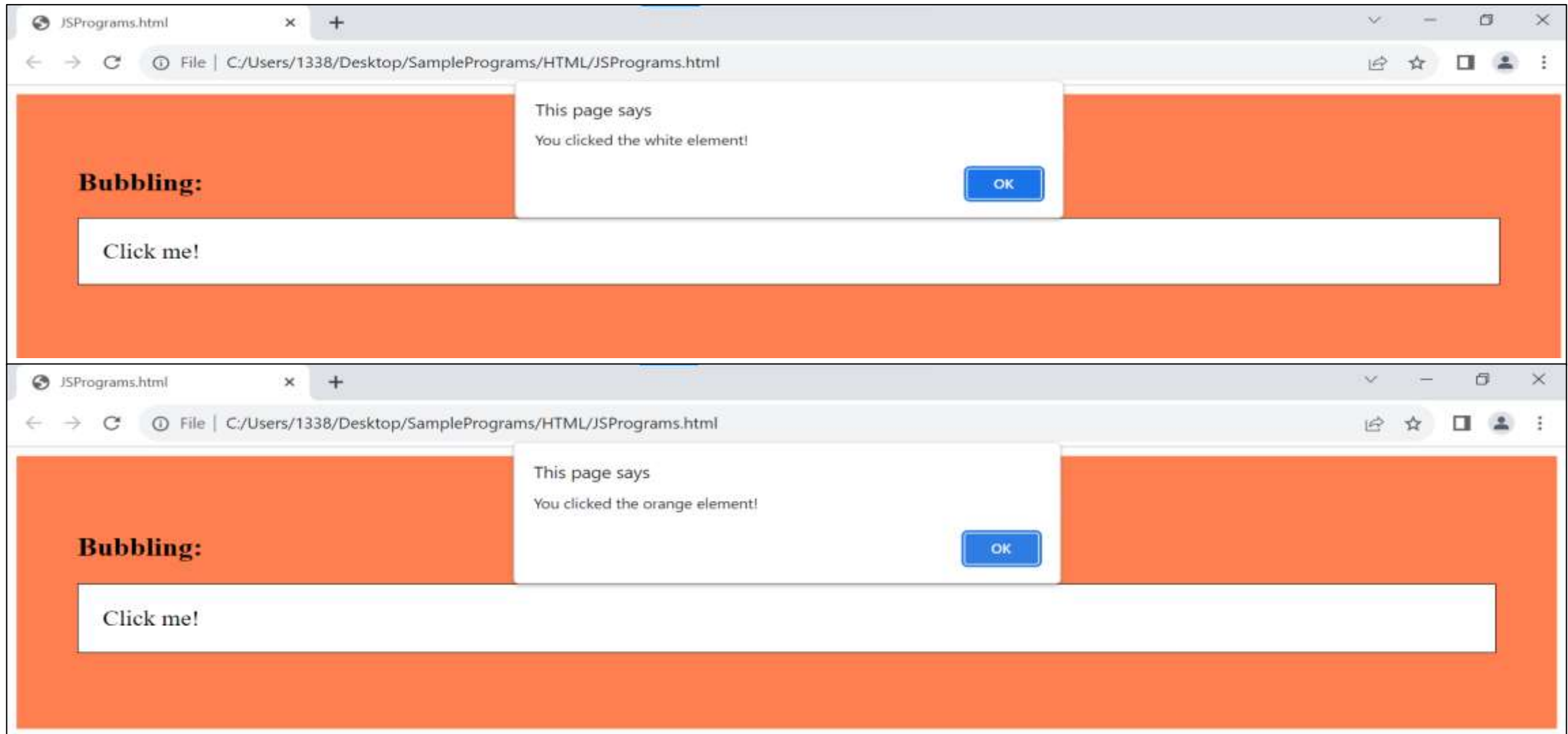
DOM(Document Object Model)

HTML DOM Event Listener : Bubbling : Example #4



DOM(Document Object Model)

HTML DOM Event Listener : Bubbling : Example #4



DOM(Document Object Model)

HTML DOM Collections : Example #1

- An HTMLCollection object is an **array-like list (collection)** of HTML elements.
- The **getElementsByTagName()** method returns an HTMLCollection object.
- The following code selects all **<p>** elements in a document.
- The **length** property defines the **number of elements in an HTMLCollection**.

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript HTML DOM</h2>
<p>Hello World!</p>
<p>Hello Norway!</p>
<p id="demo"></p>
<p id="demo1"></p>
<button onclick="myFunction()">Try it</button>
```

HTML DOM Collections : Example #1

```
<script>
const myCollection = document.getElementsByTagName("p");
document.getElementById("demo").innerHTML = "This document contains " + myCollection.length + " paragraphs.";
document.getElementById("demo1").innerHTML = "The innerHTML of the second paragraph is: " +
    myCollection[1].innerHTML;

function myFunction() {
for (let i = 0; i < myCollection.length; i++) {
    myCollection[i].style.color = "red";
}
}

</script>
</body>
</html>
```

DOM(Document Object Model)

HTML DOM Collections : Example #1

 JSPrograms.html x +     File C:/Users/1338/Desktop/SampleProgram JavaScript HTML DOM Hello World! Hello Norway! This document contains 4 paragraphs. The innerHTML of the second paragraph is: Hello Norway! Try it	 JSPrograms.html x +     File C:/Users/1338/Desktop/SampleProgram JavaScript HTML DOM Hello World! Hello Norway! This document contains 4 paragraphs. The innerHTML of the second paragraph is: Hello Norway! Try it
--	---

HTML DOM Collections : Example #2

```
<!DOCTYPE html>
<html>
<body>

<form id="frm1" action="">
First name: <input type="text" name="fname" value="Sachin"><br>
Last name: <input type="text" name="lname" value="Tendulkar"><br><br>
<input type="submit" value="Submit">
</form>

<p>Click "Display" to display the value of each element in the form.</p>
<button onclick="myFunction()">Display</button>
<p id="demo"></p>
```

HTML DOM Collections : Example #2

```
<script>
function myFunction() {
    var x = document.getElementById("frm1");
    var text = "";
    var i;
    for (i = 0; i < x.length ;i++) {
        text += x.elements[i].value + "<br>";
    }
    document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>
```

DOM(Document Object Model)

HTML DOM Collections : Example #2

JSPrograms.html

←

→

↻

File | C:/Users/1338/Desktop/SampleProgram

First name:
 Last name:

Submit

Click "Display" to display the value of each element in the form.

Display

JSPrograms.html

←

→

↻

File | C:/Users/1338/Desktop/SampleProgram

First name:
 Last name:

Submit

Click "Display" to display the value of each element in the form.

Display

Sachin
 Tendulkar
 Submit

DOM(Document Object Model)

The DOM CSS

Changing HTML Style

- To change the style of an HTML element, use this syntax:

```
document.getElementById(id).style.property = new style
```

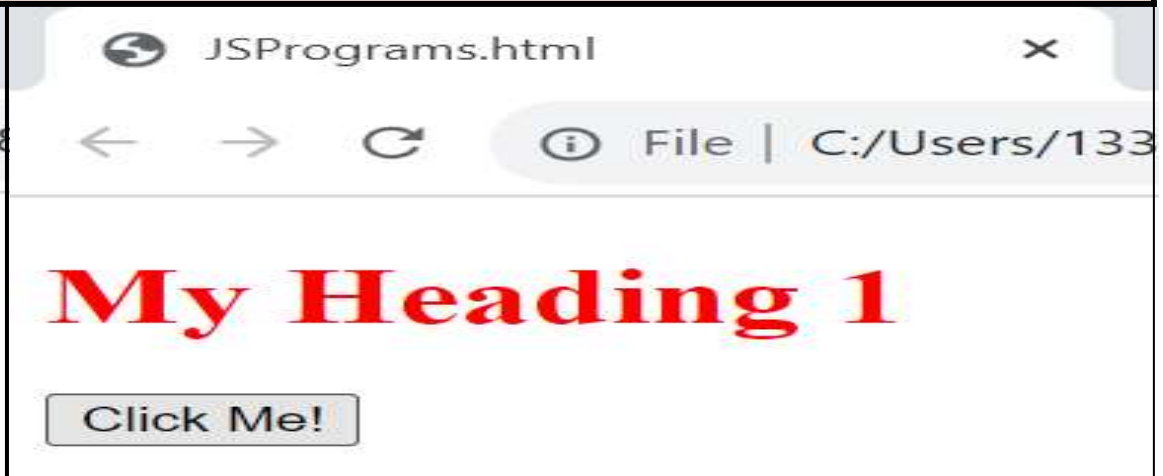
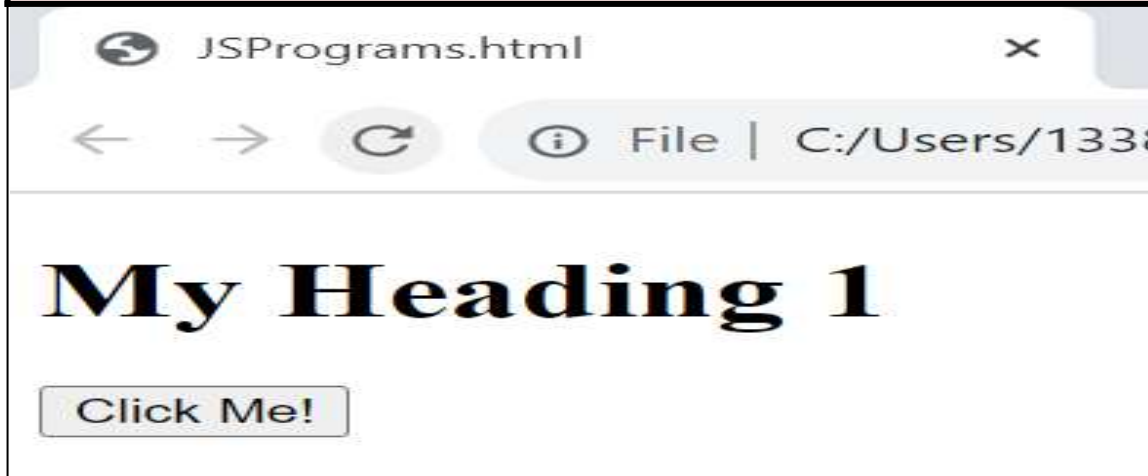
Using Events

- The HTML DOM allows you to execute code when an event occurs.
- Events are generated by the browser when "things happen" to HTML elements:
 - An element is clicked on
 - The page has loaded
 - Input fields are changed

DOM(Document Object Model)

The DOM CSS : Example #1

```
<!DOCTYPE html>
<html>
<body>
<h1 id="id1">My Heading 1</h1>
<button type="button" onclick="document.getElementById('id1').style.color = 'red'">
Click Me!</button>
</body>
</html>
```



The DOM CSS : Example #2

```
<!DOCTYPE html>
<html>
<head>
<script>
function removeElement() {
    document.getElementById("imgbox1").style.display = "none";
}

function changeVisibility() {
    document.getElementById("imgbox2").style.visibility = "hidden";
}

function resetElement() {
    document.getElementById("imgbox1").style.display = "block";
    document.getElementById("imgbox2").style.visibility = "visible";
}
</script>
```

The DOM CSS : Example #2

```
<style>
```

```
.imgbox {
```

```
float: left;
```

```
text-align: center;
```

```
width: 120px;
```

```
height: 145px;
```

```
border: 1px solid gray;
```

```
margin: 4px;
```

```
padding: 0;
```

```
}
```

The DOM CSS : Example #2

```
.thumbnail {  
    width: 110px;  
    height: 90px;  
    margin: 3px;  
}  
.box {  
    width: 110px;  
    padding: 0;  
}  
</style>  
</head>
```


The DOM CSS : Example #2

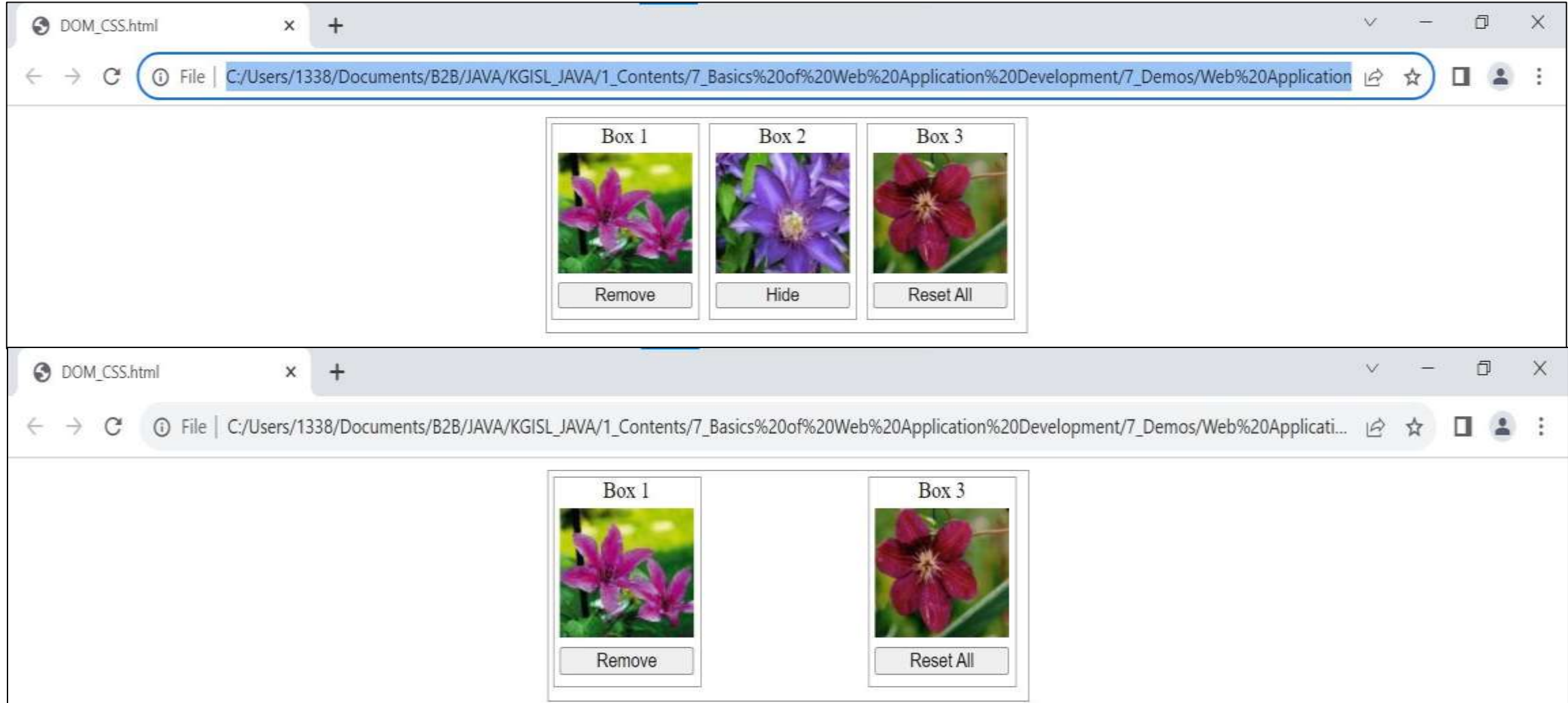
```

<body>
<div style="text-align:center">
  <div style="width:394px;height:160px;margin-left:auto;margin-right:auto;text-align:left;border:1px solid gray;">
    <div class="imgbox" id="imgbox1">Box 1<br>
      
      <input class="box" type="button" onclick="removeElement()" value="Remove">
    </div>
    <div class="imgbox" id="imgbox2">Box 2<br>
      
      <input class="box" type="button" onclick="changeVisibility()" value="Hide">
    </div>
    <div class="imgbox">Box 3<br>
      
      <input class="box" type="button" onclick="resetElement()" value="Reset All">
    </div> </div> </div>
  </body>
</html>

```

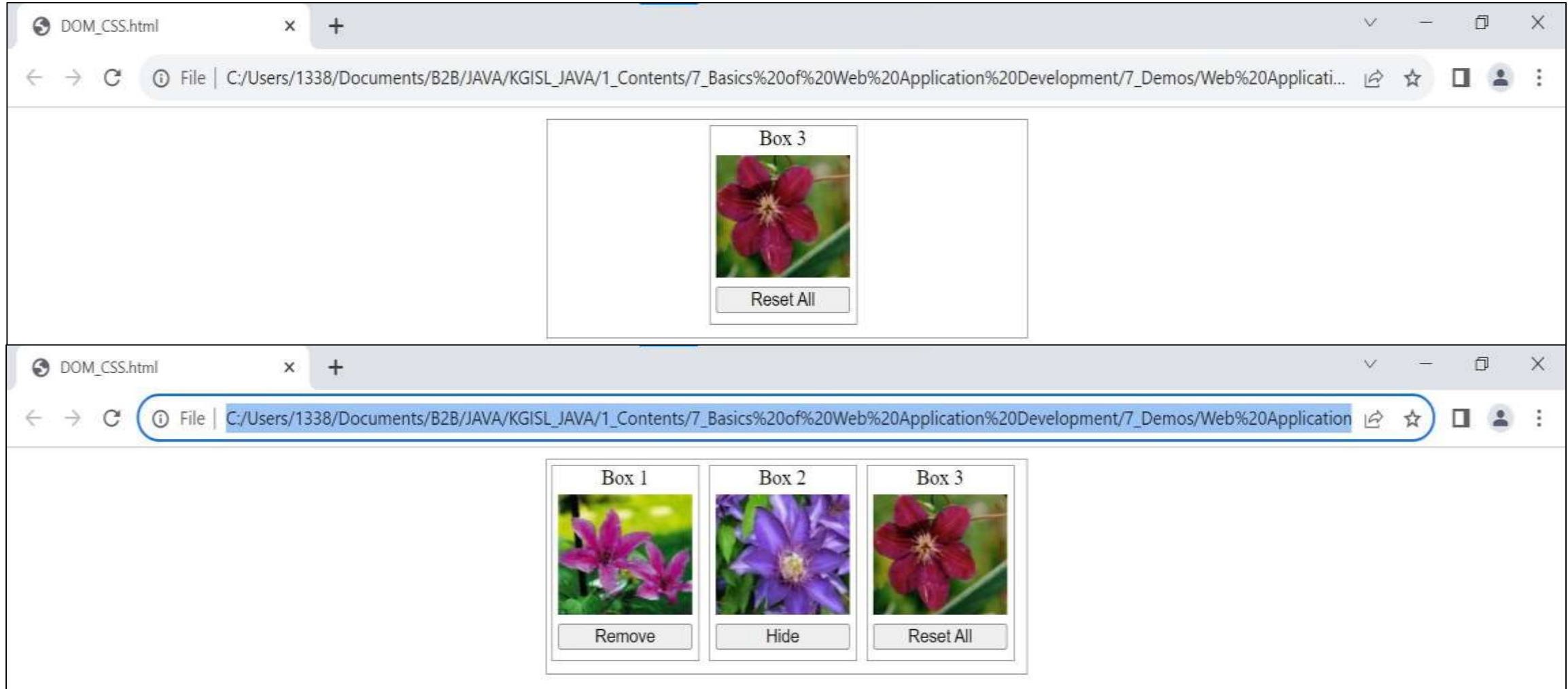
DOM(Document Object Model)

The DOM CSS : Example #2



DOM(Document Object Model)

The DOM CSS : Example #2



The DOM Forms

Data Validation

- Data validation is the process of ensuring that user input is clean, correct, and useful.
- **Typical validation tasks are:**
 - Has the user filled in all required fields?
 - Has the user entered a valid date?
 - Has the user entered text in a numeric field?
- The **purpose** of data validation is to ensure correct user input.
- Validation can be defined by many different methods, and deployed in many different ways.
- **Server-side validation** is performed by a web server after input has been sent to the server.
- **Client-side validation** is performed by a web browser before input is sent to a web server.

The DOM Forms

HTML Constraint Validation

- HTML constraint validation is based on:
 - Constraint validation HTML Input Attributes
 - Constraint validation CSS Pseudo Selectors
 - Constraint validation DOM Properties and Methods

Attribute	Description
disabled	Specifies that the input element should be disabled
max	Specifies the maximum value of an input element
min	Specifies the minimum value of an input element
pattern	Specifies the value pattern of an input element
required	Specifies that the input field requires an element
type	Specifies the type of an input element

The DOM Forms

Automatic HTML Form Validation

- HTML5 enables the features form validation that can be performed automatically by the browser.
- If a form field (fname) is empty, the required attribute prevents this form from being submitted.

```

<!DOCTYPE html>
<html>
<body>
<h2>HTML Validation</h2>
<form action="" method="post">
  <input type="text" name="fname" required>
  <input type="submit" value="Submit">
</form>
</body>
</html>

```

x
JSPrograms.html

File | C:/Users/1338

HTML Validation

!

Please fill out this field.

The DOM Forms

JavaScript Form Validation

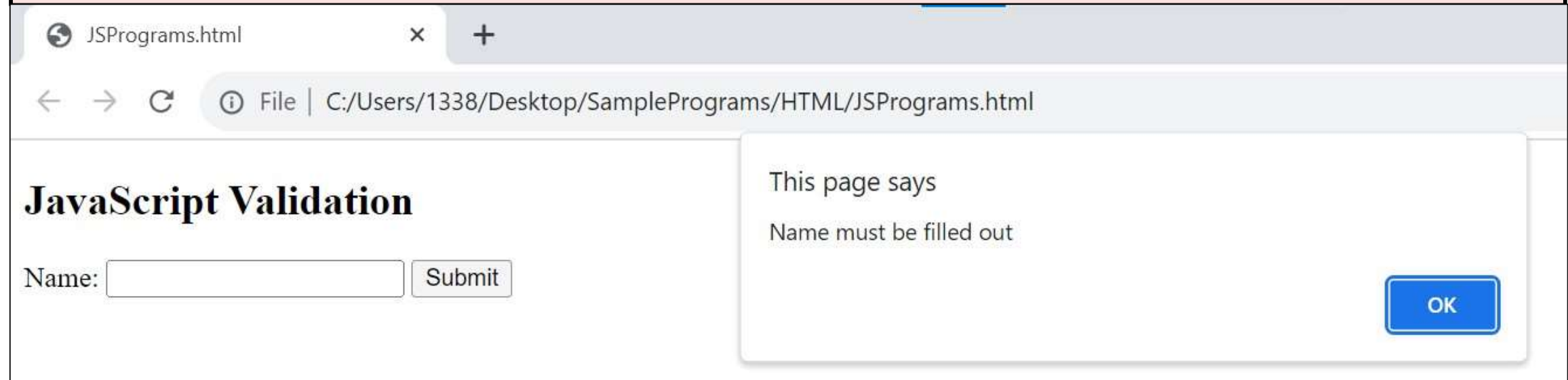
```
<!DOCTYPE html>
<html>
<head>
<script>

function validateForm() {
  let x = document.forms["Form1"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
```

The DOM Forms

JavaScript Form Validation

```
<body>
<h2>JavaScript Validation</h2>
<form name="Form1" action="" onsubmit="return validateForm()" method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
</body>
</html>
```



The screenshot shows a web browser window with the title "JSPrograms.html". The address bar shows the file path "C:/Users/1338/Desktop/SamplePrograms/HTML/JSPrograms.html". The page content displays the heading "JavaScript Validation" and a form with a text input field labeled "Name:" and a "Submit" button. An alert dialog box is open, displaying the message "This page says Name must be filled out" with an "OK" button.

HTML5 Canvas



HTML5 Canvas

- The HTML **canvas** element provides HTML a bitmapped surface to work with.
- It is used to draw graphics on the web page.
- The HTML 5 **<canvas>** tag is used to draw graphics using scripting language like **JavaScript**.
- The **<canvas>** element is only a container for graphics, we must need a scripting language to draw the graphics.
- The **<canvas>** element allows for dynamic and scriptable rendering of 2D shapes and bitmap images.
- Canvas has several methods for drawing paths, boxes, circles, text, and adding images.

HTML5 Canvas

To create a HTML canvas

- A canvas is a rectangle like area on an HTML page.
- It is specified with canvas element.
- By default, the <canvas> element has no border and no content, it is like a container.

```
<canvas id = "mycanvas" width ="200" height ="100"> </canvas>
```

- Always specify an **id** attribute (to be referred to in a script), and a **width** and **height** attribute to define the size of the canvas.
- To add a border, use the **style** attribute.

Canvas

Basic empty canvas

```
<!DOCTYPE html>

<html>

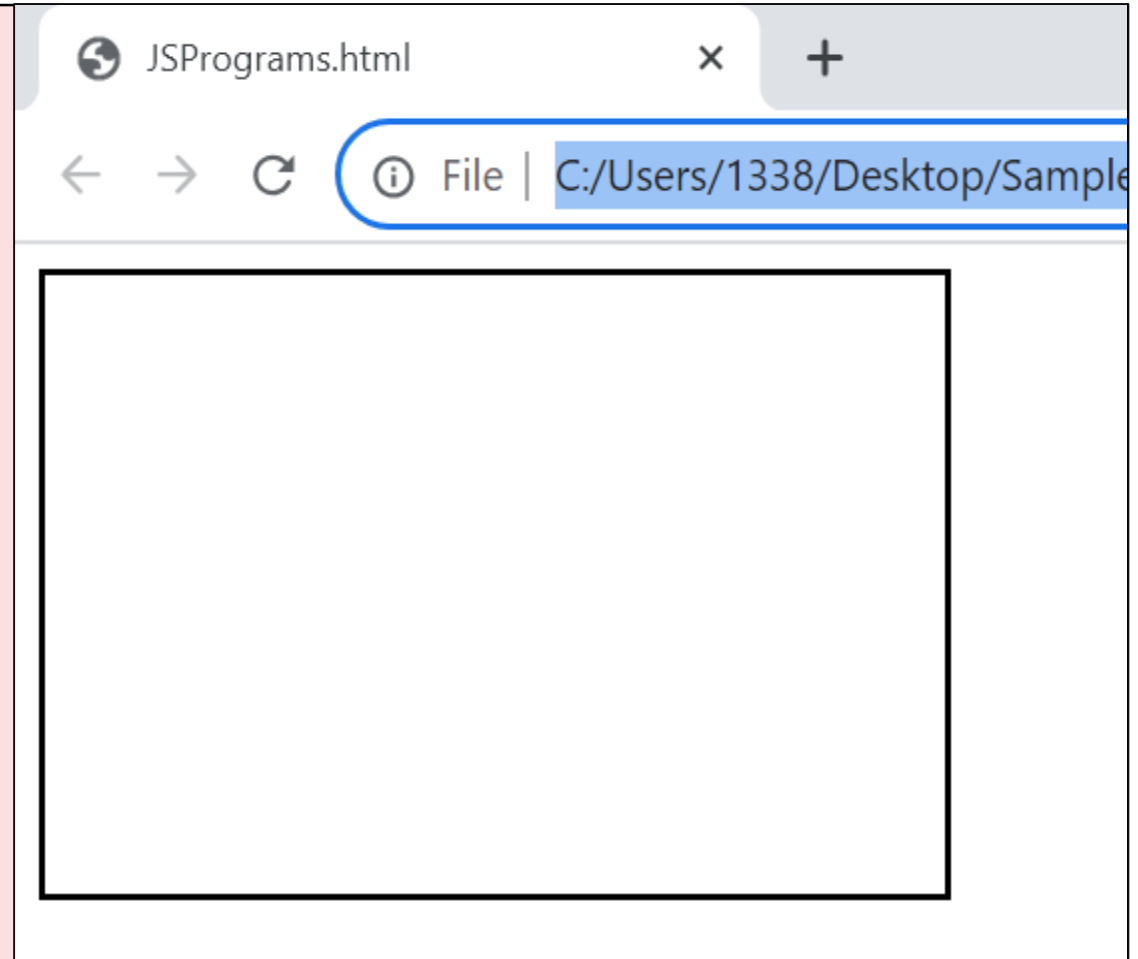
<body>

<canvas id="myCanvas" width="300"
height="200" style="border:2px solid;">

</canvas>

</body>

</html>
```



Canvas

Canvas - Line

- After creating the rectangular canvas area, we must add JavaScript to do the drawing.
- If you want to draw a straight line on the canvas, we can use the following two methods.
 - **moveTo(x,y)**: It is used to define the starting point of the line.
 - **lineTo(x,y)**: It is used to define the ending point of the line.
- If we draw a line whose starting point is (0,0) and the end point is (200,100), use the **stroke method to draw the line.**

Canvas

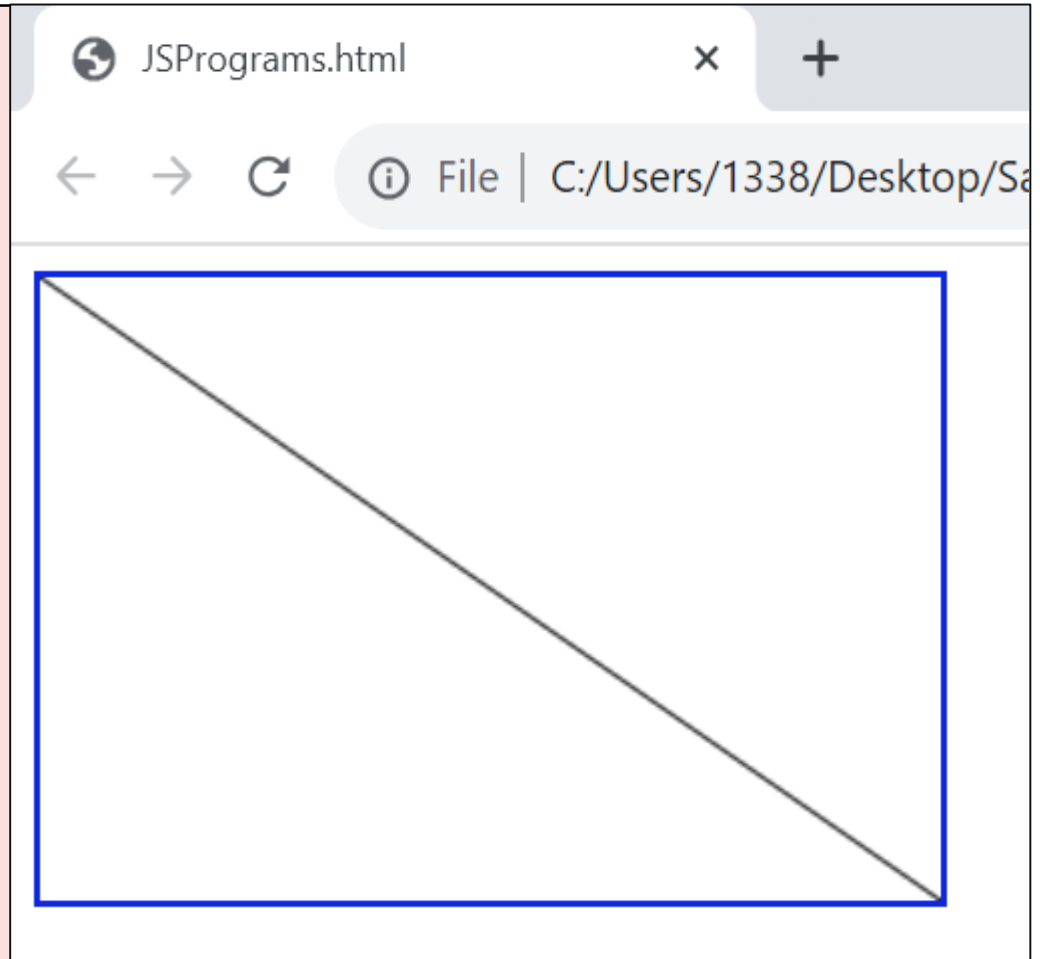
Drawing Line on Canvas

```
<!DOCTYPE html>
<html>
<body>

<canvas id="myCanvas" width="300"
height="200" style="border:2px solid #0f27dd;">
Your browser does not support the HTML canvas tag.
</canvas>

<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.moveTo(0,0);
ctx.lineTo(200,100);
ctx.stroke();
</script>

</body>
</html>
```



Canvas

Canvas - Circle

- If we want to draw a circle on the canvas, we can use the `arc()` method:

`arc(x, y, r, start, stop)`

- To sketch a circle on an HTML canvas, use one of the `ink()` methods, like `stroke()` or `fill()`.

Canvas

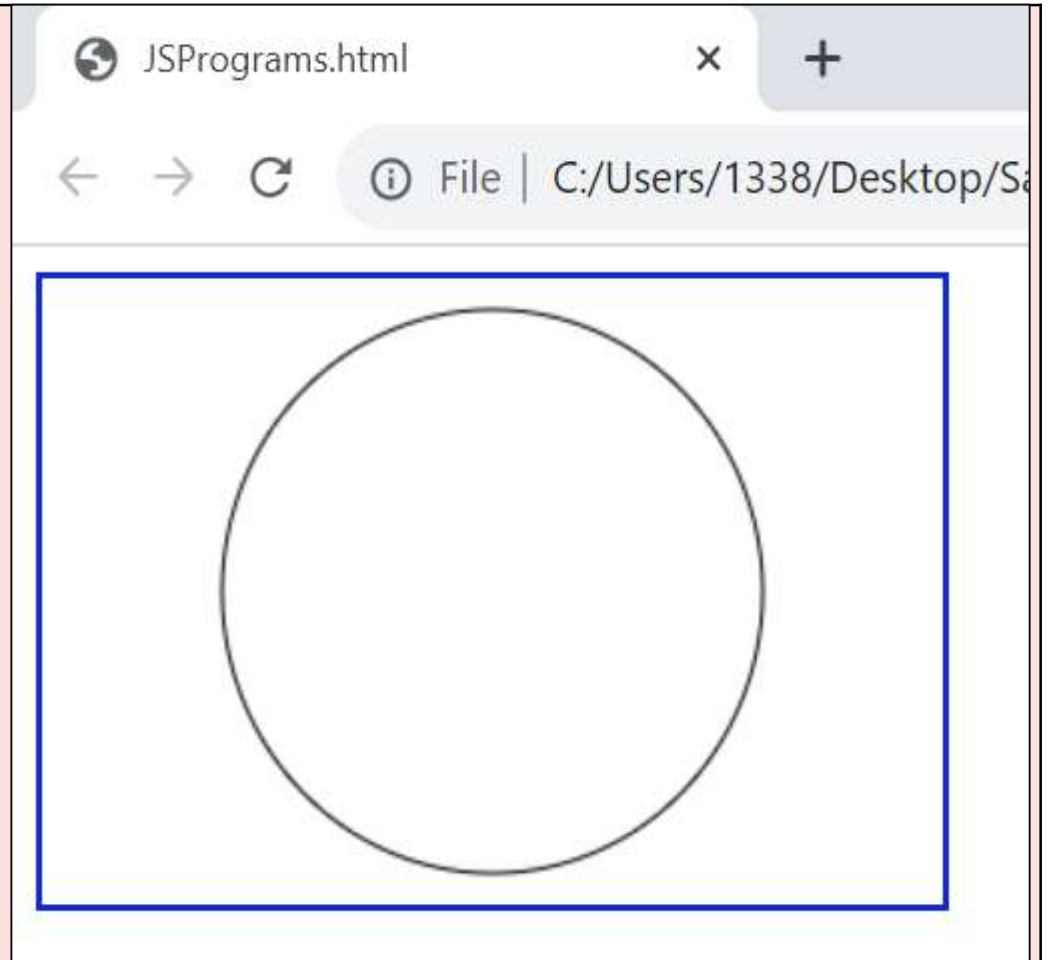
Drawing Circle on Canvas

```
<!DOCTYPE html>
<html>
<body>

<canvas id="myCanvas" width="300"
height="200" style="border:2px solid #0f27dd;">
Your browser does not support the HTML canvas tag.
</canvas>

<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.beginPath();
ctx.arc(150,100,90,0,2*Math.PI);
ctx.stroke();
</script>

</body>
</html>
```



Canvas

Canvas - Text

- There are properties and methods used for drawing text on the canvas.
- **font property:**
 - It is used to define the font property for the text.
- **fillText(text,x,y) method:**
 - It is used to draw filled text on the canvas.
 - It looks like bold font.
- **strokeText(text,x,y) method:**
 - It is also used to draw text on the canvas, but the text is unfilled.

Canvas

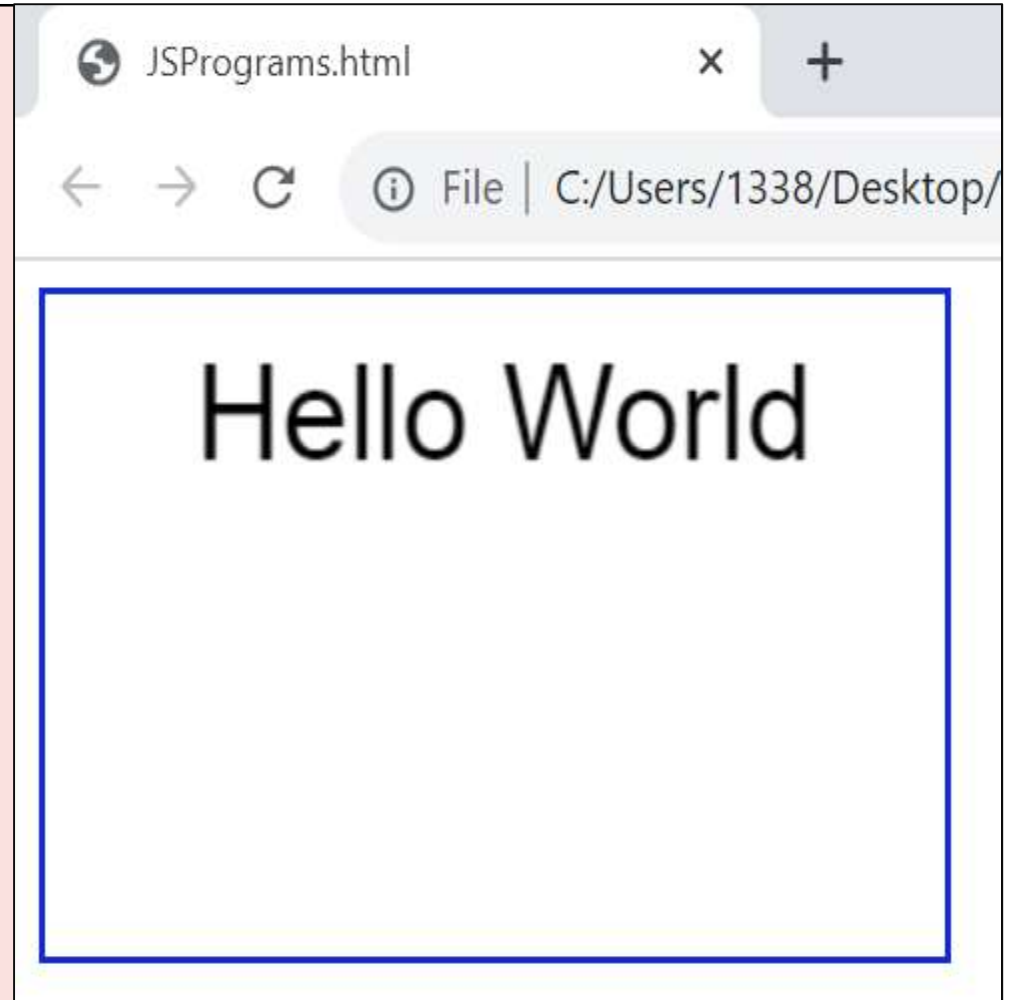
Drawing text on canvas

```
<!DOCTYPE html>
<html>
<body>

<canvas id="myCanvas" width="300"
height="200" style="border:2px solid #0f27dd;">
Your browser does not support the HTML canvas tag.
</canvas>

<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.font = "40px Arial";
ctx.fillText("Hello World",50,50);
</script>

</body>
</html>
```



Canvas

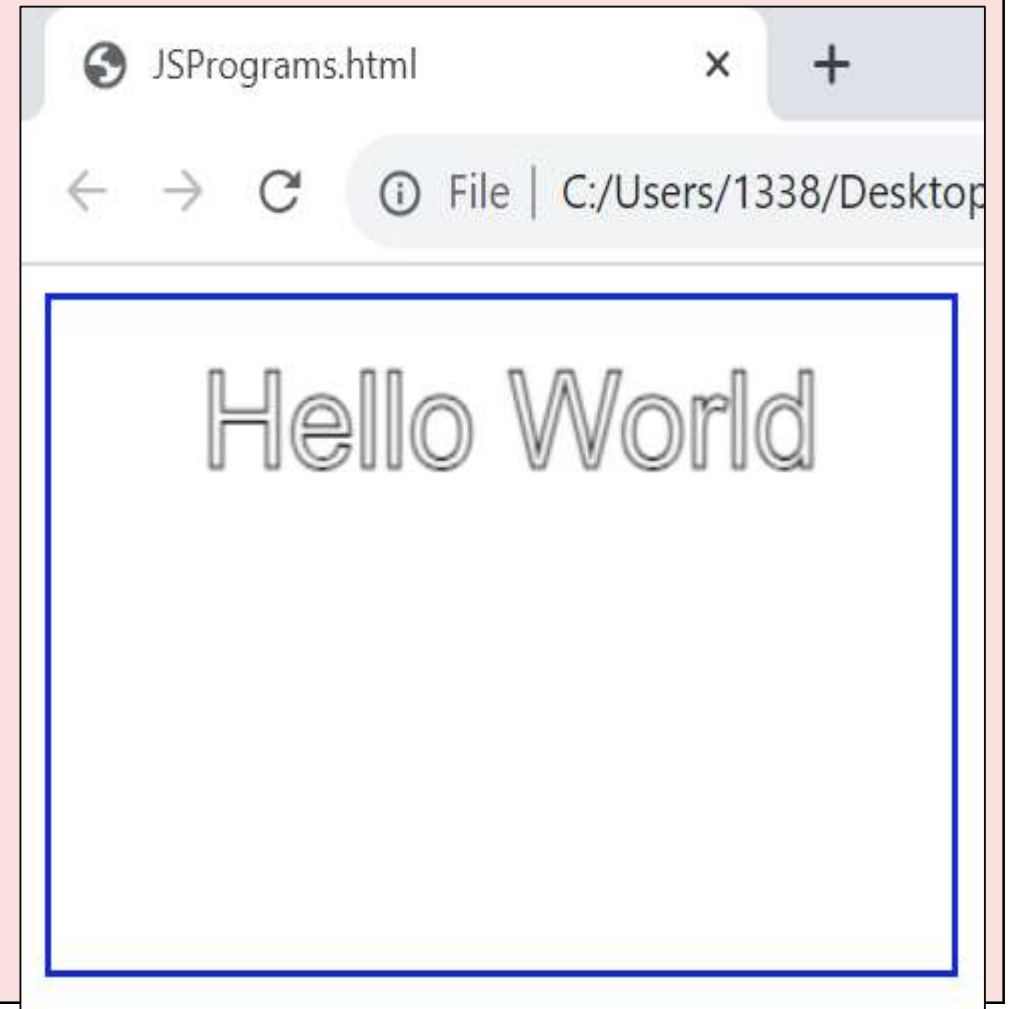
Stroke text on canvas

```
<!DOCTYPE html>
<html>
<body>

<canvas id="myCanvas" width="300"
height="200" style="border:2px solid #0f27dd;">
Your browser does not support the HTML canvas tag.
</canvas>

<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.font = "40px Arial";
ctx.strokeText("Hello World",50,50);
</script>

</body>
</html>
```



Canvas - Gradient

- Gradients can be used to fill rectangles, circles, lines, text, etc.
- Shapes on the canvas are not limited to solid colors.
- There are **two different types** of gradients:
 - **createLinearGradient(x,y,x1,y1)**
 - creates a linear gradient
 - **createRadialGradient(x,y,r,x1,y1,r1)**
 - creates a radial/circular gradient

Canvas - Gradient

- Once we have a gradient object, we must add two or more color stops.
- The **addColorStop()** method specifies the color stops, and its position along the gradient.
- Gradient positions can be anywhere between 0 to 1.
- To use the gradient, set the **fillStyle** or **strokeStyle** property to the gradient.
- Then draw the shape (rectangle, text, or a line).

Canvas

Drawing Linear Gradient on canvas

```
<!DOCTYPE html>

<html>

<body>

  <canvas id="myCanvas" width="300"
  height="200" style="border:2px solid #0f27dd;">

    Your browser does not support the HTML canvas tag.

  </canvas>

<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
```

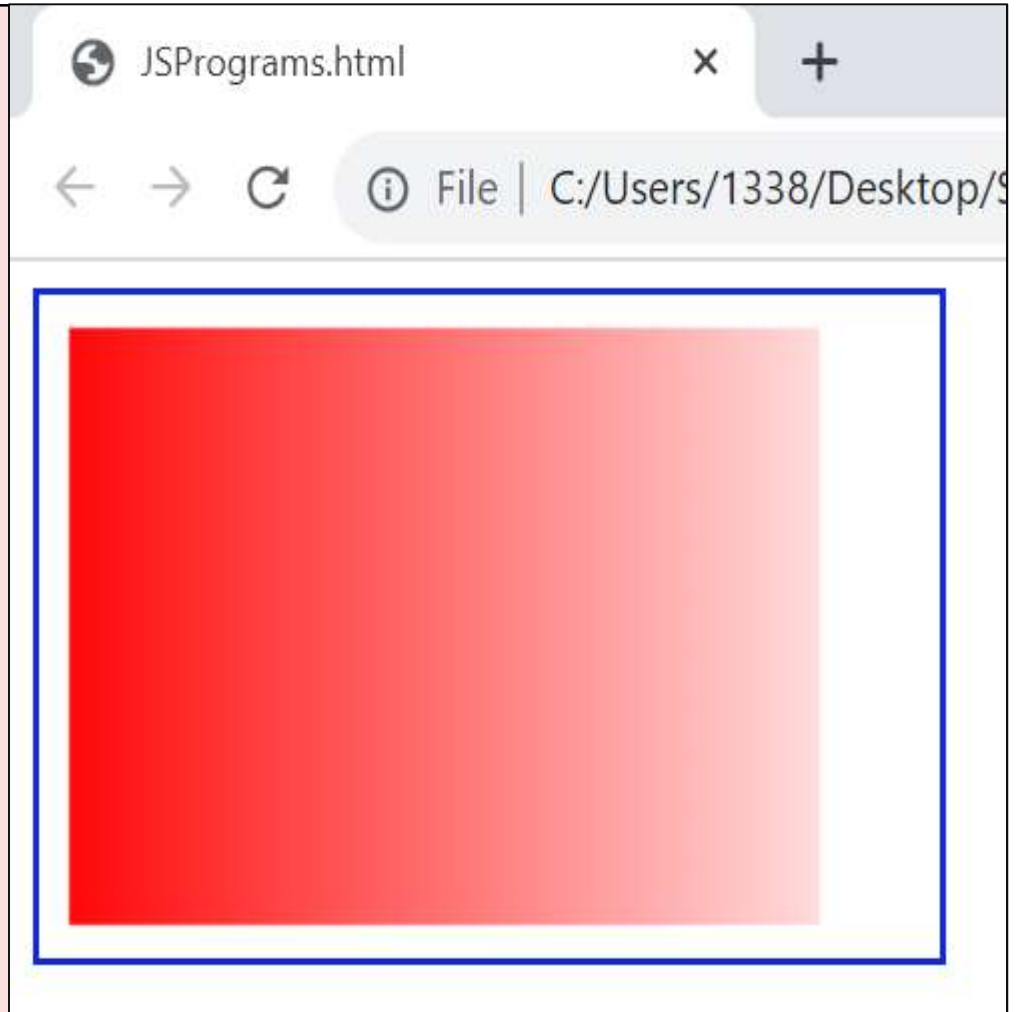
Canvas

Drawing Linear Gradient on canvas

```
// Create gradient
var grd = ctx.createLinearGradient(0,0,300,0);
grd.addColorStop(0,"red");
grd.addColorStop(1,"white");

// Fill with gradient
ctx.fillStyle = grd;
ctx.fillRect(10,10,250,180);

</script>
</body>
</html>
```



Canvas

Drawing Circular Gradient on canvas

```
<!DOCTYPE html>

<html>

<body>

  <canvas id="myCanvas" width="300"
  height="200" style="border:2px solid #0f27dd;">

    Your browser does not support the HTML canvas tag.

  </canvas>

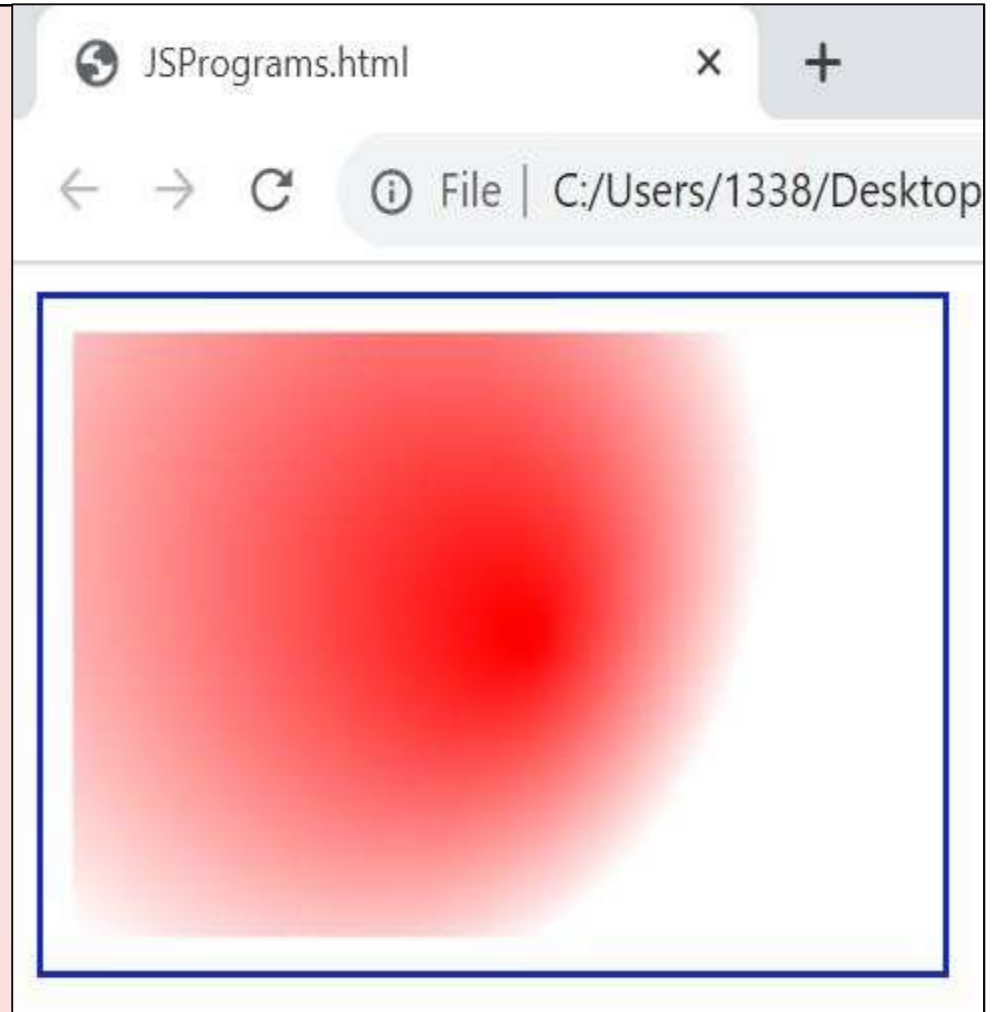
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
```


Canvas

Drawing Circular Gradient on canvas

```
// Create gradient
var grd = ctx.createRadialGradient(160,100,5,90,60,150);
grd.addColorStop(0,"red");
grd.addColorStop(1,"white");
// Fill with gradient
ctx.fillStyle = grd;
ctx.fillRect(10,10,250,180);

</script>
</body>
</html>
```



Canvas

Canvas - Image

- canvas **drawImage()** function is used to display an image or video on canvas.
- This function can be used to display the whole image or just a small part of the image.
- But, the image has to be loaded first to load it further on canvas.

```
context.drawImage(img, x, y, swidth, sheight, sx, sy, width, height);
```

Canvas

Canvas - Image

- **img**: the image or video to draw on canvas.
- **x**: the x-coordinate where the image has to be placed.
- **y**: the y-coordinate where the image has to be placed.
- **swidth**: the width of the clipped image (optional).
- **sheight**: the height of the clipped image (optional).
- **sx**: x-coordinate where to start the clipping (optional).
- **sy**: y-coordinate where to start the clipping (optional).
- **width**: the width of the image to use (optional).
- **height**: the height of the image to use (optional).

Canvas

Drawing Image on canvas

```
<!DOCTYPE html>

<html>

<body>

<p>Image to use:</p>



<p>Canvas:</p>

<canvas id="myCanvas" width="279" height="214" style="border:2px solid #ef0a21;">

Your browser does not support the HTML5 canvas tag.

</canvas>
```

Canvas

Drawing Image on canvas

```
<script>

window.onload = function() {

    var c = document.getElementById("myCanvas");

    var ctx = c.getContext("2d");

    var img = document.getElementById("dog");

    ctx.drawImage(img, 10, 10);

}

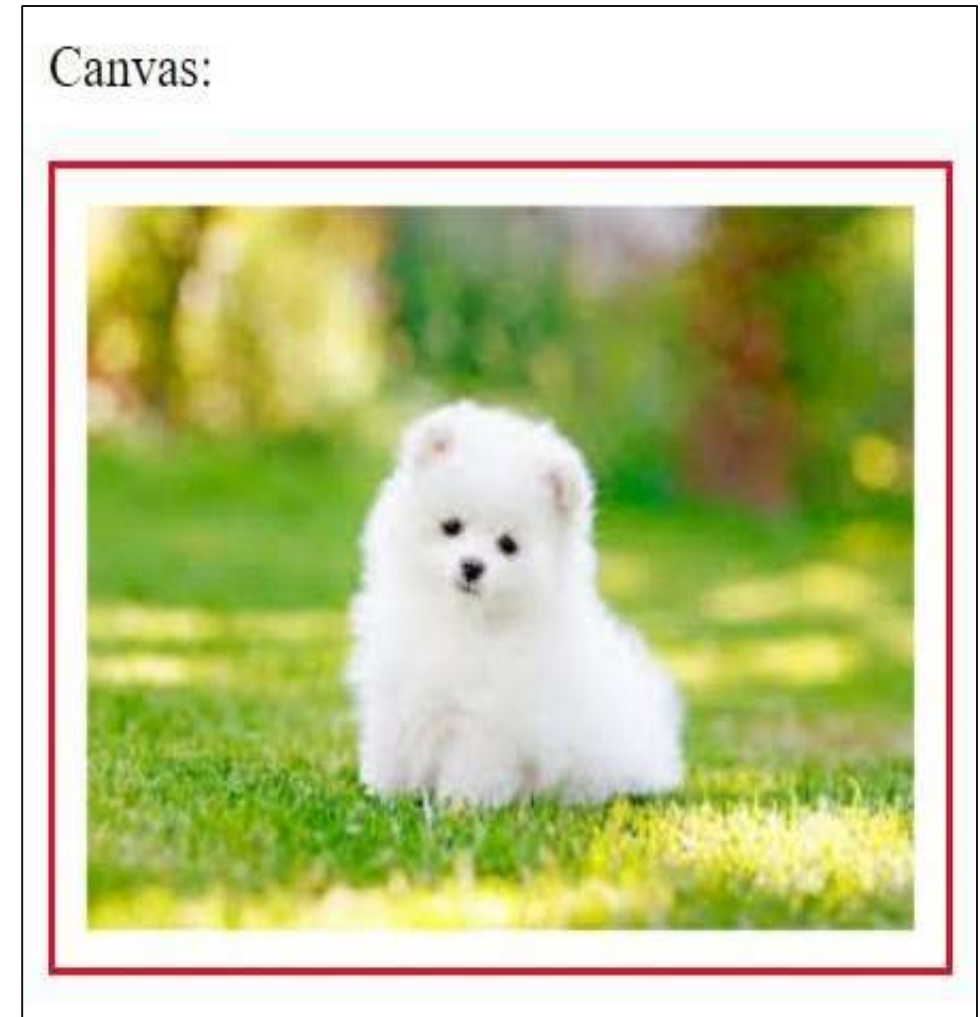
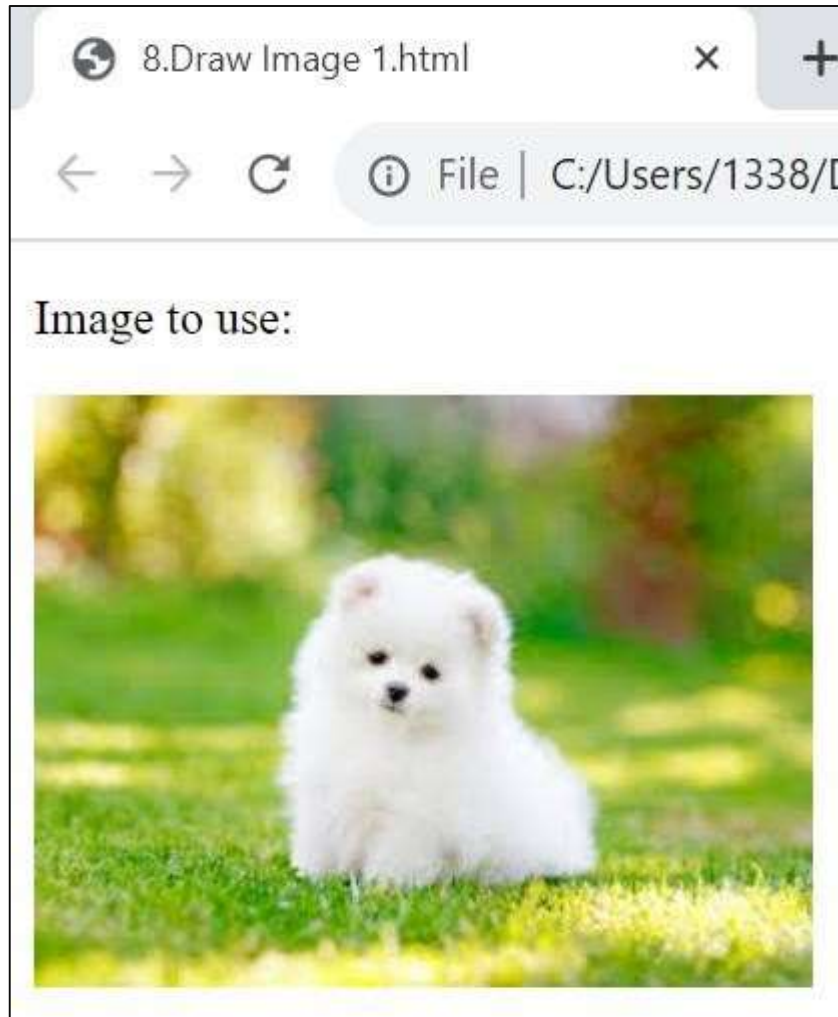
</script>

</body>

</html>
```

Canvas

Canvas - Image



Canvas

Drawing Image on canvas

```
<!DOCTYPE html>

<html>

<body>

<p>Image to use:</p>



<p>Canvas:</p>

<canvas id="myCanvas" width="279" height="214" style="border:2px solid #ef0a21;">

Your browser does not support the HTML5 canvas tag.

</canvas>
```

Canvas

Drawing Image on canvas

```
<script>

window.onload = function() {

    var c = document.getElementById("myCanvas");

    var ctx = c.getContext("2d");

    var img = document.getElementById("dog");

    ctx.drawImage(img, 10, 10, 180, 180);

}

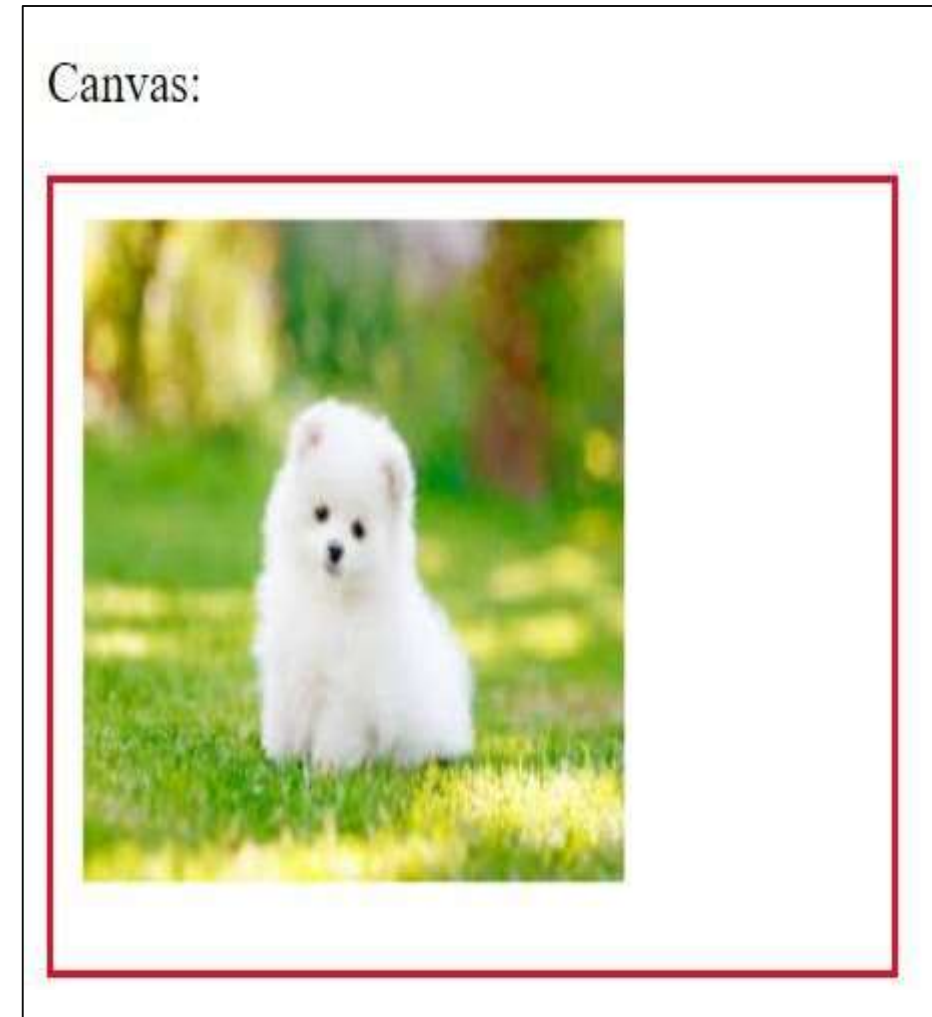
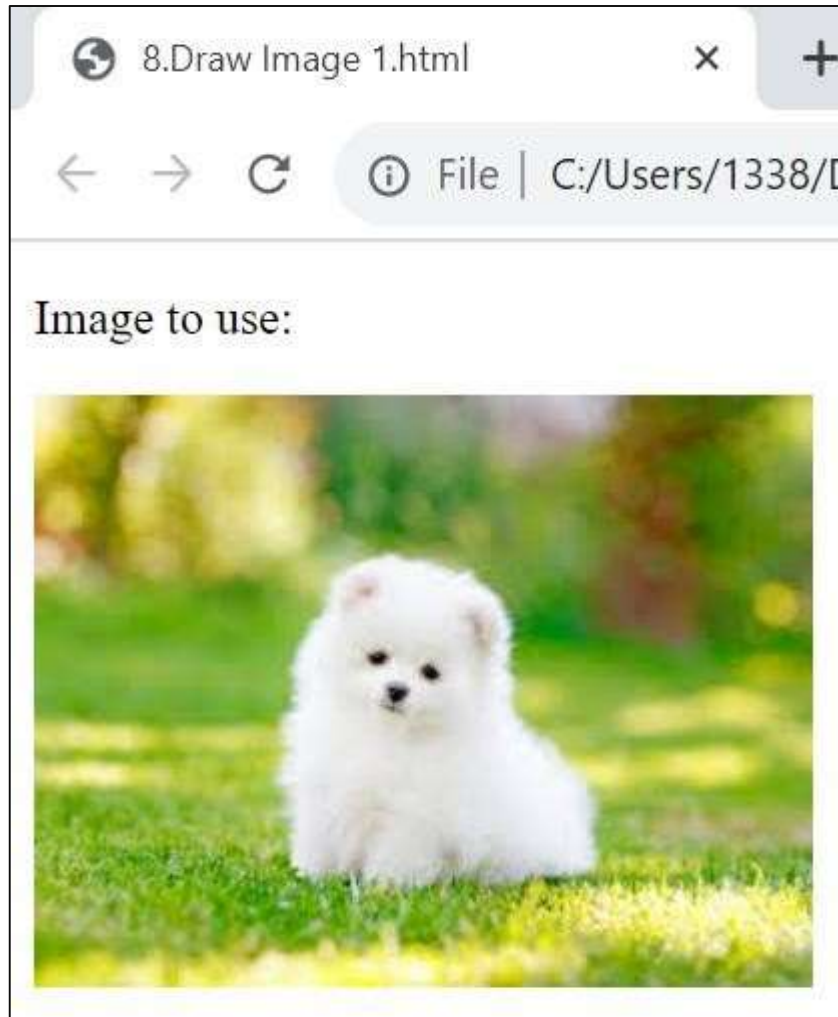
</script>

</body>

</html>
```


Canvas

Canvas - Image



Canvas

Drawing Image on canvas

```
<!DOCTYPE html>

<html>

<body>

<p>Image to use:</p>



<p>Canvas:</p>

<canvas id="myCanvas" width="279" height="214" style="border:2px solid #ef0a21;">

Your browser does not support the HTML5 canvas tag.

</canvas>
```

Canvas

Drawing Image on canvas

```
<script>

window.onload = function() {

    var c = document.getElementById("myCanvas");

    var ctx = c.getContext("2d");

    var img = document.getElementById("dog");

    ctx.drawImage(img, 100,60,50,60,10,10,50,60);

}

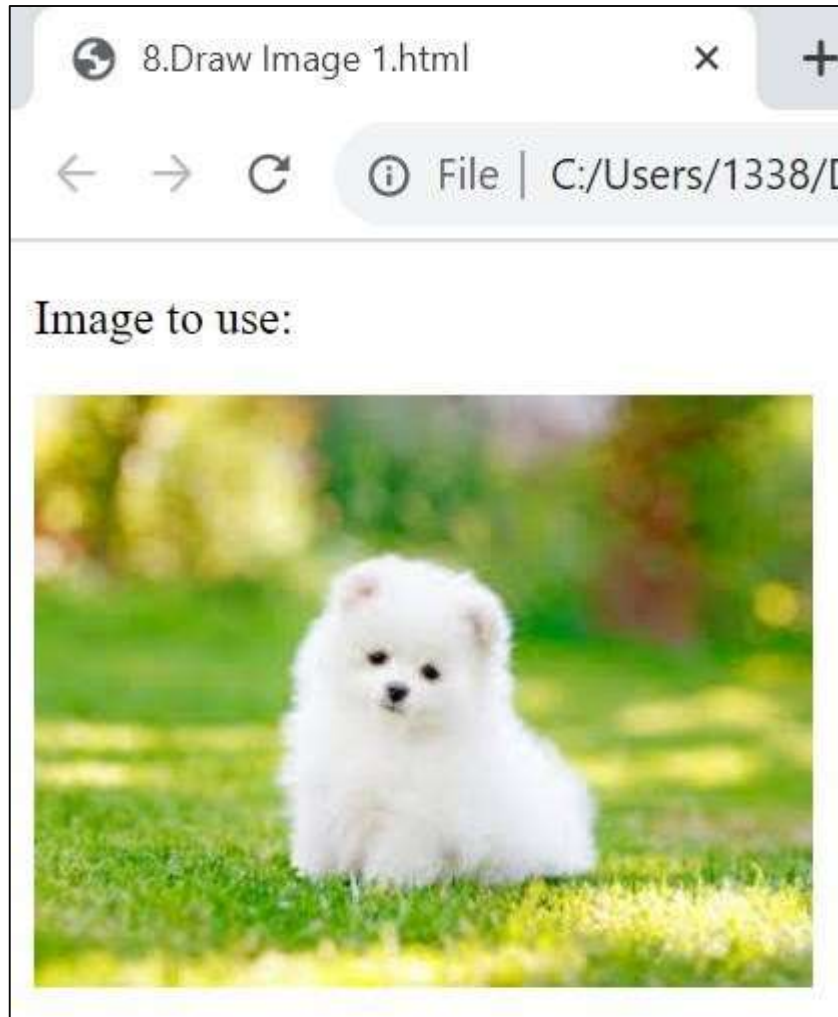
</script>

</body>

</html>
```

Canvas

Canvas - Image



Functions – A Deep Dive



Anonymous Function

- Usually, while defining a function, we specify a name, sometimes parameters, and then whatever we want the function to do in form of expressions.
- **Anonymous Functions** are functions that are **not given any name or identity**. In other words, they are not assigned with identifiers.
- In JavaScript, Anonymous functions are **created at run time**.

Syntax:

```
(function (parameters){  
    //body of the function  
});
```

Anonymous Function

- It is important to put the function in a bracket() followed by a semi-colon.
- It is mandatory to use a semicolon because it **returns an object**. Otherwise, it generates a run time error.

```
function (){  
  console.log("Anonymous Function");  
}
```

✖ Uncaught SyntaxError: Function statements require a function name

Calling Anonymous Functions

- Anonymous functions can only be called as and when declared without any hassle, they are preferred in situations where the functions need to be executed as soon as it is declared.

```
(function () {  
    console.log("Anonymous Function");  
})();
```

- Because the functions were kept inside the brackets(), it returned an object which is used in the function call.

Immediately Invoked Function Expressions

- An **IIFE** (Immediately Invoked Function Expression) is a JavaScript function that runs as soon as it is defined.
- It is a design pattern which is also known as a **Self-Executing Anonymous Function**.

```
for (var i = 1; i <= 3; i++) {  
  (function (counter) {  
    setTimeout(function () {  
      console.log(`Counted till ${counter} after ${counter} seconds`);  
    }, 1000 * i);  
  })(i);  
}
```

```
Counted till 1 after 1 seconds  
Counted till 2 after 2 seconds  
Counted till 3 after 3 seconds
```

First Class Functions

- Functions lie at the heart of JavaScript, serving as the building blocks that make this versatile programming language a powerhouse for web development.
- This unique characteristic of JavaScript functions empowers developers to write clean, modular, and expressive code.
- Functions can encapsulate logic, making it reusable and maintainable.
- In JavaScript, functions are not just pieces of code that perform tasks; they are **first-class citizens** of the language.

First Class Functions

- First-class functions, also known as first-class citizens or first-class objects, are a fundamental concept in programming languages.
- They refer to functions that **can be treated in the same way as any other data type**, such as numbers, strings, or objects.
- In JavaScript, this means that functions can be assigned to a variable, passed as arguments, and returned from a function.
- These abilities makes a function more powerful and leads to achieve great features of the JavaScript language.

Properties of First-Class Functions

- **Functions as Values:** Functions are treated as values, which means they can be assigned to variables just like any other data type.
- **Function Expressions:** Functions can be defined as expressions and assigned to variables. This includes **anonymous** functions and **arrow** functions.
- **Passing Functions as Arguments:** Functions can be passed as arguments to other functions. This is a fundamental concept for **callback** functions and event handling.
- **Functions as Return Values:** Functions can be returned as the result of another function. This is essential for creating **higher-order functions**.

Properties of First-Class Functions

- Assigning Functions to a Variable

```
// Assigning a function expression to a variable
```

```
var greet = function(name) {  
    console.log(`Hello, ${name} !`);  
}
```

```
//Calling the function with the variable name
```

```
greet('Smith');
```

Properties of First-Class Functions

- Passing Functions as Arguments

```
function add(a, b) {  
    return a + b;  
}  
  
function subtract(a, b) {  
    return a - b;  
}  
  
function calculate(operation, num1, num2) {  
    return operation(num1, num2);  
}  
  
//Passing the function as argument  
console.log(calculate(add, 5, 10));           //15  
  
console.log( calculate(subtract, 10, 5));      //5
```

Properties of First-Class Functions

- Returning Functions from Functions

```
function multiplier(factor) {  
  return function(x){  
    return x * factor;  
  }  
}  
  
const double = multiplier(2);  
const triple = multiplier(3);  
  
console.log(double(5));           // 10  
console.log(triple(5));           // 15
```

Higher-Order Functions

- A higher-order function is a function that either:
 - Takes one or more functions as arguments, or
 - Returns a function as its result.
- In essence, a higher-order function operates on other functions, either by using them to perform tasks or by producing new functions with customized behavior.
- Higher-order functions are a powerful concept in JavaScript and functional programming, enabling code that is more concise, modular, and expressive.

Characteristics of Higher-Order Functions

- **Function as an Argument:** Higher-order functions can accept other functions as arguments, often referred to as callback functions. They are used for data processing, filtering, or handling events.
- **Function as a Return Value:** Higher-order functions can return another function, allowing for the creation of specialized functions tailored to specific tasks.
- **Abstraction:** Higher-order functions abstract common patterns and operations, promoting code reusability and modularity.
- **Composition:** Multiple higher-order functions can be combined to create complex data transformations or processing pipelines.

Higher-Order Functions Example

//The **map** and **filter** functions takes other function as argument

```
function doubleElements(x){
```

```
    return 2*x;
```

```
}
```

```
let DoubleResult=[1,2,3,4,5,6].map(doubleElements)
```

```
function filterElemLesThanTwo(x){
```

```
    return x<2
```

```
}
```

```
let ResultLessThanTwo=[1,2,3,4,5,6].filter(filterElemLesThanTwo)
```

```
console.log(DoubleResult)
```

```
console.log(ResultLessThanTwo)
```

Function Composition

- Function composition is the process of chaining together multiple functions to form a new function.
- It involves applying a series of transformations or operations to an input value, where the output of one function becomes the input of the next function in the composition chain.
- To harness the power of function composition, we need to define the compose function. As a simple implementation, the compose function accepts any number of functions as arguments and returns a new function that iterates over the functions in reverse order using **reduceRight()** of Array.
- The '**compose()**' function takes in two or more functions and returns a new function that applies these functions in **right-to-left** order.

Function Composition – Example

```
const compose = (...functions) => {  
  return (input) => {  
    return functions.reduceRight(function (acc,  
fn) {  
      return fn(acc);  
    }, input);  
  };  
};
```

```
const add5 = (x) => x + 5;  
const multiplyBy3 = (x) => x * 3;  
const subtract10 = (x) => x - 10;  
const composedFunction = compose(subtract10,  
multiplyBy3, add5);  
const result = composedFunction(7);  
  
console.log(result);
```

Currying

- Currying is a technique used in functional programming that allows you to **transform a function** with multiple arguments into a sequence of functions, **each taking only one argument** at a time.
- In JavaScript, currying is a powerful tool that can improve code **reusability**, **composability**, and **maintainability**.

```
function add(x) {  
  return function(y) {  
    return x + y;  
  };  
}  
  
console.log(add(3)(4));
```

Pure Functions

- Pure functions are essential for a variety of purposes, including functional programming, reliable concurrency, and many more.
- A pure function is a function which:
- Given the same input, always returns the same output.
- Produces no side effects.
- You can use pure functions to ensure your code is clean, maintainable, and testable.
- These functions are ideal for these tasks because they are predictable and do not modify external states.

Same Input , Same Output

Pure Function

```
function multiply(a, b) {  
  return a * b;  
}
```

```
multiply(2, 3); // returns 6  
multiply(2, 3); // returns 6  
multiply(2, 3); // returns 6
```

Non-pure Function

```
function multiplyRandomNumber(num) {  
  return num * Math.floor(Math.random() * 10);  
}
```

```
multiplyRandomNumber(5); // Unpredictable  
multiplyRandomNumber(5); // Unpredictable  
multiplyRandomNumber(5); // Unpredictable
```

Functions – A Deep Dive

No Side Effects

- A pure function must not produce any side effects.
- A side effect refers to any modification of state or behavior outside the function's scope, such as modifying global variables, console output, network requests, or DOM manipulation.

```
let count = 0;  
function increment() {  
  count++;  
  console.log(count);  
}  
increment(); // Logs 1  
increment(); // Logs 2  
increment(); // Logs 3
```

Non-Pure

```
function increment(count) {  
  return count + 1;  
}  
  
increment(1); // returns 2  
increment(1); // returns 2  
increment(1); // returns 2
```

Pure

Scope

- The scope is the current context of execution in which values and expressions are "visible" or can be referenced.
- If a variable or expression is not in the current scope, it will not be available for use.
- Scopes can be layered in a hierarchy, so that child scopes have access to parent scopes, but not vice versa.
- JavaScript has the following kinds of scopes:
 - **Global scope:** The default scope for all code running in script mode.
 - **Module scope:** The scope for code running in module mode.
 - **Function scope:** The scope created with a function.
 - **Block scope:** The scope created with a pair of curly braces (a block).

Scope Chain

- JavaScript scope is all about space.
- A **scope chain** refers to the unique spaces that exist from the scope where a variable got called to the global scope.
- In the following example, there are four (4) spaces from fullName's invocation scope to its lexical scope (the global scope in this instance).
- JavaScript's scope chain determines the hierarchy of places the computer must go through — one after the other — to find the lexical scope (origin) of the specific variable that got called.

Scope Chain

```
const fullName = "John Smith";    // Define a variable in the global scope
// Define nested functions:
function profile() {
  function sayName() {
    function writeName() {
      return fullName;
    }
    return writeName();
  }
  return sayName();
}
```

- **Note:** The global scope is the last link in JavaScript's scope chain.

Lexical Scope

- Lexical refers to the **definition of things**.
- Anything related to creating words, expressions, or variables is termed lexical.
- **Lexical scope** is the **definition area** of an expression.
- An item's lexical scope is the **place in which the item got created**.
- Another name for lexical scope is **static scope**.
- Lexical scope means definition space — not invocation space.
- Code within an item's lexical scope can access it.

Lexical Scope

```
// Define a variable in the global scope:  
const myName = "JohnSmith";  
  
// Call myName variable from a function:  
function getName() {  
    return myName;  
}
```

- myName's lexical scope is not the function's local scope but the global scope because we defined myName in the global environment.
- An alternative to the lexical scope is the dynamic scope — but it rarely gets used in programming.

Closures

A closure is the combination of a function bundled together (enclosed) with references to its surrounding state (the lexical environment).

- Closures are functions that refer to independent (**free**) variables. In other words, the function defined in the closure ‘remembers’ the environment in which it was created.
- **Free variables** are variables that are neither locally declared nor passed as parameter.
- In JavaScript, closures are created every time a function is created, at function creation time.

Closure – Example 1

```
function makeFunc() {  
  const name = "Smith";  
  function displayName() {  
    console.log(name);  
  }  
  return displayName;  
}  
  
const myFunc = makeFunc();  
myFunc(); // Smith  
  
console.log(myFunc);  
console.dir(myFunc);
```



```
Smith  
  
f displayName() {  
  console.log(name);  
}  
  
▼ f displayName() ⓘ  
  arguments: null  
  caller: null  
  length: 0  
  name: "displayName"  
  ► prototype: {}  
    [[FunctionLocation]]: test.html:8  
  ► [[Prototype]]: f ()  
  ▼ [[Scopes]]: Scopes[3]  
    ► 0: Closure (makeFunc) {name: 'Smith'}  
    ► 1: Script {myFunc: f}  
    ► 2: Global {window: Window, self: Window, document: document, name: '', 1}
```

Closure – Example 2

```
function makeAdder(x) {  
  return function (y) {  
    return x + y;  
  };  
}  
  
const add5 = makeAdder(5);  
const add10 = makeAdder(10);  
  
console.log(add5(2)); // 7  
console.log(add10(2)); // 12  
  
console.log(add5);  
console.dir(add10);
```

```
7  
12  
f (y) {  
  return x + y;  
}  
  
▼ f anonymous(y) ⓘ  
  arguments: null  
  caller: null  
  length: 1  
  name: ""  
  ▶ prototype: {}  
    [[FunctionLocation]]: test.html:7  
  ▶ [[Prototype]]: f ()  
  ▼ [[Scopes]]: Scopes[3]  
    ▶ 0: Closure (makeAdder) {x: 10}  
    ▶ 1: Script {add5: f, add10: f}  
    ▶ 2: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
```


ECMAScript 6 – ES6



Introduction



- ES6 or ECMAScript 2015 is the 6th version of the ECMAScript programming language.
- ECMAScript is the **standardization of JavaScript** which was released in 2015, and subsequently renamed as ECMAScript 2015.
- It was introduced by '**ECMA International**' and is basically an implementation with which we learn how to create a scripting language.
- It allows us to write code in a clever way which basically makes the code more modern and more readable. It's fair to say that with the use of ES6 features we **write less and do more**.

ES6

Why ES6?

- No wonder ES6 is one of the efficient programming languages used for building robust applications.
- You can write short and readable codes with ES6.
- You can quickly build effective programs with fewer codes
- ES6 offers wonderful features such as Arrow functions, destruction methods, modules, block-scoped functions, and many more.
- ES6 uses Object-oriented classes
- The great thing about ES6 is that it is the next-generation JavaScript language.

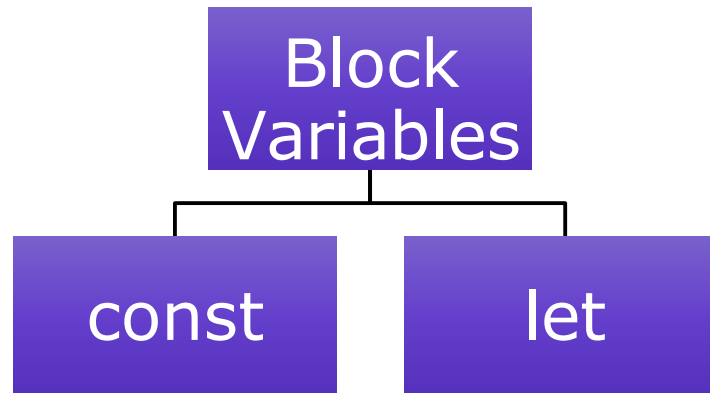


ES6 – New Syntax



New Syntax: Block Variables

- Block-scoped variables are the variables that hold their value until the end of the block only.
- After that, the variables will take the values of the outer block.
- ES6 offers two block-scoped keywords, such as **let** and **const**.
- The variables are declared using **const** and **let** keywords follow block-scoping rules.



Block Variables

- **'let'** Keyword
 - ES6 provides a new way of declaring a block scope variable by using the let keyword.
 - The let keyword is like the var keyword, except scope.

Example:

```
let x = 10;  
if (x == 10) {  
  let x = 20;  
  console.log(x); // 20: reference x inside the block  
}  
console.log(x); // 10: reference at the beginning of the script
```

Block Variable

- **'const'** keyword
 - Like the **let** keyword, the **const** keyword declares blocked-scope variables.
 - However, the block-scoped variables declared by the **const** keyword **can't be reassigned**.
 - The variables declared by the **let** keyword are **mutable**. But the variables created by **const** keyword are **immutable**.

```
const RATE = 0.1;  
RATE = 0.2;           // TypeError  
const person = { age: 20 }; // const with objects  
person.age = 30;      // OK  
console.log(person.age); // 30
```

Default Parameters

- Default parameters are parameters which are given by default while declaring a function.
- But it's value can be changed when calling the function.

Syntax:

```
function fn(param1=default1, param2=default2,..) {  
}
```

Example:

```
function say(message='Hi') {  
  console.log(message);  
}  
say();           // 'Hi'  
say(undefined); // 'Hi'  
say('Hello');    // 'Hello'
```


Rest Parameters

- ES6 provides a new kind of parameter called rest parameter that has a prefix of three dots (...).
- A rest parameter allows you to represent an indefinite number of arguments as an array.

Syntax:

```
function fn(a,b,...args) {  
  //...  
}
```

Example:

```
function sum(...args) {  
  let total = 0;  
  for (const a of args) {  
    total += a;  
  }  
  return total;  
}  
sum(1, 2, 3);
```

Spread Operator

- ES6 provides a new operator called spread operator that consists of three dots (...).
- The spread operator allows you to spread out elements of an iterable object such as an array, map, or set.

Example:

```
const odd = [1,3,5];  
const combined = [2,4,6, ...odd];  
console.log(combined);    // Output: [ 2, 4, 6, 1, 3, 5 ]
```

- **Spread operator Vs Rest Operator**
 - The **spread** operator (...) **unpacks** the elements of an iterable object.
 - The **rest** parameter (...) **packs** the elements into an array.

for...of Loop

- ES6 introduced a new statement for...of that iterates over an iterable object such as:
 - Array, String, Map, Set, ...
 - Gives the values of the objects where as for...in gives the indexes.

Syntax:

```
for (variable of iterable) {  
  // ...  
}
```

Example:

```
let scores = [80, 90, 70];  
  
for (let score of scores) {  
  score = score + 5;  
  console.log(score);      // Prints 85 95 75  
}
```

ES6 – Destructuring



Destructuring: Array Destructuring

- ES6 provides a new feature called destructuring assignment that allows you to destructure properties of an object or elements of an array into individual variables.
- **Array Destructuring:** This method allows destructuring arrays by swapping variables. Otherwise, we need to use temporary variables. Using this method, we can easily assign elements of arrays to variables.

Example:

```
function getScores() {  
    return [70, 80, 90, 100];  
}
```

```
let [x, y ,...args] = getScores();  
console.log(x);    // 70  
console.log(y);    // 80  
console.log(args); // [90, 100]
```

Destructuring: Object Destructuring

- It is yet another destructuring method that allows assigning object properties to variables. Moreover, this method assigns default values to variables if object properties are absent. When we use this method, we must put values in curly brackets, which will help to retrieve object properties.

Example:

```
let person = {  
  firstName: 'John',  
  lastName: 'Doe'  
};  
let { firstName: fname, lastName: lname } = person;  
(OR)  
let { firstName, lastName } = person;
```

ES6 – Modules



Modules

- A module is just a **file**. One script is one module.
- Modules can load each other and use special directives `export` and `import` to interchange functionality, call functions of one module from another one:
 - **export** keyword labels variables and functions that should be accessible from outside the module.
 - **import** allows the import of functionality from other modules.
- An ES6 module is a JavaScript file that executes in **strict mode** only.
- It means that any variables or functions declared in the module won't be added automatically to the global scope.

Executing Modules

message.js:

```
export let message = 'ES6 Modules';
```

app.js:

```
import { message } from './message.js'  
const h1 = document.createElement('h1');  
h1.textContent = message;  
document.body.appendChild(h1)
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="utf-8"> <title>ES6 Modules</title>
```

```
</head>
```

```
<body>
```

```
<script type="module" src="./app.js"></script>
```

```
</body>
```

```
</html>
```

Exporting Modules

- To export a variable, a function, or a class, you place the **export** keyword in front of it.
- Allows you to define a variable, a function, or a class first and then export it later also.

```
log.js :  
export let message = 'Hi';  
export function getMessage() {  
  return message;  
}  
export function setMessage(msg) {  
  message = msg;  
}
```

(OR)

```
log.js :  
let message = 'Hi';  
function getMessage() {  
  return message;  
}  
function setMessage(msg) {  
  message = msg;  
}  
export message, getMessage, setMessage;
```

Importing Modules

- Once you define a module with exports, you can access the exported variables, functions, and classes in another module by using the **import** keyword.

- Syntax:

```
import { what, ever } from './other_module.js';
```

- First, specify what to import inside the curly braces, which are called **bindings**.
- Then, specify the module from which you import the given bindings.
- When you import a binding from a module, the binding behaves like it was defined using **const**. It means you can't have another identifier with the same name or change the value of the binding.

Importing Modules

// greeting.js

export let message = 'Hi';

**export function setMessage(msg) {
 message = msg;
}**

// app.js

**import {message, setMessage } from './greeting.js';
console.log(message); // 'Hi'**

**setMessage('Hello');
console.log(message); // 'Hello'**

message = 'Hallo'; // error – Can't assign value directly

Import Multiple Bindings

cal.js:

```
export let a = 10,  
      b = 20,  
      result = 0;  
  
export function sum() {  
  result = a + b;  
  return result;  
}  
  
export function multiply() {  
  result = a * b;  
  return result;  
}
```

app.js:

```
import {a, b, result, sum, multiply } from './cal.js';  
  
sum();  
console.log(result); // 30  
  
multiply();  
console.log(result); // 200
```

Import Module as an Object / Namespace Import

app.js:

//importing all of app.js

import * as cal from './cal.js';

cal.a;

cal.b;

cal.sum();

app.js:

// importing individual elements

import { a } from './cal.js';

import { b } from './cal.js';

import {result} from './cal.js';

ES6 – Classes



Classes

- A JavaScript class is a blueprint for creating objects. A class encapsulates data and functions that manipulate data.
- Classes are the backbone of Object-oriented programming languages simply because they provide security and structure to codes.
- In fact, in JavaScript a class is **a type of function**.
- To create a class, we need to use the **class** keyword in curly brackets followed by the name of the class.
- Important note is that we need to use the **constructor** method whenever we use a class.
- This method allows assigning properties to objects.

Creating Class

Syntax:

// Creating Class

```
class ClassName {  
    constructor(Parameter List) {  
        // Initialize properties using parameters  
        ...  
    }  
    method1() {  
    }  
    method2() {  
    }  
    ...  
}
```

Example: // Creating Class

```
class Person {  
    constructor(name) {  
        this.name = name;  
    }  
    getName() {  
        return this.name;  
    }  
}  
  
// Creating Object Accessing Class members  
let john = new Person("John Doe");  
let name = john.getName();  
console.log(name); // "John Doe"
```

Getters and Setters

```
/** Using Getters and Setters to manipulate the  
* properties of the class  
*/
```

Example:

```
class Person {  
  constructor(name) {  
    this.setName(name);  
  }  
  getName() {  
    return this.name;  
  }  
  setName(newName) {  
    newName = newName.trim();
```

```
    if (newName === '') {  
      throw 'The name cannot be empty';  
    }  
    this.name = newName;  
  }  
}  
  
let person = new Person('Smith John');  
console.log(person); // Smith John  
  
person.setName('Jane Smith');  
console.log(person.getName()); // Jane Smith
```

Class Expressions

- Similar to functions, classes have expression forms.
- A class expression provides you with an **alternative way** to define a new class.
- A class expression **doesn't require an identifier** after the class keyword.
- And you can use a class expression in a variable declaration and pass it into a function as an argument.

Example:

```
let Person = class {  
    constructor(name) {  
        this.name = name;  
    }  
  
    getName() {  
        return this.name;  
    }  
}  
  
let person = new Person('Annie Jin');
```

Static Methods

- In ES6, you define static methods using the static keyword.
- Static methods are not bounded to the instances rather they are bounded to the classes.
- Therefore, static methods are useful for defining helper or utility methods.

Example:

```
class Person {  
    constructor(name) {  
        this.name = name;  
    }  
    getName() {  
        return this.name;  
    }  
}
```

```
    static createAnonymous(gender) {  
        let name = gender == "male" ? "John  
Doe" : "Jane Doe";  
        return new Person(name);  
    }  
}  
let anonymous = Person.createAnonymous("male");
```

ES6 – Arrow Functions



Arrow Functions

- ES6 arrow functions provide you with an alternative way to write a shorter syntax compared to the function expression.
- Arrow functions eliminate the need to use the return keyword, function keyword, and curly brackets.
- We can use arrow functions with other functions such as filter, reduce, map, etc.
- Note that you must define arrow functions before you use them.

Syntax:

```
let func = (arg1, arg2, ..., argN) => expression;
```

Arrow Functions: Examples

Example - 1:

```
let add = () => 10+20;  
console.log(add()); // 30;
```

Example - 2: // with parameter

```
let add = (x, y) => x + y;  
console.log(add(10, 20)); // 30;
```

Example - 3: // with return value

```
let add = (x, y) => { return x + y; };
```

Example - 4:

// Dynamic creation of functions

```
let age = prompt("What is your age?", 18);
```

```
let welcome = (age < 18) ?
```

```
    () => alert('Hello!') :
```

```
    () => alert("Greetings!");
```

```
welcome();
```

When not to use arrow functions?

- An arrow function doesn't have its own this value and the arguments object.
- Therefore, arrow functions are allowed to use in the following cases:
 - Event Handlers
 - Object Methods
 - Prototype Methods
 - Functions that use the arguments object

ES6 – Iterators & Generators



ES6

Iteration

- ES6 introduces a new mechanism for traversing data: **iteration**.
- Two concepts are central to iteration:
 - **Iterable:**
 - An iterable is a **data structure** that wants to make its elements accessible to the public.
 - It does so by implementing a method whose key is Symbol.iterator.
 - That method is a factory for iterators.
 - **Iterator:**
 - An iterator is a pointer for traversing the elements of a data structure (think cursors in databases)

Iteration Protocols

- There are two iteration protocols: **iterable protocol** and **iterator protocol**.

Iterator protocol

- An object is an iterator when it implements an interface (or API) that answers two questions:
 - Is there any element left? And If there is, what is the element?
- An object is an iterator when it has a `next()` method that returns an object with two properties:
 - **done:** a boolean value indicating whether there are any more elements that could be iterated upon.
 - **value:** the current element.
- Each time you call the `next()`, it returns the next value in the collection

Iteration Protocols

Iterable protocol

- An object is iterable when it contains a method called **[Symbol.iterator]** that takes no argument and returns an object which conforms to the iterator protocol.
- The [Symbol.iterator] is one of the built-in well-known symbols in ES6.

Iterators

- Since ES6 provides built-in iterators for the collection types **Array**, **Set**, and **Map**, you don't have to create iterators for these objects.
- If you have a custom type and want to make it iterable so that you can use the for...of loop construct, you need to **implement the iteration protocols**.

Iterators – Example

Example : // Iterators

```
class Sequence {  
  constructor( start = 0, end = Infinity, interval = 1 ) {  
    this.start = start;  
    this.end = end;  
    this.interval = interval;  
  }  
  [Symbol.iterator]() {  
    let counter = 0;  
    let nextIndex = this.start;  
    return {  
      next: () => {  
        if ( nextIndex <= this.end ) {
```

```
        let result = { value: nextIndex, done: false }  
        nextIndex += this.interval;  
        counter++;  
        return result;  
      }  
    }  
  }  
};  
  
let evenNumbers = new Sequence(2, 10, 2);  
for (const num of evenNumbers) {  
  console.log(num);  
}
```

Generators

- A regular function is executed based on the run-to-completion model. It cannot pause midway and then continues from where it paused.
- ES6 introduces a new kind of function that is different from a regular function: **function generator** or generator.
 - A generator can pause midway and then continues from where it paused.
- You can think of generators as processes (pieces of code) that you can pause and resume while executing that particular code.

Generators – Examples

Example - 1 :

```
function* forever() {  
  let index = 0;  
  while (true) {  
    yield index++;  
  }  
}  
  
let f = forever();  
console.log(f.next()); // 0  
console.log(f.next()); // 1  
console.log(f.next()); // 2
```

Example - 2:

```
class Sequence {  
  constructor( start = 0, end = Infinity, interval = 1 ) {  
    this.start = start;  
    this.end = end;  
    this.interval = interval;  
  }  
  * [Symbol.iterator]() {  
    for( let index = this.start; index <= this.end; index += this.interval ) {  
      yield index;  
    }  
  }  
}  
  
let oddNumbers = new Sequence(1, 10, 2);  
for (const num of oddNumbers) {  
  console.log(num);  
}
```

ES6 – Promises



Promises

- By definition, a **promise** is an object that encapsulates the result of an asynchronous operation.
- A Promise is a proxy for a value not necessarily known when the promise is created.
- It allows you to associate handlers with an asynchronous action's eventual success value or failure reason.

Benefits of Promises

- Improves Code Readability
- Better handling of asynchronous operations
- Better flow of control definition in asynchronous logic
- Better Error Handling

Promise – States

- A Promise has four states:
 - **fulfilled:** Action related to the promise succeeded
 - **rejected:** Action related to the promise failed
 - **pending:** Promise is still pending i.e. not fulfilled or rejected yet
 - **settled:** Promise has fulfilled or rejected

Creating a promise

- To create a promise object, you use the Promise() constructor:

Syntax:

```
const promise = new Promise((resolve, reject) => {  
  // contain an operation  
  // ...  
  // return the state  
  if (success) {  
    resolve(value);  
  } else {  
    reject(error);  
  }  
});
```

Consuming a Promise: then, catch, finally

The then() method:

- then() is invoked when a promise is either resolved or rejected.
- It may also be defined as a carrier which takes data from promise and further executes it successfully.

Syntax:

```
.then(function(result){  
    //handle success  
}, function(error){  
    //handle error  
})
```

Consuming a Promise: then, catch, finally

The catch() method:

- catch() is invoked when a promise is either rejected or some error has occurred in execution. It is used as an Error Handler whenever at any step there is a chance of getting an error.

Syntax:

```
.catch(function(error){  
    //handle error  
})
```

Consuming a Promise: then, catch, finally

The finally() method:

- Sometimes, you want to execute the same piece of code whether the promise is fulfilled or rejected.

Syntax:

```
.finally(() => {  
    render();  
})
```

Consuming a Promise – Example

Example - 1:

```
var promise = new Promise(function(resolve,
reject) {

  const x = "admin";
  const y = "admin"
  if(x === y) {
    resolve();
  } else {
    reject();
  }
});
```

promise.

```
then(function () {

  console.log('Success, You are a valid person');
}).
catch(function () {

  console.log('Some error has occurred');
});
```

Consuming a Promise – Example

Example - 2:

```
var promise = new Promise(function(resolve,
reject) {
    resolve('Resolved');
})
promise
    .then(function(successMessage) {
        //success handler function is invoked
        console.log(successMessage);
    }, function(errorMessage) {
        console.log(errorMessage);
    })
```

Example - 3:

```
var promise = new Promise(function(resolve,
reject) {
    reject('Rejected')
})
promise
    .then(function(successMessage) {
        console.log(successMessage);
    }, function(errorMessage) {
        //error handler function is invoked
        console.log(errorMessage);
    })
```


Promise Chaining

- Used to execute two or more related asynchronous operations, where the next operation starts with the result from the previous step

Example :

```
let p = new Promise((resolve, reject) => {  
    setTimeout(() => {  
        resolve(10);  
    }, 3 * 100);  
});  
p.then((result) => {  
    console.log(result); // 10
```

```
        return result * 2;  
    }).then((result) => {  
        console.log(result); // 20  
        return result * 3;  
    }).then((result) => {  
        console.log(result); // 60  
        return result * 4;  
    });
```

ES6 – Collections



Collections: Map

- ES6 provides a new collection type called Map that addresses the following deficiencies of an object mapping:
 - An object always has a default key like the prototype.
 - A key of an object must be a string or a symbol, you cannot use an object as a key.
 - An object does not have a property that represents the size of the map.
- A Map object holds key-value pairs.
- Keys are unique in a Map's collection.
- Keys and values of a Map can be any values.

Map

- When iterating a Map object, each iteration returns a 2-dimensional array of [key, value].
- The iteration order follows the insertion order which corresponds to the order in which each key-value pair was first inserted into the Map by the set() method.

```
var NewMap = new Map();  
NewMap.set('name', 'John');  
NewMap.set('id', 2345796);  
NewMap.set('interest', ['js', 'ruby', 'python']);  
NewMap.get('name'); // John  
NewMap.get('id'); // 2345796  
NewMap.get('interest'); // ['js', 'ruby', 'python']
```

Some useful methods of Map

- **clear()** – removes all elements from the map object.
- **delete(key)** – removes an element specified by the key. It returns if the element is in the map, or false if it does not.
- **entries()** – returns a new Iterator object that contains an array of [key, value] for each element in the map object. The order of objects in the map is the same as the insertion order.
- **forEach(callback[, thisArg])** – invokes a callback for each key-value pair in the map in the insertion order. The optional thisArg parameter sets the this value for each callback.
-

Some useful methods of Map

- **get(key)** – returns the value associated with the key. If the key does not exist, it returns undefined.
- **has(key)** – returns true if a value associated with the key exists or false otherwise.
- **keys()** – returns a new Iterator that contains the keys for elements in insertion order.
- **set(key, value)** – sets the value for the key in the map object. It returns the map object itself therefore you can chain this method with other methods.
- **values()** – returns a new iterator object that contains values for each element in insertion order.

Collections: SET

- ES6 provides a new type named **Set** that stores a collection of values.
- A set is a collection of items that are **unique** i.e no element can be repeated.
- Set in ES6 are **ordered**: elements of the set can be iterated in the insertion order.
- Set can store **any type** of value whether primitive or objects.

Syntax :

```
let setObject = new Set(iterableObject);
```

Creating Set

Example :

```
// it contains ["sumit","amit","anil","anish"]
```

```
var set1 = new Set(["sumit","sumit","amit","anil","anish"]);
```

```
// it contains 'f', 'o', 'd'
```

```
var set2 = new Set("fooooooooood");
```

```
// it contains [10, 20, 30, 40]
```

```
var set3 = new Set([10, 20, 30, 30, 40, 40]);
```

```
// it is an empty set
```

```
var set4 = new Set();
```


Some useful methods of Set

- **add(value)** – appends a new element with a specified value to the set.
- **clear()** – removes all elements from the Set object.
- **delete(value)** – deletes an element specified by the value.
- **entries()** – returns a new Iterator that contains an array of [value, value] .
- **forEach(callback [, thisArg])** – calls a callback for each element with value sets to thisArg in each call.
- **has(value)** – returns true if an element with a given value is in the set, or false if it is not.
- **keys()** – is the same as values() function.

ES6 Extensions



Extensions – Array, Object and String

- `Array.of()` – improve array creation.
- `Array.from()` – create arrays from array-like or iterable objects.
- `Array find()` – find an element in an array
- `Array findIndex()` – find the index of an element in an array.
- `Object.assign()` – copy an object or merge objects.
- `Object.is()` – check if two values are the same value.
- `String startsWith()` – check if a string starts with another string.
- `String endsWith()` – determine if a string ends with another string.
- `String includes()` – check if a string contains another string.

Asynchronous JS



Asynchronous JS

Synchronous Code

- When we write a program in JavaScript, it executes line by line.
- When a line is completely executed, then and then only does the code move forward to execute the next

line.

```
let greet_salute = "Hello"  
let greet_to= "World!!!"  
console.log(greet_salute)  
for(let i=0;i<1000000000;i++){  
}  
console.log(greet_to);
```

- The execution order of the above code will be as follows:
 - greet_salute logs first. Then it waits for a couple of seconds and then logs greet_to. This is called **Synchronous code**.

Synchronous Code – Backlog

- Synchronous code creates lot of problems.
- For example, if a certain piece of code takes 10 seconds to execute, the code after it won't be able to execute until it's finished, causing delays.
- The reason for this is that this **JavaScript program is single-threaded**. A thread is a sequence of instructions that a program follows.
- Because the program consists of a single thread, it can only **do one thing at a time**: so if it is waiting for our long-running synchronous call to return, it can't do anything else.

Asynchronous JS

Solution

What we need is a way for our program to:

- Start a long-running operation by calling a function.
- Have that function start the operation and return immediately, so that our program can still be responsive to other events.
- Have the function execute the operation in a way that does not block the main thread, for example by starting a new thread.
- Notify us with the result of the operation when it eventually completes.

So, JavaScript offers Asynchronous Programming.

Asynchronous Code

- JavaScript runs code sequentially in top-down order. However, there are some cases that code runs (or must run) after something else happens and also not sequentially. This is asynchronous programming.
- **Asynchronous programming** is a technique that enables your program to start a potentially long-running task and still be able to be responsive to other events while that task runs, rather than having to wait until that task has finished.
- Once that task has finished, your program is presented with the result.
- Example: Event Handlers in JS is really a form of Asynchronous programming.

Asynchronous Function - Example

setTimeout(function, milliseconds)

- function: a function that includes a code block.
- milliseconds: the time it takes for the function to be executed

```
function greet() {  
    console.log("Hello world");  
}  
function sayName(name) {  
    console.log("Hello" + " " + name);  
}  
// calling the function  
setTimeout(greet, 2000);  
sayName("John");
```

Writing Asynchronous JavaScript

- JavaScript can behave in an asynchronous way.
- Let's have a look at some techniques that are useful for asynchronous JavaScript:
 - **Callbacks:** This allows for asynchronous code to be written in a synchronous fashion.
 - **Async / await:** It is a more recent technique for creating asynchronous JavaScript code. It helps you write more succinct and readable code that will execute after a set period of time or when a set condition is met.
 - **Promises:** It enables you to create code that runs after a predetermined period of time or when a predetermined condition is satisfied.

Callback Function

- We can pass objects to functions as parameters.
- In JavaScript, **functions are objects**. So, we can also pass functions as parameters to other functions.
- A **callback** function is a **function passed into another function** as an argument, which is then invoked inside the outer function to complete some kind of routine or action.
- The caller is responsible for passing the right parameters into the callback function.
- The caller may also expect a particular return value from the callback function, which is used to instruct further behavior of the caller.

Callback Function

- There are two ways in which the callback may be called:
 - Synchronous
 - Asynchronous
- **Synchronous** callbacks are **called immediately** after the invocation of the outer function, with no intervening asynchronous tasks.
- **Asynchronous** callbacks are **called at some point later**, after an asynchronous operation has completed.

Synchronous callback – Example

```
function CallBackTeaser(callback, param){  
    console.log(`Calling the callback function`);  
    callback(param);  
    console.log(`callback function completed execution`);  
}  
  
function consoleMyDetails(person){  
    if(person){  
        console.log(`Hi, my name is ${person.name}, i am ${person.job} & super exited about ${person.hobby}`);  
    }  
}  
  
CallBackTeaser(consoleMyDetails, {'name': 'Smith', 'job': 'Developer', 'hobby': 'learning new technologies.'});
```

Calling the callback function

Hi, my name is Smith, i am Developer & super exited about learning new technologies.

callback function completed execution

Asynchronous callback – Example

```
function CallBackTeaser(callback, param) {  
  console.log(`Calling the callback function`);  
  setTimeout(() => { callback(param); }, 0);  
  console.log(`callback function completed execution`);  
}  
  
function consoleMyDetails(person) {  
  if (person) {  
    console.log(`Hi, my name is ${person.name}, i am ${person.job} & super exited about ${person.hobby}`);  
  }  
}  
  
CallBackTeaser(consoleMyDetails, {name: "Smith", job: "Developer", hobby: "learning new technologies."});
```

Calling the callback function

callback function completed execution

Hi, my name is Praveen, i am Developer & super exited about learning new technologies.

Asynchronous - Promise

- Promises are the **foundation of asynchronous programming** in modern JavaScript.
- A promise is an object returned by an asynchronous function, which represents the current state of the operation.
- At the time the promise is returned to the caller, the operation often isn't finished, but the promise object provides methods to handle the eventual success or failure of the operation.
- With a promise-based API, the asynchronous function starts the operation and returns a Promise object.
- You can then attach handlers to this promise object, and these handlers will be executed when the operation has succeeded or failed.

Promise based API call – Example

```
const fetchPromise = fetch( "https://mdn.github.io/learning-  
area/javascript/apis/fetching-data/can-store/products.json",  
);  
  
console.log(fetchPromise);  
  
fetchPromise.then((response) => {  
  console.log(`Received response: ${response.status}`);  
});  
  
console.log("Started request...");
```

```
▼ Promise {<pending>} ⓘ  
  ► [[Prototype]]: Promise  
  ► [[PromiseState]]: "fulfilled"  
  ► [[PromiseResult]]: Response
```

Started request...

Received response: 200

Promise based API call with Chaining and Error Handling

```
const fetchPromise = fetch(
  "bad-scheme://mdn.github.io/learning-
  area/javascript/apis/fetching-data/can-
  store/products.json",
);

fetchPromise
  .then((response) => {
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
  })
```

```
return response.json();
})
.then((data) => {
  console.log(data[0].name);
})
.catch((error) => {
  console.error(`Could not get products: ${error}`);
});
```

Async / Await

- Since JavaScript is a synchronous language, Async functions let us write **promise-based** code as if it were **synchronous**, but without blocking the execution thread that allows the code to run asynchronously.
- Roughly, **async** functions provide **better syntax** for code that uses Promises.
- Adding **async** at the start of a function makes it an async function. It will **always return** a value.
- Using **async** implies that a promise will be returned. If a promise is not returned, JavaScript automatically wraps it in a resolved **promise with its value**.

Syntactic sugar built on top of promises

Async / Await

Function that returns a Promise:

```
function example() {  
    return Promise.resolve("Hello! This is asynchronous  
promise.");  
}  
  
console.log(example());
```

```
▼ Promise {<fulfilled>: 'Hello! This is asynchronous promise.'} ⓘ  
  ► [[Prototype]]: Promise  
    [[PromiseState]]: "fulfilled"  
    [[PromiseResult]]: "Hello! This is asynchronous promise."
```

Async function without using Promise:

```
async function example() {  
    return "Hello! This is asynchronous function.";  
}  
  
console.log(example());
```

```
▼ Promise {<fulfilled>: 'Hello! This is asynchronous function.'} ⓘ  
  ► [[Prototype]]: Promise  
    [[PromiseState]]: "fulfilled"  
    [[PromiseResult]]: "Hello! This is asynchronous function."
```

Async / Await

- **await** is another useful keyword that is often used alongside async functions.
- The keyword await makes JavaScript wait until that promise settles and returns its result.

```
function resolved() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve('resolved');  
    }, 2000);  
  });  
}
```

```
async function example() {  
  console.log('wait for 2 seconds...');  
  const result = await resolved();  
  console.log(result);  
  
  //"resolved"  
}  
  
example();
```

Async / Await

- The **await** keyword must be **used inside an asynchronous** function or a module.

```
const response = await fetch('https://jsonplaceholder.typicode.com/todos/1');  
const json = await response.json();  
console.log(json)
```

✖ Uncaught SyntaxError: await is only valid in async functions and the top level bodies of modules

- This above code will give the error as above. The correct code has given below.

```
async function runProcess() {  
  const response = await fetch('https://jsonplaceholder.typicode.com/todos/1');  
  const json = await response.json();  
  console.log(json)  
}  
runProcess();
```

Async / Await in IIFE & Arrow Functions

IIFE Asynchronous Function Example:

```
(async function () {  
  try {  
    const response = await  
    fetch('https://jsonplaceholder.typicode.com/to  
    dos/1');  
    const json = await response.json();  
    console.log(json);  
  } catch (error) {  
    console.log(error);  
  }  
})();
```

Arrow Asynchronous Function Example:

```
const runProcess = async () => {  
  try {  
    const response = await  
    fetch('https://jsonplaceholder.typicode.com/to  
    dos/1');  
    const json = await response.json();  
    console.log(json);  
  } catch (error) {  
    console.log(error);  
  }  
};  
runProcess();
```

Why Use the `async/await` Syntax?

- The `async/await` syntax enables you to handle promises without using `.then()` and `.catch()` method chaining.
- This benefit is significant when you have a complex process after the promise is settled.
- Consider this given example that uses Promise which has a need of `if...else`

```
fetch('https://jsonplaceholder.typicode.com/todos/1')  
  .then(response => response.json())  
  .then(json => {  
    if (json.userId == 1) {  
      json.completed == false;  
    } else {  
      json.completed == true;  
    }  
  })  
  .catch(error => console.log(error));
```

Why Use the async/await Syntax?

- The previous code is hard to reason with and modify when compared to the async/await version as given.
- By using the async/await syntax, you **reduce** the need for **method chaining and nested callbacks**.
- This impact the readability of your code, especially when you have nested code like if/else and a for loop block.

```
(async function () {  
  try {  
    const response = await  
    fetch('https://jsonplaceholder.typicode.com/todos/1');  
    const json = await response.json();  
    console.log(json);  
  } catch (error) {  
    console.log(error);  
  }  
})();
```


Exception Handling



Introduction

- Murphy's law states that "**Anything that can go wrong will go wrong.**" This applies in the programming world too.
- If you create an application, chances are you'll create bugs and other issues. **Errors** in JavaScript are one such common issue!
- It can happen when a program doesn't know how to handle the job at hand, such as when trying to open a non-existent file or reaching out to a web-based API endpoint while there's no network connectivity.
- The program collects as much information as possible about the error and then reports that it can not move ahead.

Introduction

- Intelligent programmers try to predict and cover these scenarios so that the user doesn't have to figure out a technical error message like "404" independently.
- Instead, they show a much more understandable message: "The page could not be found."
- **Exception handling** is a crucial aspect of writing robust and maintainable JavaScript code.
- It allows developers to **manage errors gracefully**, ensuring that the application continues to function, even when unexpected issues arise.
- When a JavaScript statement results in an error, it's said to **throw an exception**. JavaScript **creates** and **throws an Error object** describing the error.

Error Object

- In JavaScript, errors are represented by the **Error object** or one of its **subclasses**.
- The Error object is a built-in object that provides a standard set of properties
- These standard and custom properties of Error helps to understand the cause and effects of the error.
- By default, errors in JavaScript contain three properties:
 - **message**: A string value that carries the error message
 - **name**: The type of error that occurred
 - **stack**: The stack trace of the code executed when the error occurred.

Exception Handling

Error Object

- Additionally, errors can also carry properties like `columnNumber`, `lineNumber`, `fileName`, etc., to describe the error better.
- However, these properties are not standard and may or may not be present in every error object generated from your JavaScript application.

```
const customError = new Error('Something went wrong');  
  
console.log(customError.name);           // "Error"  
  
console.log(customError.message);        // "Something went wrong"
```

Stack Trace

- A stack trace is the list of method calls a program was in when an event such as an exception or a warning occurs.
- This is what a sample stack trace accompanied by an exception looks like:

```
TypeError: Numeric argument is expected
    at Timeout.setTimeout [as _onTimeout] (/tmp/rWzN9KhDTV.js:5:10)
    at ontimeout (timers.js:436:11)
    at tryOnTimeout (timers.js:300:5)
    at listOnTimeout (timers.js:263:5)
    at Timer.processTimers (timers.js:223:10)
```

Built-in Error Types

- **Syntax Errors:** Syntax errors occur due to incorrect code structure, such as missing semicolons or mismatched parentheses. These are typically caught by the JavaScript interpreter at compile time.
- **Reference Errors:** Reference errors occur when a developer attempts to access a variable or object that doesn't exist in the current scope. This can lead to a reference to an undefined variable or property.
- **Type Errors:** Type errors arise when there's an attempt to perform an operation on a data type that's incompatible with the operation. An example is when you are trying to call a non-existent function or access properties of non-objects.

Built-in Error Types

- **Range Errors:** Range errors happen when an operation is performed outside the valid range, often encountered with array manipulation, like accessing an index that doesn't exist.
- **Runtime Errors:** Runtime errors occur during the execution of code and are often caused by logical issues or unexpected conditions in the program.
- **Network Errors:** These errors relate to network-related operations, such as failing to fetch resources from external servers or encountering issues with AJAX requests.
- **Custom Errors:** Developers can create custom errors to handle application-specific issues or exceptions in a more controlled manner.

Handling Errors – Fundamentals

- The most fundamental way of handling errors that have been thrown either manually or by the runtime is to catch them.
- Like most other languages, JavaScript offers a set of keywords to handle errors.
- **throw:**
 - The first and most basic keyword is throw.
 - As evident, the throw keyword is used to throw errors to create exceptions in the JavaScript runtime manually.

```
throw TypeError("Please provide a string")
```

Handling Errors – Fundamentals

- The 'try-catch' statement is a fundamental exception handling mechanism in JavaScript.
- It allows you to execute a block of code and catch any exceptions that may occur during its execution.
- If an exception is thrown within the 'try' block, the code in the 'catch' block will be executed.
- The error parameter contains information about the exception.
- It's important to note that a catch block must always follow the try block to handle errors effectively.

```
try {  
    // business logic code  
} catch (exception) {  
    // error handling code  
}
```

Exception Handling

try, catch, throw – Example

```
function divide(a, b) {  
  if (b === 0) {  
    throw new Error('Division by zero is not allowed');  
  }  
  return a / b;  
}  
  
try {  
  const result = divide(10, 0);  
  console.log(result);  
} catch (error) {  
  console.log(`An error occurred: ${error.message}`);  
}
```

The finally Block

- The finally block is an optional part of the try-catch statement that is always executed, regardless of whether an exception was thrown or caught.
- It is useful for performing cleanup tasks, such as closing resources or releasing memory.

```
try {  
    // Code that might throw an exception  
} catch (error) {  
    // Handle the exception  
} finally {  
    // Code to always execute, regardless of whether an exception occurred  
}
```

Exception Handling

finally – Example

```
let isOpen = false;

function openResource() {
  isOpen = true;
  console.log('Resource opened');
}

function closeResource() {
  isOpen = false;
  console.log('Resource closed');
}
```

```
try {
  openResource();

  // Code that might throw an exception
} catch (error) {
  // Handle the exception
} finally {
  closeResource();
}
```

Error Events

Global Error Handlers:

- When an error is not caught within a try-catch block, it propagates to the global level, triggering the error event.
- You can attach an event listener to the **window** object to handle global errors:

```
window.addEventListener('error', (event) => {  
    console.log(`An unhandled error occurred: ${event.message}`);  
});
```

- This technique is useful for logging unhandled errors or notifying users about issues.

Error Events

Unhandled Promise Rejections:

- Unhandled promise rejections trigger the **unhandledrejection** event.
- You can listen for this event to handle unhandled promise rejections:

```
window.addEventListener('unhandledrejection', (event) => {  
    console.log(`An unhandled promise rejection occurred: ${event.reason}`);  
});
```

Error Handling with Promises

Catching Errors with Promises:

- Promises provide a convenient way to handle asynchronous operations and errors.
- The `catch()` method allows you to handle errors within a promise chain:

```
fetch('https://api.example.com/data')  
  
  .then((response) => response.json())  
  
  .then((data) => console.log(data))  
  
  .catch((error) => console.log(`An error occurred: ${error.message}`));
```


Error Handling with async/await

Handling Errors with async/await:

- The async/await syntax allows you to handle errors in a more synchronous-like manner.
- You can use **try-catch** blocks within an async function to handle errors:

```
async function fetchData() {  
  try {  
    const response = await  
    fetch('https://api.example.com/data');  
    const data = await response.json();  
    console.log(data);  
  }
```

```
  } catch (error) {  
    console.log(`An error occurred:  
    ${error.message}`);  
  }  
}  
  
fetchData();
```

Exception Handling

Custom Error Handling

- To create more specific error types, you can extend the built-in Error class.
- This allows us to create custom error types with their own names and messages, making it easier to handle specific error cases.

```
// Creating Error of our own
class CustomError extends Error {
  constructor(message) {
    super(message);

    this.name = 'CustomError';
  }
}
```

```
// Using the created Error
try {
  throw new CustomError('This is a custom
error');
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}
```

Custom Error Handling – Example

```
class ValidationError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = 'ValidationError';  
  }  
}  
  
class EmailValidationError extends ValidationError {  
  constructor(message) {  
    super(message);  
    this.name = 'EmailValidationError';  
  }  
}
```

Custom Error Handling – Example

```
class PasswordValidationError extends ValidationError {  
  constructor(message) {  
    super(message);  
    this.name = 'PasswordValidationError';  
  }  
}  
  
function validateUserInput(email, password) {  
  if (!email.includes('@')) {  
    throw new EmailValidationError('Invalid email format');  
  }  
}
```

Custom Error Handling – Example

```
if (password.length < 8) {  
    throw new PasswordValidationError('Password must be at least 8 characters long');  
}  
return true;  
}  
try {  
    validateUserInput('invalidemail.com', 'short');  
} catch (error) {  
    if (error instanceof ValidationError) {  
        console.log(`Validation error: ${error.message}`);  
    } else {  
        console.log(`Unknown error: ${error.message}`);  
    }  
}
```

Javascript

Quiz

1. What is the correct JavaScript syntax to change the content of the HTML element below?

`<p id="demo">This is a demonstration.</p>`

a) `document.getElementById("demo").innerHTML="Hai"`

b) `document.getElement("p").innerHTML="Hai"`

c) `#demo.innerHTML="Hai"`

d) `document.getElement("p").innerHTML="Hai"`

a) `document.getElementById("demo").innerHTML="Hai"`



Javascript

Quiz

2. Which event occurs when the user clicks on an HTML element?



a) onclick

b) onchange

c) onmouseclick

d) onmouseover

a) onclick

Javascript

Quiz

3. To which object does the location property belong?



a) Window

b) Position

c) Location

d) None of these

a) Window

Javascript

Quiz

4. The statement `a===b` refers to _____



a) Both a and b are equal in value, type and reference address

b) Both a and b are equal in value

c) Both a and b are equal in value and type

d) There is no such statement

c) Both a and b are equal in value and type

Javascript

Quiz

5. What will be returned from following JavaScript code?



```
<p id="demo"></p>
<script>
function myFunction()
{
    document.getElementById("demo").innerHTML=
    Math.abs(-7.25);
}
</script>
```

a) -7.25

b) 7

c) 7.25

d) -7

c) 7.25

Javascript

Quiz

6. The HTML canvas is a



a) Three-dimensional grid

b) Two-dimensional grid

c) One-dimensional grid

d) None of the above

b) Two-dimensional grid

Javascript

Quiz

7. Which of the following element is used for canvas graphics?



a) `<graphic>`

b) `<canvas>`

c) `<css>`

d) `<paint>`

b) `<canvas>`

Javascript

Quiz

8. To draw text on a canvas, which property and method is used.



a) font

b) fillText(text,x,y)

c) strokeText(text,x,y)

d) Both A & B

c) strokeText(text,x,y)

THANK YOU